

Lecture Notes on Machine Learning

Sébastien DA VEIGA
ENSAI - CREST

2023–2024

Abstract

These lecture notes have been prepared for third-year students at ENSAI following the Machine Learning course. They consist of seven 3-hours lectures, dedicated to the following topics:

- Reminder on statistical learning, risk and model validation / selection
- Variants of Ordinary Least Squares (PCA-OLS, Ridge, Lasso)
- Optimization of general parametric models
- Non-parametric models for classification with a focus on Support Vector Machines
- Non-parametric models for regression with a focus on linear smoothers, splines and kernel methods
- Ensemble learning
- Practical considerations

Contents

1	Reminder on statistical learning	4
1.1	Problem setting and concept of risk	4
1.2	Resampling methods	10
1.2.1	Hold-out validation	10
1.2.2	Cross-validation	11
1.2.3	Monte-Carlo validation	12
1.2.4	Additional remarks on resampling methods	13
1.3	Concluding remarks	13
2	Variants of Ordinary Least-Squares	14
2.1	Reminder on OLS	14
2.1.1	Definition of the OLS estimator	14
2.1.2	Statistical Properties of OLS estimation	16
2.2	Ridge regularization	18
2.2.1	Definition of the ridge estimator	19
2.2.2	Statistical properties of the ridge estimator	22
2.2.3	Concluding remarks on the ridge estimator	25
2.3	Dimension reduction for linear regression	25
2.3.1	Preliminary dimension reduction with unsupervised methods	25
2.3.2	Lasso regularization	26
2.4	Concluding remarks and introduction to the next class	32
3	Optimization for machine learning	34
3.1	Introduction & motivation	34
3.2	Gradient descent	35
3.2.1	Convex functions	36
3.2.2	Analysis of gradient descent	38
3.2.3	Variants of gradient descent	39
3.3	Stochastic gradient descent	42
3.4	Concluding remarks	44
4	Reminder on classification	48
4.1	Problem setting and notations	48
4.2	Linear classifiers	50

5	Support Vector Machines	54
5.1	Linearly separable case	54
5.2	Non-linearly separable case	58
5.3	Non-linear extension with kernels	62
5.4	The kernel jungle / The kernel zoo	66
5.5	Concluding notes	68
5.5.1	Choice of hyperparameters	68
5.5.2	Extension to multi-class classification	68
6	Reminder on regression	70
6.1	Problem settings and notations	70
6.2	Local averaging techniques	71
6.3	Splines	74
7	Reproducing Kernel Hilbert Spaces (RKHS)	78
7.1	Definitions and notations	78
7.2	RKHS properties for machine learning	80
7.2.1	Representer theorem	80
7.2.2	Feature maps	83
7.2.3	Smoothing	84
7.3	Building kernels	84
7.4	Final remarks	85
8	Ensemble learning	87
8.1	Bagging	87
8.1.1	General principle	87
8.1.2	Random Forests	90
8.2	Boosting	94
8.3	Stacking	98
9	Practical machine learning	101
9.1	Handling categorical features	102
9.1.1	Impact on the models	102
9.1.2	General strategy: Encoding	102
9.1.3	Handling rare categories	104
9.2	Large datasets	104
9.3	Handling high dimensionality of explanatory variables	104
9.3.1	Motivation and context	104
9.3.2	Feature selection	105
9.3.3	Projection / Embedding	106
9.4	Normalization and preprocessing of variables	108
9.4.1	Impact on models	108
9.4.2	Transformations	109
9.5	Handling missing data	109
9.5.1	General overview	109
9.5.2	Handling missing values	110
9.6	Hyperparameter optimization	112
9.7	Visualize, Visualize, Visualize	112
9.8	Bonus 1: Handling more "exotic" explanatory variables	114
9.9	Bonus 2: Semi-supervised learning	117

Lecture 1: Ordinary Least Squares variants

September 11, 2023

- Reminder on statistical learning
 - Risk
 - Model validation
 - Ordinary Least Squares
- Ordinary Least Squares variants
 - Ridge regularization
 - PCA-OLS
 - Lasso regularization
 - Elastic-Net
 - Concluding notes

Chapter 1

Reminder on statistical learning

1.1 Problem setting and concept of risk

In supervised statistical learning, we are given a sample $\mathcal{D}_n = (\mathbf{x}_i, y_i)_{i=1, \dots, n}$ of size n from an unknown joint distribution $\mathbb{P}_{\mathbf{X}Y}$ defined on a product space $\mathcal{X} \times \mathcal{Y}$. Here $\mathbf{X} = (X^{(1)}, \dots, X^{(d)}) \in \mathcal{X} \subset \mathbb{R}^d$ is the vector of the explanatory variables or inputs or *features* and $Y \in \mathcal{Y}$ is the *target* variable, or output. The goal is to build a map $g : \mathcal{X} \mapsto \mathcal{Y}$, called a *predictor* or *prediction function* or *prediction rule*, from the observations in the sample $\mathcal{D}_n$ and which predicts the value of the target variable $y = g(\mathbf{x})$ of any other individual with features $\mathbf{x}$.

When the target variable Y lives in a space $\mathcal{Y} = \{C_1, \dots, C_M\}$ where the C_j , $j = 1, \dots, M$ are M categories, the supervised learning problem is referred to as *classification*, while when $\mathcal{Y} \subset \mathbb{R}$ this is a *regression* problem. In classification we will use the notation $h : \mathbb{R}^d \mapsto \{C_1, \dots, C_M\}$ for the prediction function and $f : \mathbb{R}^d \mapsto \mathbb{R}$ in regression, leaving the notation g for a predictor in a general setting.

In order to estimate a prediction rule g from the sample $\mathcal{D}_n$, we first need a central building block which is able to compare two values in the output space $\mathcal{Y}$:

Definition 1 (Loss function). A measurable function $l : \mathcal{Y} \times \mathcal{Y} \mapsto \mathbb{R}_+$ is a **loss function** if $l(y, y) = 0$ and $l(y, y') > 0$ for $y \neq y'$.

In particular, if for given features $\mathbf{x}$ the associated true output value is y , then $l(y, g(\mathbf{x}))$ measures the loss incurred when the rule g predicts $g(\mathbf{x})$ for $\mathbf{x}$. Such a definition leaves a lot of flexibility in the choice of the loss function, but specific choices are very frequently used in practice:

- In **regression**, we define the L^p ($p \geq 1$) loss functions

$$l(y, y') = |y - y'|^p.$$

For $p = 1$, this is the absolute loss and for $p = 2$ this is the quadratic loss

$$l(y, y') = (y - y')^2.$$

- In **classification**, the 0-1 loss is given by

$$l(y, y') = \mathbb{1}_{y \neq y'}.$$

But we will see later in this course that for classification other choices are popular. Now that we have a loss function, it allows us to compute the error made by a predictor for a given value of $(\mathbf{x}, y)$. The final quality of the predictor is then obtained by calculating the average behavior of the loss, averaged in the sense that $(\mathbf{x}, y)$ follows the underlying distribution $\mathbb{P}_{\mathbf{X}Y}$ of the problem. This average behavior is called the risk or the generalization error.

Definition 2 (Risk). Given a loss function l and a joint probability distribution $\mathbb{P}_{\mathbf{X}Y}$ for $(\mathbf{X}, Y)$, the **risk** of a predictor g is defined as

$$R_{\mathbb{P}_{\mathbf{X}Y}}(g) = \mathbb{E}_{(\mathbf{X}, Y) \sim \mathbb{P}_{\mathbf{X}Y}}[l(Y, g(\mathbf{X}))].$$

It is also referred to as the theoretical risk.

The predictor g^* which minimizes the risk among a set of functions $\mathcal{G}$ is called the **oracle predictor** and is given by

$$g^* = \arg \min_{g \in \mathcal{G}} R_{\mathbb{P}_{\mathbf{X}Y}}(g).$$

Example (Bayes classifier). Last year you saw that in binary classification for the 0-1 loss, this oracle predictor is the **Bayes classifier** when $\mathcal{G}$ consists of all measurable functions.

Now in practice, if we want to compare several predictors according to their risk, or if we want to identify the best one, the theoretical risk is out of reach since we do not know the joint distribution $\mathbb{P}_{\mathbf{X}Y}$: we only observe a random sample $(\mathbf{x}_i, y_i)_{i=1, \dots, n}$ from $\mathbb{P}_{\mathbf{X}Y}$. This is why we often consider instead the *empirical risk* defined as an empirical mean on the random sample instead of the expectation.

Definition 3 (Empirical risk). Given a loss function l and an n -sample $\mathcal{D}_n = (\mathbf{x}_i, y_i)_{i=1, \dots, n}$, the **empirical risk** of a predictor g is defined as

$$\hat{R}_{\mathcal{D}_n}(g) = \frac{1}{n} \sum_{i=1}^n l(y_i, g(\mathbf{x}_i)).$$

The empirical risk serves as a proxy of the theoretical risk, and is often used to identify the predictor $\hat{g}_{\text{ERM}}$ which minimizes the empirical risk among $\mathcal{G}$, a set of candidate functions. This is the *empirical risk minimization* (ERM):

Definition 4 (Empirical risk minimization). The empirical risk minimizer $\hat{g}_{\text{ERM}}$ is the minimizer

$$\hat{g}_{\text{ERM}} = \arg \min_{g \in \mathcal{G}} \hat{R}_{\mathcal{D}_n}(g)$$

over a set of functions $\mathcal{G}$.

Example (Linear regression). If we use the loss function $l(y, y') = (y - y')^2$ and limit the choice of g to the set of linear functions

$$\mathcal{G} = \left\{ g : g(\mathbf{x}) = \theta_0 + \sum_{j=1}^d \theta_j x^{(j)}, \theta_j \in \mathbb{R}, j = 0 \dots, d \right\},$$

we recover the standard ordinary least squares problem

$$\arg \min_{\theta_0, \dots, \theta_d} \frac{1}{n} \|Y - X\boldsymbol{\beta}\|_2^2$$

where $Y = (y_1, \dots, y_n)$, $\boldsymbol{\beta} = (\theta_0, \dots, \theta_d)$ and $[X]_{ij} = x_i^{(j)}$ with $x^{(0)} := 1$.

When $\mathcal{G} = \{g_1, \dots, g_M\}$ is a set of finite cardinality M , referred to in this case as a *finite dictionary*, we have the following inequality.

Theorem 1 (Oracle inequality for finite dictionary). Suppose that the loss function l is valued in $[0, 1]$ and that $\mathcal{G} = \{g_1, \dots, g_M\}$. Then for all $\delta \in (0, 1)$ it holds with probability $1 - \delta$

$$R_{\mathbb{P}_{\mathbf{X}Y}}(\hat{g}_{\text{ERM}}) - R_{\mathbb{P}_{\mathbf{X}Y}}(g^*) \leq \sqrt{\frac{2}{n} \log(2M/\delta)}.$$

If we take a close look at this inequality, we can point out two rather illuminating facts:

- **Fact 1:** When $n \rightarrow \infty$, the risk of the empirical risk minimizer approaches the risk of the oracle, meaning that when the number of data increases the accuracy of the approximate predictor $\hat{g}_{\text{ERM}}$ is close to the optimal one,
- **Fact 2:** When the set of candidate functions we consider is small, the gap between the risks is also smaller.

Fact 1 formalizes the intuition that the more data we have to build a predictor, the more accurate it will be. On the other hand Fact 2 necessitates an additional discussion. Indeed at first sight we may tend to conclude that it is better to choose a dictionary with very few candidate functions because it implies that we will be close to the optimal risk. However it should be kept in mind that the smaller M , the higher the optimal risk: this is very intuitive since in this case we do not give a lot of degrees of freedom to our predictor. Mathematically speaking, this is easily seen by the fact that if we consider a sequence

of nested sets $\mathcal{G}_1 \subset \mathcal{G}_2 \subset \dots \subset \mathcal{G}_m$ of increasing cardinality, it is obvious that

$$\min_{g \in \mathcal{G}_1} R_{\mathbb{P}_{\mathbf{X}Y}}(g) \geq \min_{g \in \mathcal{G}_2} R_{\mathbb{P}_{\mathbf{X}Y}}(g) \geq \dots \geq \min_{g \in \mathcal{G}_m} R_{\mathbb{P}_{\mathbf{X}Y}}(g).$$

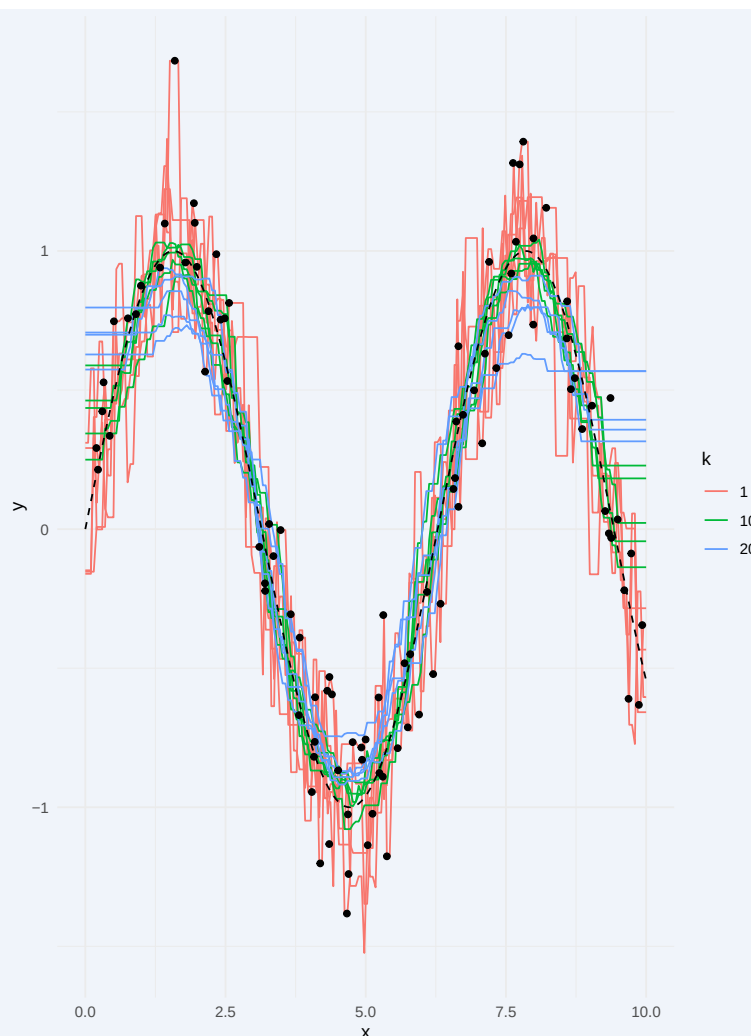
We will use this remark again just below, so keep it in mind!

Another point worth mentioning is that you may think of this inequality as a justification for considering more complex models (and thus increasing the size of the set of candidate functions $\mathcal{G}$) such that the risk is small. But in the oracle inequality the gap between the approximate predictor and the ideal one will increase! In other words, you are facing the famous **bias-variance** trade-off:

- If $\mathcal{G}$ increases the bias diminishes (modeling error, gap due to model choice)
- But if $\mathcal{G}$ increases, the variance increases (statistical error, variability due to randomness in data)

In such a case we will say that the model *overfits* the data, a phenomenon referred to as **overfitting**. On the contrary, when $\mathcal{G}$ decreases, the bias increases and the variance diminishes: this is **underfitting**. We give an example below.

Example. On a one-dimensional regression problem, we train several times a nearest-neighbor predictor on five n -samples generated independently, and we compare the results for $k = 1, 10, 20$ nearest neighbors. The true regression function which was used to generate the datasets is represented with a dashed black line, and one of the n -samples is displayed with black points.



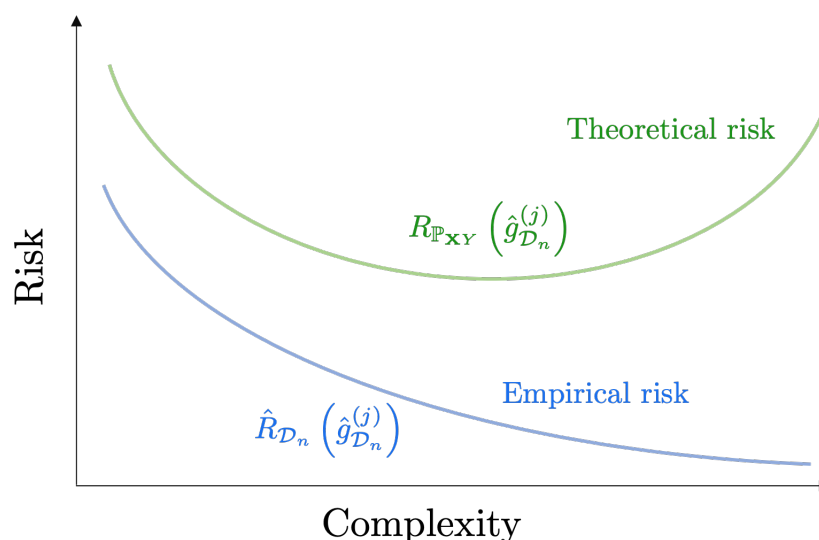
We observe that for $k = 1$ in red, in average the predictor is unbiased but exhibits a large variability when re-trained several times. At the opposite for $k = 20$ the result is stable but we see a systematic estimation bias. $k = 10$ seems to be a good compromise in this example.

In order to be able to select which model complexity would be the best for a given problem, it is then natural to train several predictors $\hat{g}_{\mathcal{D}_n}^{(1)}, \dots, \hat{g}_{\mathcal{D}_n}^{(m)}$ with increasing complexity and search for the one which, after training, performs the best in terms of theoretical risk (think for example about identifying the best number of nearest neighbors for a nearest-neighbor predictor, the depth of the tree for the CART algorithm or selecting only a subset of variables in linear regression). But in practice this is out of reach since we cannot compute $R_{\mathbb{P}_{\mathbf{X} \times \mathbf{Y}}}(g)$ for any g as already mentioned. A straightforward idea is to use instead the empirical risk as a proxy to compare such predictors. But if we use

the previous remark we immediately have that their empirical risks satisfy

$$\hat{R}_{\mathcal{D}_n}(\hat{g}_{\mathcal{D}_n}^{(1)}) \geq \dots \geq \hat{R}_{\mathcal{D}_n}(\hat{g}_{\mathcal{D}_n}^{(m)}),$$

which means that with the empirical risk as a way to compare them, you will always select the most complex predictor ($k = 1$ nearest neighbors, trees of maximum depth or linear regression with all variables if we come back to the previous examples). Graphically, we can illustrate this situation with a display you have seen last year.



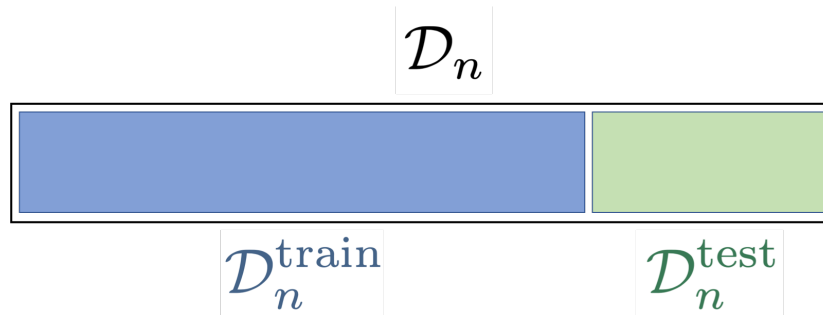
Here we also give a "virtual" representation of what the (inaccessible) theoretical risk would look like. If you recall, this phenomenon is due to the fact that the empirical risk is computed with the exact same sample that was used to estimate the predictors $\hat{g}_{\mathcal{D}_n}^{(j)}$. This means in particular that the empirical risk, if calculated on the same dataset:

1. Cannot be used to properly evaluate the accuracy of any predictor (*model validation*)
2. Cannot be used to tune the hyperparameters of a predictor (*model selection*)

In practice, model validation in 1. is usually performed by splitting the initial dataset in two parts:

$$\mathcal{D}_n = \mathcal{D}_n^{\text{train}} \cup \mathcal{D}_n^{\text{test}}$$

where $\mathcal{D}_n^{\text{train}}$ is the **training set** used to train the predictor $\hat{g}_{\mathcal{D}_n^{\text{train}}}$, and $\mathcal{D}_n^{\text{test}}$ is the **test set** used to evaluate its accuracy with the risk $\hat{R}_{\mathcal{D}_n^{\text{test}}}(\hat{g}_{\mathcal{D}_n^{\text{train}}})$ as an estimation of $R_{\mathbb{P}_{\mathbf{X}Y}}(\hat{g}_{\mathcal{D}_n^{\text{train}}})$.



As for model selection in 2., there are mainly two paths:

- **Estimation by regularization.** This is the historical approach in statistics, based on the idea that models are compared with the empirical risk on the training set $\hat{R}_{\mathcal{D}_n^{\text{train}}}(\hat{g}_{\mathcal{D}_n^{\text{train}}})$ but are **penalized** by their complexity. The final criterion writes

$$\hat{R}_{\mathcal{D}_n^{\text{train}}}(\hat{g}_{\mathcal{D}_n^{\text{train}}}) + P(\hat{g}_{\mathcal{D}_n^{\text{train}}})$$

where P is a penalty or regularization function which is increasing with respect to the complexity. This will lead to a compromise between empirical risk and complexity which, from an intuitive point of view, tries to transform the blue curve into the green one in the previous visualization. In the context of linear regression you have already seen such a criterion with for example the AIC or BIC penalties, and we will discuss in the second part of this class other popular ones called the Ridge or the Lasso penalties.

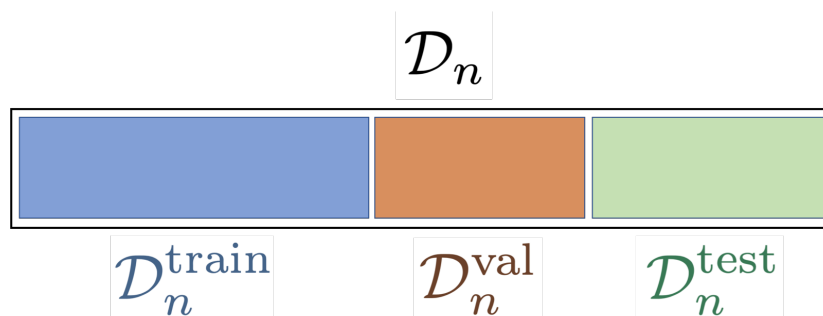
- **Estimation by resampling.** The idea here is to extend the training/test split introduced above for model selection. Since this is now a mainstream method which is applicable in a general setting (as opposed to the first one which is limited to parametric predictors), we dedicate the next section to summarize some of the most used approaches in this category.

1.2 Resampling methods

In this class of methods there are mainly three techniques.

1.2.1 Hold-out validation

The underlying principle is a straightforward extension of the trick used to build $\mathcal{D}_n^{\text{test}}$: we simply define another independent dataset $\mathcal{D}_n^{\text{val}}$ dedicated only to the selection of the best hyperparameters, which is ultimately validated on $\mathcal{D}_n^{\text{test}}$, see below for an algorithmic description of the procedure.



Algorithm 1 Hold-out Validation

For a given family of predictors (e.g., knn).

Step 1: Train the predictor on $\mathcal{D}_n^{\text{train}}$ for each set of hyperparameters, obtaining the collection

$$\hat{g}_{\mathcal{D}_n^{\text{train}}}^{(1)}, \dots, \hat{g}_{\mathcal{D}_n^{\text{train}}}^{(m)}$$

Step 2: Calculate the risk of each of them on the validation sample

$$\hat{R}_{\mathcal{D}_n^{\text{val}}}(\hat{g}_{\mathcal{D}_n^{\text{train}}}^{(1)}), \dots, \hat{R}_{\mathcal{D}_n^{\text{val}}}(\hat{g}_{\mathcal{D}_n^{\text{train}}}^{(m)})$$

Step 3: Choose the best among them

$$j^* = \arg \min_j \hat{R}_{\mathcal{D}_n^{\text{val}}}(\hat{g}_{\mathcal{D}_n^{\text{train}}}^{(j)})$$

which is denoted as

$$\hat{g}^{(\text{final})} = \hat{g}_{\mathcal{D}_n^{\text{train}}}^{(j^*)}$$

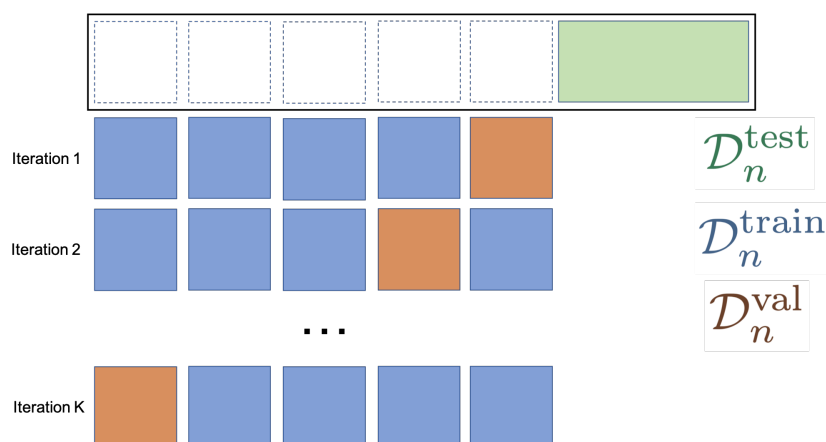
Step 4: The quality of the final predictor is determined by its risk on the test sample

$$\hat{R}_{\mathcal{D}_n^{\text{test}}}(\hat{g}^{(\text{final})})$$

The choice of the best final predictor in step 3 can be quite dependent on the splitting we have done to construct $\mathcal{D}_n^{\text{val}}$: the most commonly employed solution currently is therefore the cross-validation described below.

1.2.2 Cross-validation

While always keeping $\mathcal{D}_n^{\text{test}}$ aside, in cross-validation, we will repeat the splitting of $\mathcal{D}_n^{\text{train}} / \mathcal{D}_n^{\text{val}}$ multiple times. If we split it K times, it is called **K-fold cross-validation**.

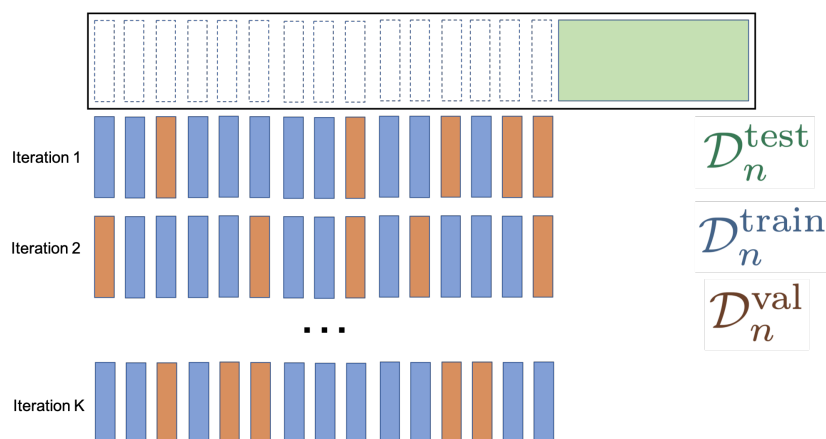


In practice, in the previous simple validation algorithm, we put steps 1 and 2 in a loop over the K -fold splits, and in step 3, we choose the best predictor (or rather, the best hyperparameters) based on the average of the risks on each of the $\mathcal{D}_n^{\text{val}}$ from the K -folds. Typically, one chooses $K = 10$ or $K = 5$. The extreme case, $K = |\mathcal{D}_n^{\text{train}}|$, is called "leave-one-out".

With this procedure, the predictor for which we identified the hyperparameters as the best has only been trained on a subset of the data, due to the K -fold-based procedure. In practice, we then perform a final step called **re-calibration**: the final predictor is obtained by training on all the data (except the test set, of course) with the optimal hyperparameters identified by cross-validation.

1.2.3 Monte-Carlo validation

The drawback of K -folds is that the same block of observations does not change, meaning a set of observations is always considered together. The idea of Monte-Carlo is to 'break' these sets, with a sufficiently large number of repetitions K , on the order of hundreds:



The Monte-Carlo approach avoids potential bias induced by these sets, but

it disrupts the balance of the K-folds approach because some data can be used more for training than for validation. Finally, Bootstrap cross-validation is similar to Monte-Carlo, with the only difference being that samples are chosen with replacement instead of without replacement. This means that some observations may end up in both $\mathcal{D}_n^{\text{train}}$ and $\mathcal{D}_n^{\text{val}}$. This introduces an optimistic bias that can be corrected, for example, with "leave-one-out Bootstrap 0.632".

As with cross-validation, the final predictor must be constructed with a re-calibration step.

1.2.4 Additional remarks on resampling methods

- For a given family of models (for example, knn/polynomials/trees/...), one of the previously described validation procedures is used to choose the best hyperparameters. For each family, we thus have an "optimized" model, and its quality is calculated using the risk on the $\mathcal{D}_n^{\text{test}}$ sample. Your final choice will be the **best of the best** models, selecting the one with the highest quality on the $\mathcal{D}_n^{\text{test}}$ sample.
- If you have no hyperparameters to optimize, you can simply perform the single split of $\mathcal{D}_n^{\text{train}}$ / $\mathcal{D}_n^{\text{test}}$; no need for validation!
- In cases where there are no hyperparameters but you have a dataset with few observations, even a single split can potentially cause a significant loss of information. One solution is to use a cross-validation procedure to replace $\mathcal{D}_n^{\text{test}}$.
- Finally, in the most challenging situation where you have limited data and need to optimize hyperparameters, one possibility is to use "nested cross-validation".

1.3 Concluding remarks

The concepts we just discussed are very close to the material discussed in your supervised learning class last year. In this machine learning course, we will elaborate on more some specific aspects, in particular:

- Optimization, since it is at the heart of the resolution of an empirical risk minimization
- New loss functions for classification
- Regularization, as a way to penalize the complexity of a parametric predictor
- New classes of non-parametric predictors

Chapter 2

Variants of Ordinary Least-Squares

In this class we will introduce (or re-introduce) variants of linear regression with quadratic loss, known as *Ordinary Least-Squares* (OLS in the following), but before diving into the details we will first briefly recap the standard OLS setting.

2.1 Reminder on OLS

2.1.1 Definition of the OLS estimator

We consider the following setting:

- $n \geq 1$ observations and $d \geq 1$ covariates.
- For each $i = 1, \dots, n$, we have $y_i \in \mathbb{R}$ and $\mathbf{z}_i \in \mathbb{R}^{d+1}$ such that

$$\mathbf{z}_i = (x_{i,0}, \dots, x_{i,d})^T = (x_{i,0}, \mathbf{x}_i^T)$$

with $x_{i,0} = 1$ (to include the intercept in the same way as the other coefficients).

Definition 5 (OLS). The **OLS estimator** is any coefficient vector $\hat{\boldsymbol{\theta}}_n = (\hat{\theta}_{n,0}, \dots, \hat{\theta}_{n,d})^T \in \mathbb{R}^{d+1}$ such that

$$\hat{\boldsymbol{\theta}}_n \in \arg \min_{\boldsymbol{\theta} \in \mathbb{R}^{d+1}} \sum_{i=1}^n (y_i - \mathbf{z}_i^T \boldsymbol{\theta})^2.$$

In matrix notation, if we define

$$X = \begin{pmatrix} \mathbf{x}_1^T \\ \vdots \\ \mathbf{x}_n^T \end{pmatrix} \in \mathbb{R}^{n \times d}, \quad Z = \begin{pmatrix} \mathbf{z}_1^T \\ \vdots \\ \mathbf{z}_n^T \end{pmatrix},$$

$$Y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}, \quad \mathbf{1}_n = \begin{pmatrix} 1 \\ \vdots \\ 1 \end{pmatrix}.$$

where X is called the *design matrix* and Z is the *extended design matrix*, we have:

$$\hat{\boldsymbol{\theta}}_n \in \arg \min_{\boldsymbol{\theta} \in \mathbb{R}^{d+1}} \|Y - Z\boldsymbol{\theta}\|^2, \quad (2.1)$$

where $\|\cdot\|$ stands for the Euclidean norm.

The prediction at $\mathbf{x} = (x^{(1)}, \dots, x^{(d)})$ is given by

$$\hat{g}(\mathbf{x}) = \hat{\boldsymbol{\theta}}_{n,0} + \sum_{k=1}^d \hat{\boldsymbol{\theta}}_{n,k} x^{(k)}.$$

In order to solve the ERM problem in Equation (2.1), we will rely on the so-called *normal equation*. Of course $\hat{Y} = Z\hat{\boldsymbol{\theta}}_n$ is the closest point to Y in the linear subspace $\text{span}(Z) \subset \mathbb{R}^n$. This point is characterized by $Z^T(Y - \hat{Y}) = 0$. Since $\hat{Y} = Z\hat{\boldsymbol{\theta}}_n$, we obtain

$$Z^T Z \hat{\boldsymbol{\theta}}_n = Z^T Y.$$

Note that in contrast with $\hat{Y}$ (which is always unique), the vector $\hat{\boldsymbol{\theta}}_n$ is not uniquely defined without further assumptions on the data. For instance, take $u \in \ker(Z)$ then $\hat{\boldsymbol{\theta}}_n + u$ verifies the *normal equations* as well as $\hat{\boldsymbol{\theta}}_n$.

- $\hat{G}_n = Z^T Z / n$ is called the Gram matrix.
- Denote by $\hat{H}_{n,Z} \in \mathbb{R}^{n \times n}$ the orthogonal projector on $\text{span}(Z)$.

The OLS estimator always exists, and the associated prediction is given by $\hat{Y} = \hat{H}_{n,Z} Y$. It is either

1. Uniquely defined. This happens if and only if the Gram matrix is invertible, which is equivalent to $\ker(Z) = \ker(Z^T Z) = \{0\}$. In this case, the OLS has the following expression:

$$\hat{\boldsymbol{\theta}}_n = (Z^T Z)^{-1} Z^T Y.$$

2. Or not unique, with an infinite number of solutions. This happens if and only if $\ker(Z) \neq \{0\}$. In this case, the set of solutions writes $\hat{\boldsymbol{\theta}}_n + \ker(Z)$ where $\hat{\boldsymbol{\theta}}_n$ is a particular solution.

Corollary. The set of OLS solutions is given by $\{(Z^T Z)^+ Z^T Y + u : u \in \ker(Z)\}$ where A^+ denote the generalized inverse of A .

Finally, in order to evaluate the accuracy of the OLS estimator on the training set, we recall the definition of the determination coefficient.

Definition 6 (Determination coefficient). Suppose that $\sum_{i=1}^n (y_i - \bar{y}^n)^2 > 0$, i.e., that the sequence y_i is not constant. The **determination coefficient** R^2 is defined as

$$R^2 = \frac{\sum_{i=1}^n (\hat{y}_i - \bar{y}^n)^2}{\sum_{i=1}^n (y_i - \bar{y}^n)^2} = \frac{\|\hat{Y} - \bar{y}^n \mathbf{1}_n\|^2}{\|Y - \bar{y}^n \mathbf{1}_n\|^2}.$$

It is the quotient between *explained sum of squares* and *total sum of squares*. We also have that

$$R^2 = 1 - \frac{\|\hat{Y} - Y\|^2}{\|Y - \bar{y}^n \mathbf{1}_n\|^2}.$$

In this definition, the quantity $\|\hat{Y} - Y\|^2$ is called the *residual sum of squares*. The R^2 is usually interpreted as a score supporting the quality of the OLS estimation:

- $R^2 = 1$ if and only if $Y = \hat{Y}$.
- $R^2 = 0$ if and only if $\hat{Y} = \hat{H}_{1_n} Y$ implying that $\hat{\theta}_n = (\bar{y}^n, 0, \dots, 0)$ is one OLS estimator.

2.1.2 Statistical Properties of OLS estimation

The *fixed design model* is given by the equation:

$$y_i = \mathbf{z}_i^T \boldsymbol{\theta}^* + \varepsilon_i, \quad \text{for all } i = 1, \dots, n,$$

where $(\mathbf{z}_i)$ is a sequence of deterministic points in $\mathbb{R}^{d+1}$ and (ε_i) is a collection of random variables in $\mathbb{R}$ such that

$$\mathbb{E}[\varepsilon] = 0, \quad \text{Var}(\varepsilon) = \sigma^2 I_n, \quad \text{with } \varepsilon = \begin{pmatrix} \varepsilon_1 \\ \vdots \\ \varepsilon_n \end{pmatrix}.$$

The level of noise σ reflects the difficulty of the problem.

In the *sub-Gaussian assumption*, the model is described as:

$$y_i = \mathbf{z}_i^T \boldsymbol{\theta}^* + \varepsilon_i, \quad \text{for all } i = 1, \dots, n,$$

where $(\mathbf{z}_i)$ is a sequence of deterministic points in $\mathbb{R}^{d+1}$ and (ε_i) is a collection of independent random variables in $\mathbb{R}$ such that

$$\mathbb{E}[\varepsilon_i] = 0, \quad \mathbb{E}[\exp(\lambda \varepsilon_i)] \leq \exp(\lambda^2 \sigma^2 / 2).$$

Further noting that the empirical and true risks are given by

$$\hat{R}_n(\boldsymbol{\theta}) = n^{-1} \sum_{i=1}^n (y_i - \mathbf{z}_i^T \boldsymbol{\theta})^2$$

$$R(\boldsymbol{\theta}) = n^{-1} \sum_{i=1}^n \mathbb{E}_{Y_i}[(Y_i - \mathbf{z}_i^T \boldsymbol{\theta})^2],$$

we have several propositions satisfied by the OLS estimator which are recapped below.

Proposition 1. In the fixed design model, if $\ker(Z) = \{0\}$, the following holds:

1. $\hat{\boldsymbol{\theta}}_n - \boldsymbol{\theta}^* = (Z^T Z)^{-1} Z^T \varepsilon$
2. The OLS estimator is unbiased i.e., it holds that $\mathbb{E}[\hat{\boldsymbol{\theta}}_n] = \boldsymbol{\theta}^*$
3. $\text{Var}(\hat{\boldsymbol{\theta}}_n) = (Z^T Z)^{-1} \sigma^2$
4. The L_2 error is $\mathbb{E}[\|\hat{\boldsymbol{\theta}}_n - \boldsymbol{\theta}^*\|^2] = \text{trace}((Z^T Z)^{-1}) \sigma^2$
5. The expected empirical risk is $\mathbb{E}[\hat{R}_n(\hat{\boldsymbol{\theta}}_n)] = \sigma^2(1 - \frac{(d+1)}{n})$
6. $\mathbb{E}[\|Z\boldsymbol{\theta}^* - Z\hat{\boldsymbol{\theta}}_n\|^2] = (d+1)\sigma^2$

Proposition 2. In the fixed design model, if $\ker(Z) = \{0\}$, the following holds:

$$\mathbb{E}[R(\hat{\boldsymbol{\theta}}_n)] = R(\boldsymbol{\theta}^*) + \frac{(d+1)\sigma^2}{n}$$

Proposition 3. Denote by $\hat{\lambda}_n$ the smallest eigenvalue of $\hat{G}_n$ and suppose that $\hat{\lambda}_n > 0$ for all $n \geq d+1$. Then, for any $k \in \{0, \dots, d\}$, $n \geq d+1$ and $\delta > 0$, it holds with probability $1 - \delta$,

$$|\hat{\boldsymbol{\theta}}_{n,k} - \boldsymbol{\theta}_k^*| \leq \sqrt{\frac{2\sigma^2 \log(2(d+1)/\delta)}{n\hat{\lambda}_n}}.$$

Corollary.

$$\mathbb{E}[\hat{R}_n(\hat{\boldsymbol{\theta}}_n) - R(\hat{\boldsymbol{\theta}}_n)] = -2 \frac{(d+1)\sigma^2}{n}$$

Remark (The Mallows's criterion C_p). You should have seen last year that it writes

$$C_p = \hat{R}_n(\hat{\theta}_n) + 2\sigma^2 \left(\frac{d+1}{n} \right).$$

With the Corollary above, we see that:

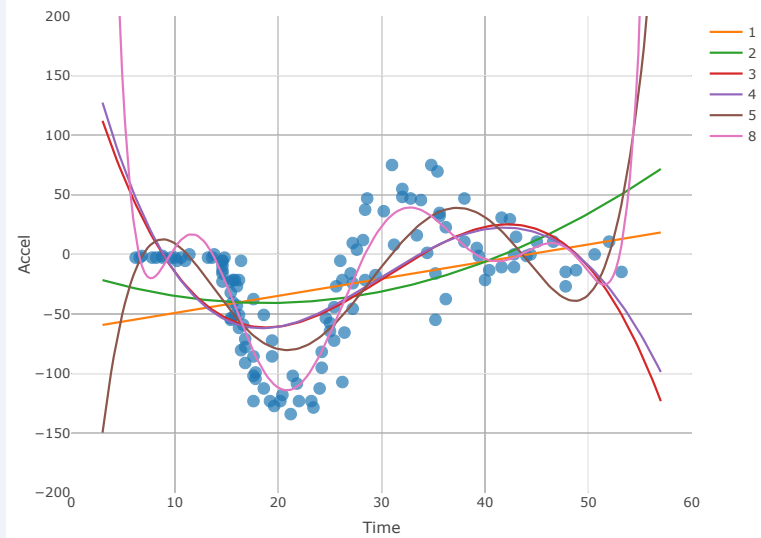
- The Mallows's C_p is a **bias-corrected risk estimate**
- Even if cross-validation is the most standard approach, the Mallows's criterion offers an alternative much less expensive for OLS estimators (only).

2.2 Ridge regularization

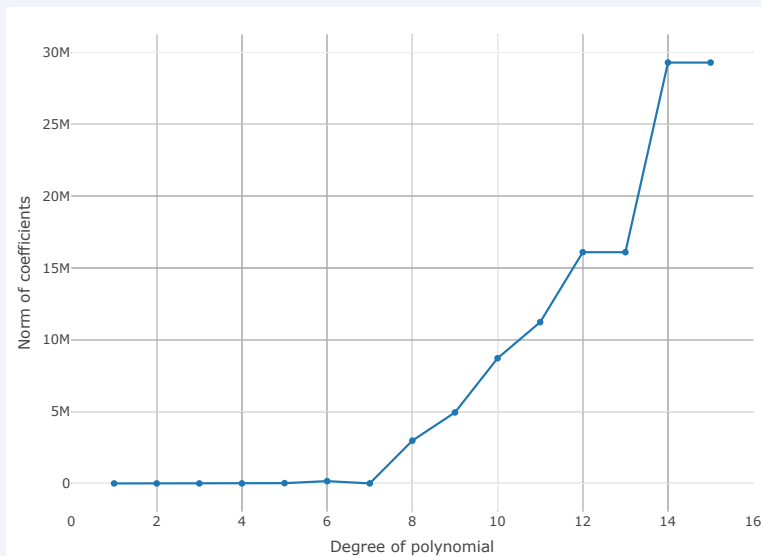
The *ridge* penalty, which makes use of the L^2 norm of the OLS estimator, is very often of great use in practice to accommodate three frequent situations:

1. **Ill-Conditioning.** When $\ker(Z) = \{0\}$ (i.e. the OLS estimator is unique) but the Gram matrix $\hat{G}_n = Z^T Z/n$ is ill-conditioned. In such a case you may encounter numerical errors when trying to compute the OLS solution. Since this involves solving a linear system, a typical fix is to add a diagonal matrix (with small amplitude) to $\hat{G}_n$, which actually corresponds to adding the L^2 norm of the coefficients as we will see below.
2. **More variables than samples.** When $n < d + 1$, we immediately have $\ker(Z) \neq \{0\}$ and then the OLS solution is not unique. But we still may be interested in a specific unique solution (the minimum L^2 norm one in this case).
3. **Controlling overfitting.** Even in the favorable case $\ker(Z) = \{0\}$ but when you include higher-order polynomials of the original features in the vector (z_i) (or if the initial features are strongly correlated), the OLS predictor may tend to overfit the data. If you have a look at the values of the coefficients in the solution $\hat{\theta}_n$ obtained in such a case, you will observe that they may have a very large absolute value. Imposing a constraint on their L^2 norm thus tends to lessen overfitting. This point is illustrated in the following example.

Example (Polynomial regression with increasing degree). We focus on the `mcycle` dataset from the `MASS` package in R, which consists of a sample of size 133 of observed head accelerations in g (the target y to be predicted) as a function of time in ms (the feature x). Here we only consider samples with times between 5 and 55, resulting in a sample of size $n = 124$. We train several polynomial regression model with a degree ranging from 1 to 15. The predictions obtained with some of these models are given below.



We observe that, although the predictions fit the training data better and better when the degree of the polynomial increases, the shape seems to be "wigglier". In addition, if we examine the value of the coefficients obtained for each models, and more specifically their L^2 norm, we clearly see that it explodes with the degree, as illustrated below.



2.2.1 Definition of the ridge estimator

Formally, the ridge regression problem writes as follows.

Definition 7 (Ridge regression). The **ridge regression problem** is given

by

$$\hat{\theta}_{\lambda}^{\text{rdg}} = \arg \min_{\theta \in \mathbb{R}^d} \left(\underbrace{\|\mathbf{y} - X\theta\|_2^2}_{\text{data fitting}} + \underbrace{\lambda \|\theta\|_2^2}_{\text{regularization}} \right)$$

In particular, the ridge estimator is **unique** for any fixed $\lambda > 0$ and we recover the limiting cases:

$$\begin{aligned} \lim_{\lambda \rightarrow 0} \hat{\theta}_{\lambda}^{\text{rdg}} &= \hat{\theta}^{\text{OLS}} (\text{solution with smallest } \|\cdot\|_2 \text{ norm}) \\ \lim_{\lambda \rightarrow +\infty} \hat{\theta}_{\lambda}^{\text{rdg}} &= 0 \in \mathbb{R}^d \end{aligned}$$

From the first order conditions we get an explicit expression of the ridge solution:

$$\nabla f(\theta) = X^T(X\theta - \mathbf{y}) + \lambda\theta = 0 \Leftrightarrow (X^T X + \lambda I_d)\theta = X^T \mathbf{y}$$

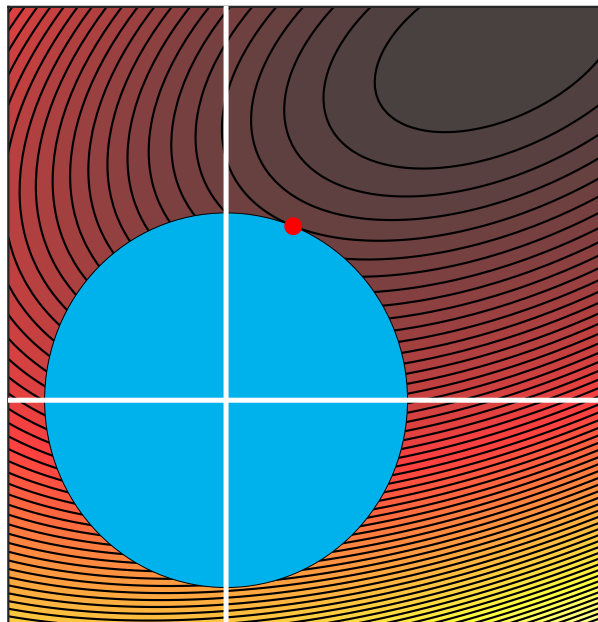
An equivalent *Lagrangian formulation* also exists, meaning that for a certain $T > 0$ the ridge regression problem has the same solution as:

$$\begin{aligned} \arg \min_{\theta \in \mathbb{R}^d} & \|\mathbf{y} - X\theta\|_2^2 \\ \text{s.t. } & \|\theta\|_2^2 \leq T \end{aligned}$$

But the link $T \leftrightarrow \lambda$ is not explicit!

- If $T \rightarrow 0$ we recover the null vector: $0 \in \mathbb{R}^d$
- If $T \rightarrow \infty$ we recover $\hat{\theta}^{\text{OLS}}$ (unconstrained)

Remark. The Lagrangian formulation has a nice geometric interpretation which helps understand what ridge regression does. In the figure below, we display the level sets of the quadratic loss. Without the ridge penalty, the optimal value of θ lies in the upper left corner. When we add the constraint on its L^2 norm, the optimal value corresponds to the intersection of the L^2 ball of radius $\sqrt{T}$ displayed in blue, with the lower valued level set of the quadratic loss: this intersection is the red point and it corresponds to the ridge solution.



The ridge estimator has the following properties:

- The *ridge* coefficients are given by

$$\hat{\boldsymbol{\theta}}_{\lambda}^{\text{rdg}} = (\lambda I_d + X^T X)^{-1} X^T \mathbf{y}$$

- The associated prediction is given by

$$\hat{\mathbf{y}} = X \hat{\boldsymbol{\theta}}_{\lambda}^{\text{rdg}} = X(\lambda I_d + X^T X)^{-1} X^T \mathbf{y}$$

- The estimator $\hat{\mathbf{y}}$ is linear w.r.t. $\mathbf{y}$
- The equivalent 'hat' matrix is $H_{\lambda} := X(\lambda I_d + X^T X)^{-1} X^T$. We no longer have $H_{\lambda}^2 = H_{\lambda} = \sum_{j=1}^{\text{rank}(X)} \mathbf{u}_j \mathbf{u}_j^T$ anymore, so H_{λ} is not a projection

You may have noticed that in the definition of the ridge estimator the optimization problem is only in dimension d instead of $d + 1$ for OLS: we have actually dropped the intercept since it should not be penalized. The definition above is thus valid only if **the features are centered beforehand**. In addition, if you want the penalty to be similar across all the features, they should also be normalized. If you do not want to center the features (not recommended!), the ridge estimator writes as

$$\hat{\boldsymbol{\theta}}_{\lambda}^{\text{rdg}} = \arg \min_{\boldsymbol{\theta} \in \mathbb{R}^{d+1}} \|\mathbf{y} - X\boldsymbol{\theta} - \theta_0 \mathbf{1}_n\|_2^2 + \lambda \sum_{j=1}^d \theta_j^2,$$

i.e. there is no constraint on the intercept θ_0 .

2.2.2 Statistical properties of the ridge estimator

As for the OLS estimator, we can derive some properties for the ridge counterpart.

Proposition 4 (Ridge bias). Under the fixed-design model, we have:

$$\begin{aligned}\mathbb{E}(\hat{\theta}_\lambda^{\text{rdg}}) &= \mathbb{E}[(\lambda I_d + X^T X)^{-1} X^T \mathbf{y}] \\ &= \mathbb{E}[(\lambda I_d + X^T X)^{-1} X^T X \theta^* + (\lambda I_d + X^T X)^{-1} X^T \varepsilon] \\ &= (\lambda I_d + X^T X)^{-1} X^T X \theta^*\end{aligned}$$

The bias is $\mathbb{E}(\hat{\theta}_\lambda^{\text{rdg}}) - \theta^* = -\lambda(X^T X + \lambda I_d)^{-1} \theta^*$

Proposition 5 (Ridge variance). Under the assumption $\mathbb{E}(\varepsilon) = 0$, and with a homoscedastic model: $\mathbb{E}(\varepsilon \varepsilon^T) = \sigma^2 I_n$

$$V_\lambda^{\text{rdg}} = \mathbb{E} \left((\hat{\theta}_\lambda^{\text{rdg}} - \mathbb{E}(\hat{\theta}_\lambda^{\text{rdg}})) (\hat{\theta}_\lambda^{\text{rdg}} - \mathbb{E}(\hat{\theta}_\lambda^{\text{rdg}}))^T \right)$$

The explicit computation is as follows:

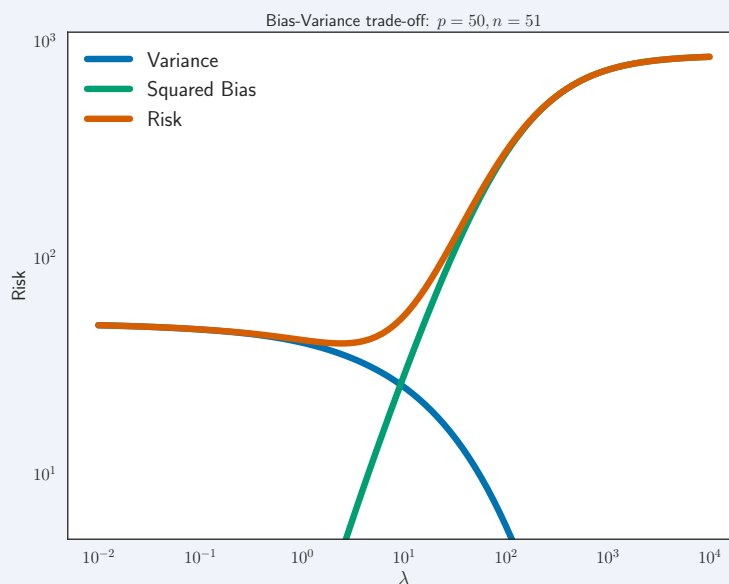
$$\begin{aligned}V_\lambda^{\text{rdg}} &= \mathbb{E}((\lambda I_d + X^T X)^{-1} X^T \varepsilon \varepsilon^T X (\lambda I_d + X^T X)^{-1}) \\ &= (\lambda I_d + X^T X)^{-1} X^T \mathbb{E}(\varepsilon \varepsilon^T) X (\lambda I_d + X^T X)^{-1} \\ &= \sigma^2 (\lambda I_d + X^T X)^{-2} X^T X \quad (\text{matrices commute here})\end{aligned}$$

One finds a null variance when $\lambda \rightarrow \infty$

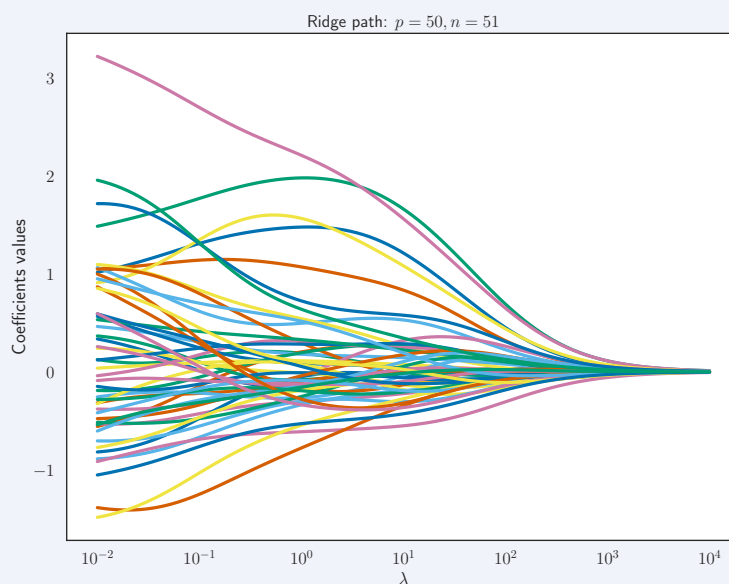
On a simulated dataset, we can compute the bias and the variance of the ridge estimator for each value of λ and deduce the risk, as illustrated in the example below. λ acts as an hyperparameter controlling the complexity of the estimator (remember the limiting case $\lambda \rightarrow 0$ and $\lambda \rightarrow \infty$), thus allowing to perform a bias-variance tradeoff.

Example. The dataset is generated as follows:

- $\theta^* = (2, 2, 2, 2, 2, 0, \dots, 0) \in \mathbb{R}^d$ (5 non-zero coefficients)
- $X \in \mathbb{R}^{n \times d}$ has columns drawn according to a Gaussian distribution
- $y = X \theta^* + \varepsilon \in \mathbb{R}^n$ with $\varepsilon \sim \mathcal{N}(0, \sigma^2 I_n)$
- We use a grid of 50 λ values
- For this example : $n = 51, d = 50, \sigma = 1$



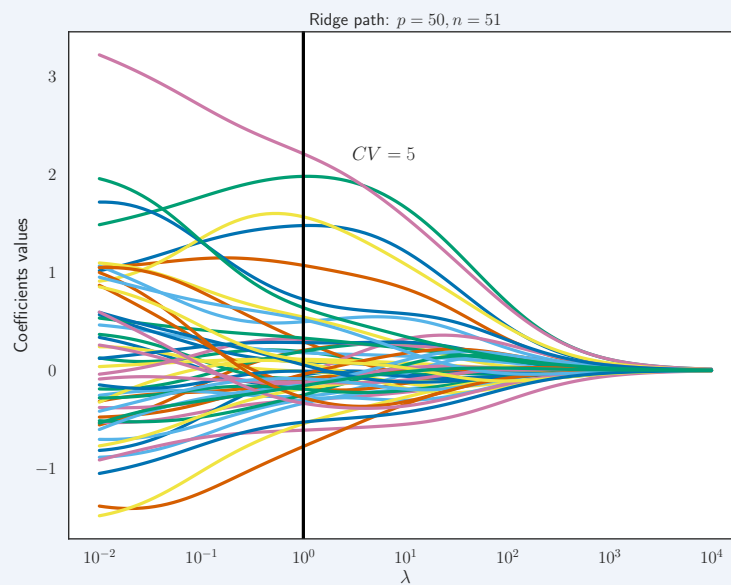
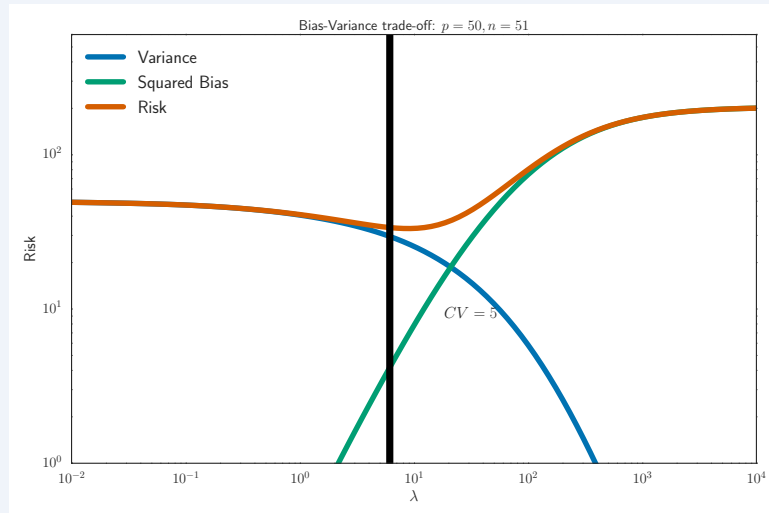
On the same example, we can also plot the values of the coefficients in $\hat{\theta}_{\lambda}^{\text{rdg}}$ when λ varies. As expected, when λ increases, the values are more penalized and all tend to 0. Such a representation is called the **ridge path**, *path* in the sense that it shows for each coefficient its evolution from unconstrained to heavily constrained along the variation of λ .



Now that we have illustrated how the ridge estimator varies with the hyper-parameter λ , the central question is obviously how we can tune it on a given

problem. This is actually straightforward: you can (and should) use cross-validation as elaborated in the first chapter. We give below the obtained result on the previous example.

Example. On the same example, we perform 5-fold cross-validation (CV) and report in the figures the trade-off and the coefficient values obtained for the tuned λ value.



2.2.3 Concluding remarks on the ridge estimator

- In practice the ridge penalty is very useful, and is quite often used by default (with a small fixed value for λ , or with a cross-validation tuning procedure)
- It is recommended when the features are correlated, but we will see below another trick to handle such a case
- Unfortunately, the ridge penalty does not help with *dimension reduction*: it is highly frequent that only a subset of the features really has an impact on the target variable, or that only a small number of feature combinations are important. The next section is dedicated to discuss how you can handle such practical situations.

2.3 Dimension reduction for linear regression

Lastly, we discuss how the problem dimensionality can be reduced for OLS which is nowadays a quasi-mandatory step in machine learning. Historically in statistics, dimension reduction was achieved in linear regression through variable selection with sequential approaches such as forward or backward selection, that you have seen last year. You should keep them in your tool-bag of course, but here we will discuss two other popular paths.

2.3.1 Preliminary dimension reduction with unsupervised methods

When facing a OLS problem with a large number of features which are correlated, an intuitive approach for dimension reduction consists in recycling tools for unsupervised learning that you learnt last year. Among them, the Principal Component Analysis (PCA) method is popular and is recalled below.

Definition 8. Principal component analysis is used to compress the data using linear projection. From the initial data X , it extracts $\mathbb{R}^d$ vectors $u_1, \dots, u_d$ such that $\text{Var}(u_1^T X) \geq \dots \geq \text{Var}(u_d^T X)$. It proceeds as follows:

- We have a given dataset $X \in \mathbb{R}^{n \times d}$
- Center X to get X_c
- Algorithm:
 - Compute $X_c^T X_c / n$
 - $u_1, \dots, u_d$ are the eigenvectors of $X_c^T X_c / n$ with corresponding eigenvalues $\lambda_1 \geq \dots \geq \lambda_d$
- We then get a compressed dataset with K new features given by $X(u_1, \dots, u_K) \in \mathbb{R}^{n \times K}$ which has kept most of the variability of X

In practice, PCA is used as a preliminary step before OLS, leading to the

following algorithm called **PCA-OLS** or *PCA before OLS*. Given a feature matrix $X \in \mathbb{R}^{n \times d}$ and a number K of principal components to keep:

- Run a PCA and keep the K first component $\tilde{X} \in \mathbb{R}^{n \times K}$
- Run OLS with $(Y, \tilde{X})$

As a side-note, even if PCA-OLS is rather popular, other methods can be envisioned in practice:

- **Unsupervised learning.** Popular non-linear dimension reduction approaches are now common in machine learning, for example *t-SNE* or *UMAP*
- **Reduction with supervision.** The drawback of unsupervised-based dimension reduction is that it is blind to the values of the target variables. You can use instead either a supervised learning method which generalizes PCA called *Partial Least-Squares* (PLS), which is still a linear method, but also for example a supervised extension of UMAP mentioned above.

Finally, it is crucial to remember that such preliminary unsupervised dimension reduction will only help you if the original features are dependent or at least correlated. If this is not the case, the main tools to be used instead are variable selection with forward or backward approaches, or with sparsity constraints detailed in what follows.

2.3.2 Lasso regularization

A different point of view consists in formalizing our objective of dimension reduction directly inside the OLS problem through a specifically tailored penalty. Among the methods adopting this point of view, the most popular by far is the so-called *least absolute shrinkage and selection operator*, or **Lasso**. It is defined formally as follows.

Definition 9 (Lasso regression). The **Lasso regression problem** is given by

$$\hat{\theta}_{\lambda}^{\text{Lasso}} = \arg \min_{\theta \in \mathbb{R}^d} \left(\underbrace{\frac{1}{2} \|\mathbf{y} - X\theta\|_2^2}_{\text{data fitting}} + \underbrace{\lambda \|\theta\|_1}_{\text{regularization}} \right)$$

where $\|\theta\|_1 = \sum_{j=1}^d |\theta_j|$ is the sum of absolute values of the coefficients.

Be aware that the Lasso estimator is not always **unique** for a fixed λ (consider cases with two equals columns in X).

We also recover the limiting cases:

$$\begin{aligned} \lim_{\lambda \rightarrow 0} \hat{\theta}_{\lambda}^{\text{Lasso}} &= \hat{\theta}^{\text{OLS}} \\ \lim_{\lambda \rightarrow +\infty} \hat{\theta}_{\lambda}^{\text{Lasso}} &= 0 \in \mathbb{R}^d \end{aligned}$$

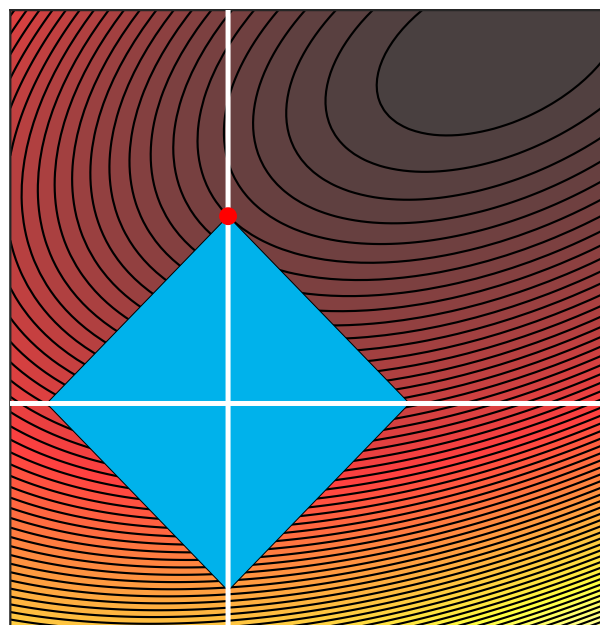
An equivalent *Lagrangian formulation* also exists, meaning that for a certain $T > 0$ the Lasso regression problem has the same solution as:

$$\begin{cases} \arg \min_{\boldsymbol{\theta} \in \mathbb{R}^d} \|\mathbf{y} - X\boldsymbol{\theta}\|_2^2 \\ \text{s.t. } \|\boldsymbol{\theta}\|_1 \leq T \end{cases}$$

But once again the link $T \leftrightarrow \lambda$ is not explicit.

- If $T \rightarrow 0$ one recovers the null vector: $\mathbf{0} \in \mathbb{R}^d$
- If $T \rightarrow \infty$ one recovers $\hat{\boldsymbol{\theta}}^{\text{OLS}}$ (unconstrained)

Remark. As for the ridge penalty, the Lagrangian formulation has a nice geometric interpretation which helps understand how the Lasso leads to variable selection. In the figure below, we display the level sets of the quadratic loss. Without the Lasso penalty, the optimal value of $\boldsymbol{\theta}$ lies in the upper left corner. When we add the constraint on its L^1 norm, the optimal value corresponds to the intersection of the L^1 ball of radius T displayed in blue, with the lower valued level set of the quadratic loss: this intersection is the red point and it corresponds to the Lasso solution. **Variable selection is achieved because of the "spikes" of the L^1 ball.**

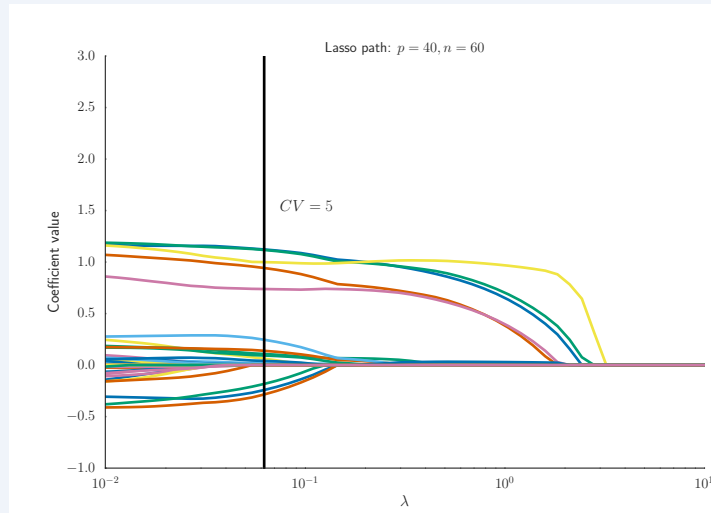


Before discussing some properties of the Lasso estimator, we illustrate its behavior and in particular the influence of λ on a synthetic example.

Example. The dataset is generated as follows:

- $\theta^* = (1, 1, 1, 1, 1, 0, \dots, 0) \in \mathbb{R}^d$ (5 non-zero coefficients)
- $X \in \mathbb{R}^{n \times d}$ has columns drawn according to a Gaussian distribution
- $y = X\theta^* + \varepsilon \in \mathbb{R}^n$ with $\varepsilon \sim \mathcal{N}(0, \sigma^2 I_n)$
- We use a grid of 50 λ values
- For this example : $n = 60, d = 40, \sigma = 1$
- We also compute the optimal value of λ over the grid with 5-fold CV

The **Lasso path** looks like this.



First, you should immediately see that the paths behave differently when compared to the ridge penalty. Indeed, when λ increases, the coefficients of $\hat{\theta}_{\lambda}^{\text{Lasso}}$ are shrunk to exactly zero, thus illustrating the variable selection effect of the Lasso. Furthermore, observe that all the features with a true coefficient equal to zero are identified rapidly when λ approaches 10^{-1} and that only the real meaningful variables still have a strictly positive estimated coefficient. Ultimately when λ is very large all estimated coefficients are equal to zero as expected from the definition above.

Finally, observe that cross-validation in this case was not able to perfectly identify the true relevant features since some of the corresponding estimated coefficients are strictly positive for unimportant features.

The Lasso estimator has the following important properties:

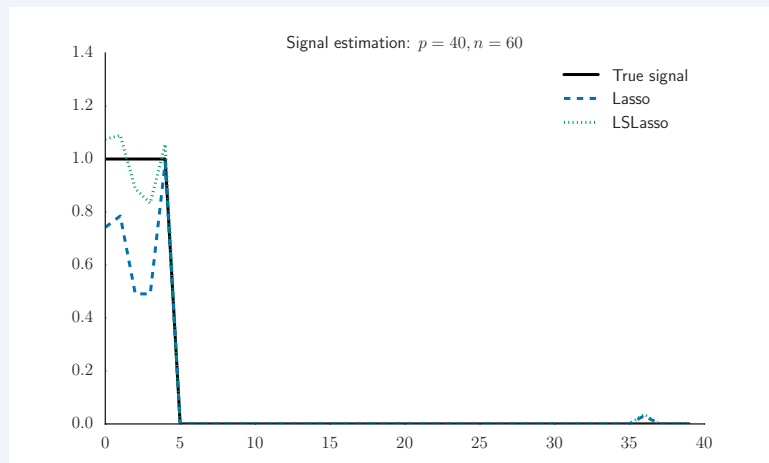
- From a numerical point of view, the Lasso does not have a nice closed-form expression as the ridge one, but it is a **convex** problem
- It performs variable selection and leads to sparse solutions: $\hat{\theta}_{\lambda}^{\text{Lasso}}$ has potentially many zeroed coefficients. The λ parameter controls the sparsity level: if λ is large, solutions are very sparse. In the previous example, we obtain 17 non-zero coefficients for LassoCV whereas RidgeCV has no zero coefficient
- It is very sensitive to correlation between features, in such cases variable selection may perform poorly
- The theory is more involved for the Lasso than for least squares / ridge, but one can prove that it has bias, as illustrated below

Example. In the same previous synthetic example, we can plot the final estimation of the coefficients for the optimally tuned value of λ and observe that there is bias. A way to reduce it is to use the *LS-Lasso* procedure which simply consists of the following steps:

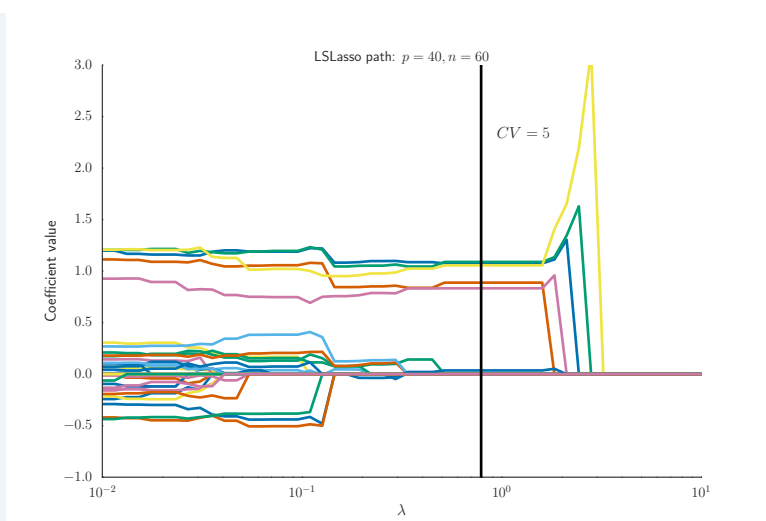
1. Lasso: compute $\hat{\theta}_{\lambda}^{\text{Lasso}}$
2. Perform least squares over selected variables: $\text{supp}(\hat{\theta}_{\lambda}^{\text{Lasso}})$

$$\hat{\theta}_{\lambda}^{\text{LSLasso}} = \arg \min_{\substack{\theta \in \mathbb{R}^d \\ \text{supp}(\theta) = \text{supp}(\hat{\theta}_{\lambda}^{\text{Lasso}})}} \frac{1}{2} \|\mathbf{y} - X\theta\|_2^2$$

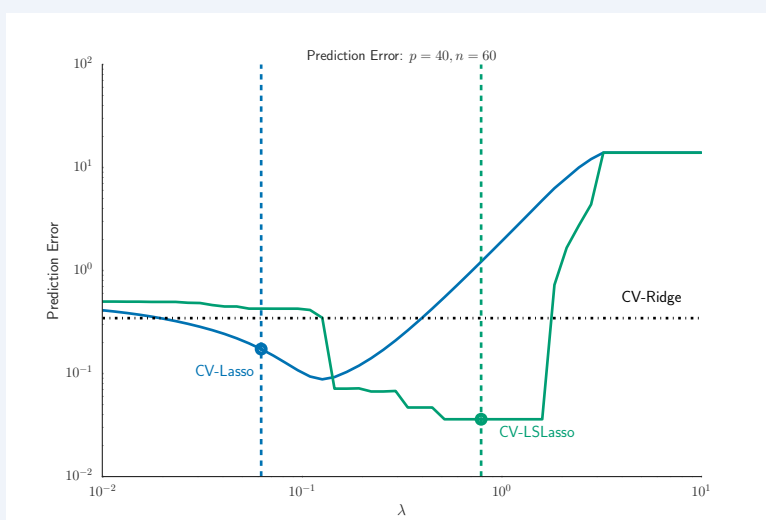
Note that you should perform CV for the double step procedure, indeed choosing λ by LassoCV and then performing OLS keeps too many variables. Standard Lasso and LS-Lasso are compared below.



We can also represent the LS-Lasso path with the value of λ tuned with the CV on the double step procedure.



Once again, we emphasize that the CV procedure should be different between Lasso and LS-Lasso, as illustrated below where we see that the optimal value of λ differs between them.



Finally, an interesting proposal consisting in combining the different strengths of ridge and Lasso (in particular correlation among features) is often used in practice. This is the *Elastic-Net* defined below.

Definition 10. The **Elastic-Net** is the (unique) solution of

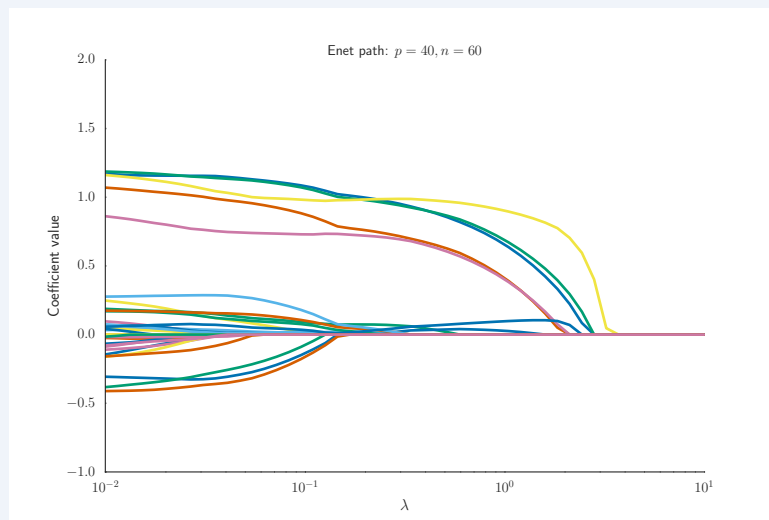
$$\hat{\theta}_\lambda = \arg \min_{\theta \in \mathbb{R}^d} \left[\frac{1}{2} \|\mathbf{y} - X\theta\|_2^2 + \lambda \left(\gamma \|\theta\|_1 + (1 - \gamma) \frac{\|\theta\|_2^2}{2} \right) \right]$$

Observe that two hyperparameters are needed, one for global regulariza-

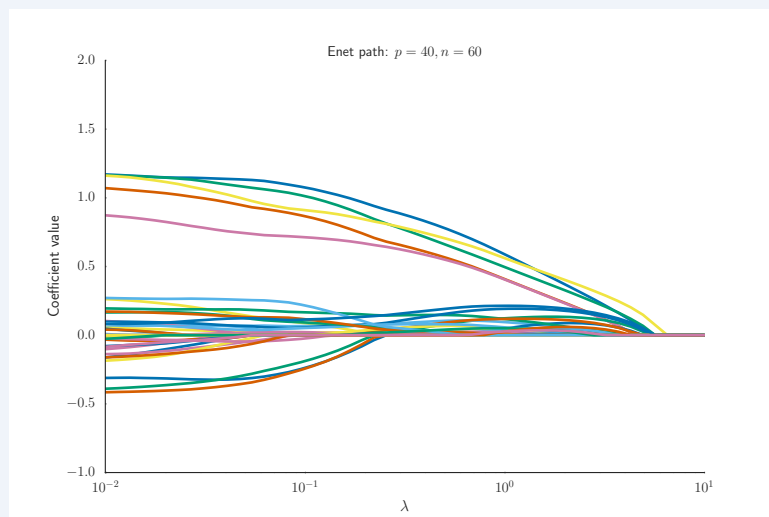
tion, one for the trade-off ridge vs. Lasso

We can furthermore illustrate its behavior on the previous example.

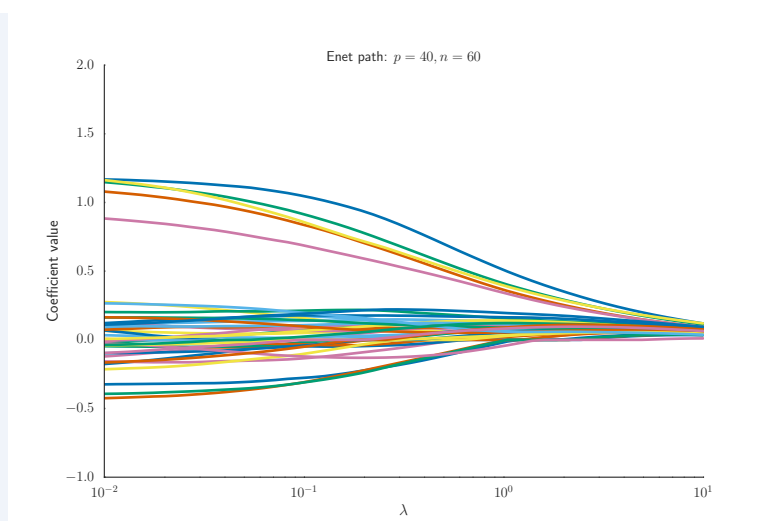
Example. On the same synthetic dataset above, the Elastic-Net path with respect to λ for different values of $\gamma = (0.95, 0.5, 0.05)$ is displayed below. For $\gamma = 0.95$:



For $\gamma = 0.5$:



For $\gamma = 0.05$:



When γ goes from 1 to 0, we observe here that the path evolves from a Lasso-like shape to a ridge-like one as expected.

2.4 Concluding remarks and introduction to the next class

Let us come back to the central object in supervised machine learning, the empirical risk minimizer

$$\hat{g}_{\text{ERM}} = \arg \min_{g \in \mathcal{G}} \hat{R}_{\mathcal{D}_n}(g) = \frac{1}{n} \sum_{i=1}^n l(y_i, g(\mathbf{x}_i)).$$

When we restrict $\mathcal{G}$ to be the set of linear functions, we discussed some prominent estimators:

1. The standard OLS setting, where the solution is explicit and involves solving a linear system
2. The ridge regression problem, which also requires solving a linear system
3. The Lasso (or Elastic-Net) regression problem, for which there is no explicit solution but with a solution given as the optimum of a convex function

For case 3. above and also when $\mathcal{G}$ is not restricted to linear functions, but to more general parametric functions, this means that computing the ERM involves optimization algorithms. But even for cases 1. and 2., when the number of samples n is large (think of the big data paradigm), solving the linear system is computationally demanding and a way to handle such practical situations also involves optimization approaches. In summary, **optimization is also a central field in machine learning**, which will be discussed in the next chapter.

Lecture 2: Optimization for machine learning

September 14, 2023

- Introduction & motivation
- Gradient Descent
 - Convex functions
 - Analysis of gradient descent
 - Variants of gradient descent
- Stochastic Gradient Descent

Chapter 3

Optimization for machine learning

3.1 Introduction & motivation

We recall that in supervised statistical learning, we are given a sample $\mathcal{D}_n = (\mathbf{x}_i, y_i)_{i=1, \dots, n}$ of size n from an unknown joint distribution $\mathbb{P}_{\mathbf{X}Y}$ defined on a product space $\mathcal{X} \times \mathcal{Y}$. Here $\mathbf{X} = (X^{(1)}, \dots, X^{(d)}) \in \mathcal{X} \subset \mathbb{R}^d$ is the vector of the explanatory variables or inputs or *features* and $Y \in \mathcal{Y}$ is the *target* variable, or output. The goal is to build a map $g : \mathcal{X} \mapsto \mathcal{Y}$, called a *predictor* or *prediction function* or *prediction rule*, from the observations in the sample $\mathcal{D}_n$ and which predicts the value of the target variable $y = g(\mathbf{x})$ of any other individual with features $\mathbf{x}$.

The prediction rule of interest is the empirical risk minimizer (ERM)

$$\hat{g}_{\text{ERM}} = \arg \min_{g \in \mathcal{G}} \hat{R}_{\mathcal{D}_n}(g) = \frac{1}{n} \sum_{i=1}^n l(y_i, g(\mathbf{x}_i))$$

where $\mathcal{G}$ is a set of candidates functions. In this chapter, we will limit ourselves to the set of **parametric predictors**:

$$\mathcal{G} = \{g_{\boldsymbol{\theta}}, \boldsymbol{\theta} \in \mathbb{R}^d\},$$

leading to a reformulation of the ERM as the following optimization problem

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta} \in \mathbb{R}^d} F(\boldsymbol{\theta}) := \frac{1}{n} \sum_{i=1}^n l(y_i, g_{\boldsymbol{\theta}}(\mathbf{x}_i))$$

where we have introduced the notation $F(\boldsymbol{\theta})$ for the empirical risk to highlight its dependence to $\boldsymbol{\theta}$. In general, the minimizer $\boldsymbol{\theta}^*$ will not have a closed-form, meaning that we have to resort to optimization algorithms which will return a certain value $\hat{\boldsymbol{\theta}}$ which we expect to be close to $\boldsymbol{\theta}^*$.

Although the restriction to such parametric models may seem strong (since the oracle will almost certainly not be in such a set), this setting actually encompasses the simple OLS and its variants discussed in the previous chapter, but

also complex models such as deep neural networks that you will learn in the Deep Learning class this year. But rest assured, we will discuss nonparametric models and the corresponding optimization problem later in this class for kernel methods.

3.2 Gradient descent

We will start by leaving aside the machine learning context and focus on the general task which consists in solving the optimization problem

$$\min_{\boldsymbol{\theta} \in \mathbb{R}^d} F(\boldsymbol{\theta})$$

for a given function $F : \mathbb{R}^d \rightarrow \mathbb{R}$. Here, the minimal property that we require for F is to be differentiable (we will discuss extensions in Section (3.2.3)):

Definition 11 (Differentiable function). A function $F : \mathbb{R}^d \rightarrow \mathbb{R}$ is said to be differentiable at $\boldsymbol{\theta}_0$ if there exists a vector $\mathbf{u} \in \mathbb{R}^d$ such that

$$\lim_{\mathbf{h} \rightarrow 0} \frac{|F(\boldsymbol{\theta}_0 + \mathbf{h}) - F(\boldsymbol{\theta}_0) - \mathbf{u}^T \mathbf{h}|}{\|\mathbf{h}\|}$$

exists. As a consequence, when taking $\mathbf{h} = t\mathbf{e}_j$ with $t \rightarrow 0$ the j -th partial derivatives exists and the **gradient** ∇F of F at $\boldsymbol{\theta}$ is given by the vector

$$[\nabla F(\boldsymbol{\theta})]_j = \frac{\partial F}{\partial \theta_j}(\boldsymbol{\theta}), \quad j = 1, \dots, d.$$

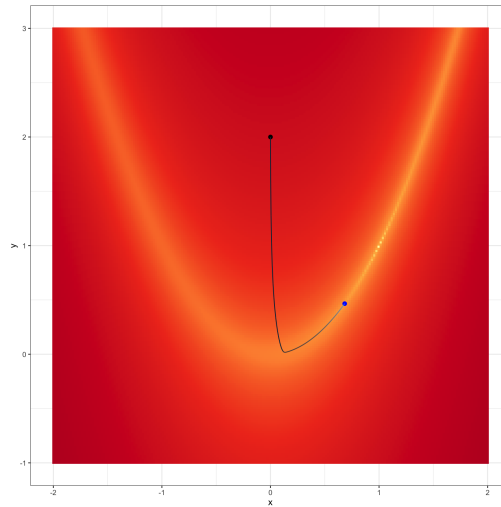
You may recall from your optimization class in first year that the direction of the gradient vector points to higher values of the function. This means that going along the negative gradient direction should help you minimize the function. This is the principle behind gradient descent, which is an iterative algorithm producing a sequence $\boldsymbol{\theta}_0, \boldsymbol{\theta}_1, \dots$ of minimization candidates.

Definition 12 (Gradient descent). Starting from $\boldsymbol{\theta}_0 \in \mathbb{R}^d$, **gradient descent** (GD) builds the sequence

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \gamma_t \nabla F(\boldsymbol{\theta}_{t-1}), \quad t \geq 1$$

where $(\gamma_t)_{t \geq 1}$ is a (well-chosen) **step-size sequence** and t denotes the iteration of GD.

Example. We illustrate GD on the Rosenbrock function with a sequence of constant step-size $\gamma = 10^{-3}$ and 1000 iterations. The initialization is the black point and the last iterate is the blue one. The true minimizer is $(1, 1)$.



Unfortunately without additional assumptions on F the convergence analysis of GD is quite complicated. This is the reason why in the following we will restrict ourselves to convex functions, even if you should keep in mind that GD is also used in practice when such assumption does not hold: this is for example the case with neural networks !

3.2.1 Convex functions

We start by the standard definition of convexity.

Definition 13 (Convex function). A function $F : \mathbb{R}^d \rightarrow \mathbb{R}$ is said to be **convex** if and only if

$$\forall(\boldsymbol{\eta}, \boldsymbol{\theta}) \in \mathbb{R}^d \times \mathbb{R}^d, \forall \alpha \in [0, 1], \quad F(\alpha\boldsymbol{\eta} + (1 - \alpha)\boldsymbol{\theta}) \leq \alpha F(\boldsymbol{\eta}) + (1 - \alpha)F(\boldsymbol{\theta}).$$

If we further assume that F is differentiable, this is equivalent to

$$\forall(\boldsymbol{\eta}, \boldsymbol{\theta}) \in \mathbb{R}^d \times \mathbb{R}^d, \quad F(\boldsymbol{\eta}) - F(\boldsymbol{\theta}) \geq \nabla F(\boldsymbol{\theta})^T(\boldsymbol{\eta} - \boldsymbol{\theta}).$$

If F is also twice-differentiable, this is equivalent to

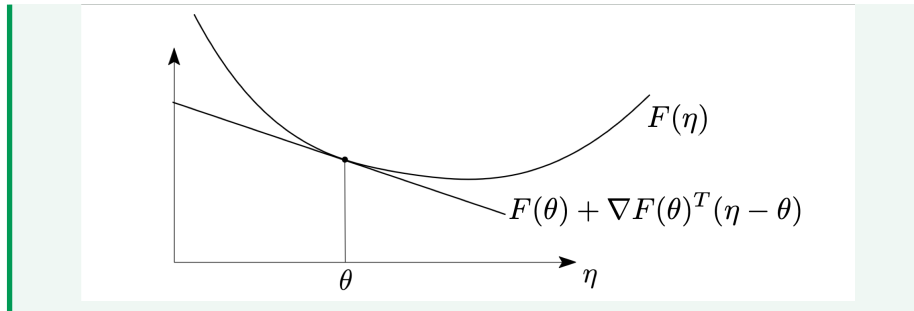
$$\nabla^2 F(\boldsymbol{\eta}) \succeq 0$$

where $\nabla^2 F(\boldsymbol{\eta})$ is the *Hessian* of F consisting of second-order partial derivatives

$$[\nabla^2 F(\boldsymbol{\eta})]_{ij} = \frac{\partial^2 F}{\partial \eta_i \partial \eta_j}(\boldsymbol{\theta}), \quad i, j = 1, \dots, d$$

and $\succeq$ denotes the semidefinite partial ordering characterized by $A \succeq B \Leftrightarrow A - B$ is positive semidefinite.

This corresponds to a function which is above its tangent as illustrated below in one dimension.



Example (ERM). In machine learning, the ERM problem is convex if the loss function l is convex in its second argument and if g_{θ} is linear in θ .

Convexity implies an immediate but particularly useful property.

Proposition 6. Assume that $F : \mathbb{R}^d \rightarrow \mathbb{R}$ is convex and differentiable. Then $\eta^* \in \mathbb{R}^d$ is a global minimizer of F if and only if

$$\nabla F(\eta^*) = 0,$$

in other words η^* is a stationary point of F .

This implies that for convex functions, we only need to look for stationary points. But be aware that **this is not the case for non-convex functions!** Finally, to facilitate further the analysis, we will use a concept stronger than just convexity.

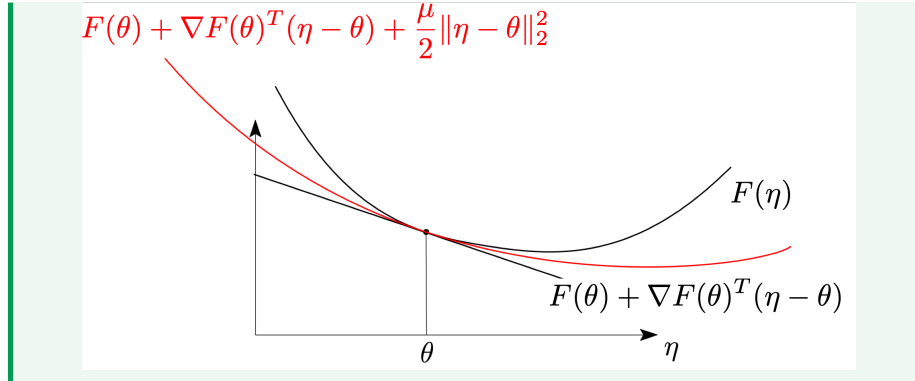
Definition 14 (Strongly convex function). A differentiable function $F : \mathbb{R}^d \rightarrow \mathbb{R}$ is said to be μ -strongly convex with $\mu > 0$ if and only if

$$\forall(\eta, \theta) \in \mathbb{R}^d \times \mathbb{R}^d, \quad F(\eta) - F(\theta) \geq \nabla F(\theta)^T(\eta - \theta) + \frac{\mu}{2} \|\eta - \theta\|_2^2$$

If F is also twice-differentiable, this is equivalent to

$$\nabla^2 F(\eta) \succeq \mu I_d.$$

This corresponds to a function F which is strictly above its tangent and the difference is at least quadratic in the distance to the point where the two coincide, as illustrated below in one dimension.



Example (ERM with ridge penalty). In machine learning, the ERM problem when we add a penalty $\lambda \|\theta\|_2^2$ is 2λ -strictly convex if the loss function l is convex in its second argument and if g_θ is linear in θ .

3.2.2 Analysis of gradient descent

We will make a final additional assumption before discussing the properties of gradient descent.

Definition 15 (L -smoothness). A differentiable function $F : \mathbb{R}^d \rightarrow \mathbb{R}$ is said to be L -smooth if and only if

$$\forall(\boldsymbol{\eta}, \boldsymbol{\theta}) \in \mathbb{R}^d \times \mathbb{R}^d, \quad |F(\boldsymbol{\eta}) - F(\boldsymbol{\theta}) - \nabla F(\boldsymbol{\theta})^T(\boldsymbol{\eta} - \boldsymbol{\theta})| \leq \frac{L}{2} \|\boldsymbol{\eta} - \boldsymbol{\theta}\|_2^2.$$

This is equivalent to the condition that ∇F is L -Lipschitz, i.e.

$$\forall(\boldsymbol{\eta}, \boldsymbol{\theta}) \in \mathbb{R}^d \times \mathbb{R}^d, \quad \|\nabla F(\boldsymbol{\eta}) - \nabla F(\boldsymbol{\theta})\|_2 \leq L \|\boldsymbol{\eta} - \boldsymbol{\theta}\|_2.$$

If F is also twice-differentiable, this is also equivalent to

$$-LI_d \preceq \nabla^2 F(\boldsymbol{\eta}) \preceq LI_d.$$

We can now state a reassuring fact about GD.

Proposition 7 (Small steps in GD cannot be bad). Suppose that F is L -smooth and that we take a constant step-size $\gamma \leq 2/L$. Then

$$\forall t \geq 1, \quad F(\boldsymbol{\theta}_t) \leq F(\boldsymbol{\theta}_{t-1}).$$

The most interesting results, however, state that GD converges to the true minimizer but with convergence rates that differ depending on the properties of F .

Theorem 2 (Convergence of GD). Suppose that F is differentiable and L -smooth.

- If F is convex, we have for a constant step-size $\gamma \leq 1/L$

$$\forall t \geq 1, \quad F(\boldsymbol{\theta}_t) - F(\boldsymbol{\theta}^*) \leq \frac{1}{2\gamma t} \|\boldsymbol{\theta}_0 - \boldsymbol{\theta}^*\|_2^2.$$

- If F is μ -strongly convex, we have for a constant step-size $\gamma \leq 1/L$

$$\forall t \geq 1, \quad F(\boldsymbol{\theta}_t) - F(\boldsymbol{\theta}^*) \leq (1 - \gamma\mu)^t \|\boldsymbol{\theta}_0 - \boldsymbol{\theta}^*\|_2^2$$

and for $\gamma = 1/L$

$$\forall t \geq 1, \quad F(\boldsymbol{\theta}_t) - F(\boldsymbol{\theta}^*) \leq \left(1 - \frac{\mu}{L}\right)^t \|\boldsymbol{\theta}_0 - \boldsymbol{\theta}^*\|_2^2$$

This result means that the convergence of GD is driven by the convexity properties of F :

- When F is convex, in order to get a function value close to the optimum with a tolerance of ε , we need to perform $O(1/\varepsilon)$ iterations: this is called *sublinear convergence*.
- When F is μ -strictly convex
 - The same tolerance is achieved with $O(\log(1/\varepsilon))$ iterations: this is called *linear convergence*.
 - The ratio $\kappa = L/\mu$, which is called the *condition number*, influences the rate of convergence; the larger the more difficult the optimization will be.

3.2.3 Variants of gradient descent

Gradient descent is the simplest algorithm with a simple analysis, but there are several extensions that are useful (in machine learning in particular). We only briefly mention some of them.

Nesterov acceleration

When F is convex, a simple modification to GD, known as *Nesterov acceleration*, makes it possible to obtain faster convergence rate. It consists in coupling two sequences of iterates $(\boldsymbol{\theta}_t)$ and $(\boldsymbol{\eta}_t)$ as follows

$$\begin{aligned} \boldsymbol{\theta}_t &= \boldsymbol{\eta}_{t-1} - \gamma_t \nabla F(\boldsymbol{\eta}_{t-1}) \\ \boldsymbol{\eta}_t &= \boldsymbol{\theta}_t + \frac{t-1}{t+2}(\boldsymbol{\theta}_t - \boldsymbol{\theta}_{t-1}). \end{aligned}$$

The convergence rate goes from $O(1/t)$ to $O(1/t^2)$. It can also be applied in a similar way to strongly convex functions, for which the modification influences the constant in the convergence rate, from $(1 - 1/\kappa)$ to $(1 - 1/\sqrt{\kappa})$ (this can have a large impact in practice).

Second-order methods

It is easy to see that GD actually corresponds to a sequence of minimization of a first-order Taylor expansion of F around the previous iterate (proof left as an exercise). A straightforward generalization, called *Newton method*, consists in considering instead a second-order Taylor expansion, leading to the sequence

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \nabla^2 F(\boldsymbol{\theta}_{t-1})^{-1} \nabla F(\boldsymbol{\theta}_{t-1}), \quad t \geq 1$$

which converges quadratically locally. But such a sequence is often very costly to compute, with a complexity of $O(d^3)$. A famous workaround, in the class of *Quasi-Newton methods*, was proposed by Broyden, Fletcher, Goldfarb and Shanno (BFGS method !) where an approximation of $\nabla^2 F(\boldsymbol{\theta}_{t-1})^{-1} \mathbf{u}$ is constructed with complexity $O(d^2)$ instead.

Non-smooth optimization problems

At the very beginning of this chapter we restricted ourselves to the case where F is at least differentiable. This may not be the case in practice, but in some situations it is possible to easily extend gradient descent to *subgradient* descent, with the notion of subgradient given below.

Definition 16 (Subgradient). Let $F : \mathbb{R}^d \rightarrow \mathbb{R}$ be a convex function. A **subgradient** of F at $\boldsymbol{\theta}$ is any vector $\mathbf{u} \in \mathbb{R}^d$ satisfying

$$\forall \boldsymbol{\eta} \in \mathbb{R}^d, \quad F(\boldsymbol{\eta}) - F(\boldsymbol{\theta}) \geq \mathbf{u}^T (\boldsymbol{\eta} - \boldsymbol{\theta}).$$

The **subdifferential** of F at $\boldsymbol{\theta}$, noted $\partial F(\boldsymbol{\theta})$, is the set of all subgradients of F at $\boldsymbol{\theta}$.

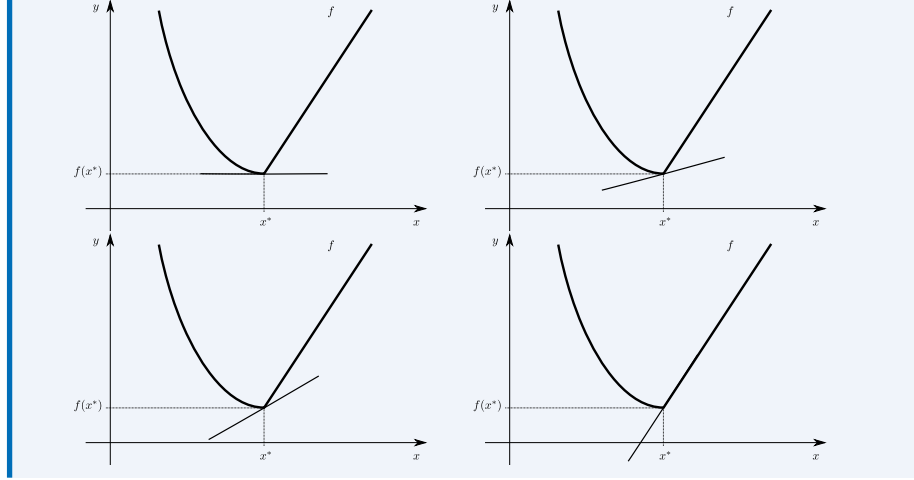
Example. We start with a function that you know well, the absolute value function defined by

$$F : \begin{cases} \mathbb{R} & \rightarrow \mathbb{R} \\ x & \mapsto |x| \end{cases}$$

Then we can show that its subdifferential is the *sign* function given by

$$\partial F(x) = \begin{cases} \{-1\} & \text{if } x \in]-\infty, 0[\\ \{1\} & \text{if } x \in]0, \infty[\\ [-1, 1] & \text{if } x = 0 \end{cases}$$

We now illustrate a more complicated case. The function displayed below is differentiable everywhere except at the point in the middle. But we can define its subdifferential since there is an infinity of subgradients satisfying the definition. We display 4 of them below.



For convex functions with subgradients, we get characterization of the minimum with stationary points just like for GD.

Proposition 8. Assume that $F : \mathbb{R}^d \rightarrow \mathbb{R}$ is convex and subdifferentiable. Then $\boldsymbol{\eta}^* \in \mathbb{R}^d$ is a minimizer of F if and only if

$$0 \in \partial F(\boldsymbol{\eta}^*).$$

Be aware that despite its name, subgradient descent is not a descent method because at some iterations the objective function may increase. Now, subgradient descent can of course be used in machine learning when there is a non-smooth but subdifferentiable function (think about the Lasso penalty), but nowadays other approaches are more popular due to the specific form of optimization problems appearing in machine learning.

When the objective function F decomposes as a sum of two functions G and H when G is smooth and H is non-smooth (such as a norm), **proximal methods** are highly popular. To introduce them, we start by noting that the GD iterations applied to the minimization of G alone are equivalent to the following optimization problem:

$$\boldsymbol{\theta}_t = \arg \min_{\boldsymbol{\theta} \in \mathbb{R}^d} G(\boldsymbol{\theta}_{t-1}) + (\boldsymbol{\theta} - \boldsymbol{\theta}_{t-1})^T \nabla G(\boldsymbol{\theta}_{t-1}) + \frac{1}{2\gamma_t} \|\boldsymbol{\theta} - \boldsymbol{\theta}_{t-1}\|_2^2,$$

where the result easily follows by taking the gradient with respect to $\boldsymbol{\theta}$ and cancelling it. Proximal methods build upon this idea by simply adding function H (left as it is) inside the iterations to get

$$\boldsymbol{\theta}_t = \arg \min_{\boldsymbol{\theta} \in \mathbb{R}^d} G(\boldsymbol{\theta}_{t-1}) + (\boldsymbol{\theta} - \boldsymbol{\theta}_{t-1})^T \nabla G(\boldsymbol{\theta}_{t-1}) + \frac{1}{2\gamma_t} \|\boldsymbol{\theta} - \boldsymbol{\theta}_{t-1}\|_2^2 + H(\boldsymbol{\theta}).$$

Now the central part is to be able to solve such a minimization problem when H is non-smooth, which requires tools to minimize functions of the form $\frac{1}{2\gamma_t} \|\boldsymbol{\theta} - \boldsymbol{\theta}_{t-1}\|_2^2 + H(\boldsymbol{\theta})$. The trick is that for some specific functions H , this minimization

can be carried out (under the name *proximal operator* of H). This is especially true in machine learning, where very often the empirical risk part will be smooth, but the penalty may not, but we have access easily to the proximal operator of the penalty (constraint to stay in a convex set, Lasso penalty and extensions, among others).

3.3 Stochastic gradient descent

We now come back to our initial motivation, the ERM which writes

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta} \in \mathbb{R}^d} F(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n l(y_i, g_{\boldsymbol{\theta}}(\mathbf{x}_i)).$$

If we want to use GD or any other variants mentioned above, at each iteration we need to compute the gradient of F given by

$$\nabla F(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \nabla_{\boldsymbol{\theta}} l(y_i, g_{\boldsymbol{\theta}}(\mathbf{x}_i)).$$

This implies that the gradient computation requires n separate computations of the loss function $\nabla_{\boldsymbol{\theta}} l(y_i, g_{\boldsymbol{\theta}}(\mathbf{x}_i))$: when n is large and / or when storing the samples is expensive (think about the $\mathbf{x}_i$ being images in a neural network designed for image recognition), this may become prohibitive. The idea of stochastic gradient descent is to replace the exact gradient with an approximation, while still keeping convergence guarantees.

The key idea starts from the naive observation that F here writes as an empirical mean, in other words it is an expectation with an underlying simple discrete distribution.

Proposition 9. Denote $F_i(\boldsymbol{\theta}) = l(y_i, g_{\boldsymbol{\theta}}(\mathbf{x}_i))$, such that F writes $F(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n F_i(\boldsymbol{\theta})$. We have

$$F(\boldsymbol{\theta}) = \mathbb{E}[F_{\xi}(\boldsymbol{\theta})]$$

where ξ follows a uniform distribution over the set of indices $\{1, \dots, n\}$. By linearity we also have

$$\nabla F(\boldsymbol{\theta}) = \mathbb{E}[\nabla F_{\xi}(\boldsymbol{\theta})]$$

provided that each $F_i(\boldsymbol{\theta})$ is differentiable.

As you surely recall, a straightforward approach to build an unbiased approximation of the expectation involving a random variable ξ is to perform Monte-Carlo estimation, where the expectation is replaced by an empirical mean over i.i.d. samples ξ_l , $l = 1, \dots, m$ drawn from the distribution of ξ . Such a Monte-Carlo approximation is the *stochastic gradient*.

Definition 17 (Stochastic gradient). The **stochastic gradient** approximation of $\nabla F(\boldsymbol{\theta}) = \mathbb{E}[\nabla F_{\xi}(\boldsymbol{\theta})]$ is

$$g_m(\boldsymbol{\theta}) = \frac{1}{m} \sum_{l=1}^m \nabla F_{\xi_l}(\boldsymbol{\theta})$$

where ξ_l are i.i.d. with uniform distribution over $\{1, \dots, n\}$.

The parameter m allows to reduce the computational cost if it is much smaller than n , at the price of increasing the bias. Now we can formally state what *stochastic gradient descent* is.

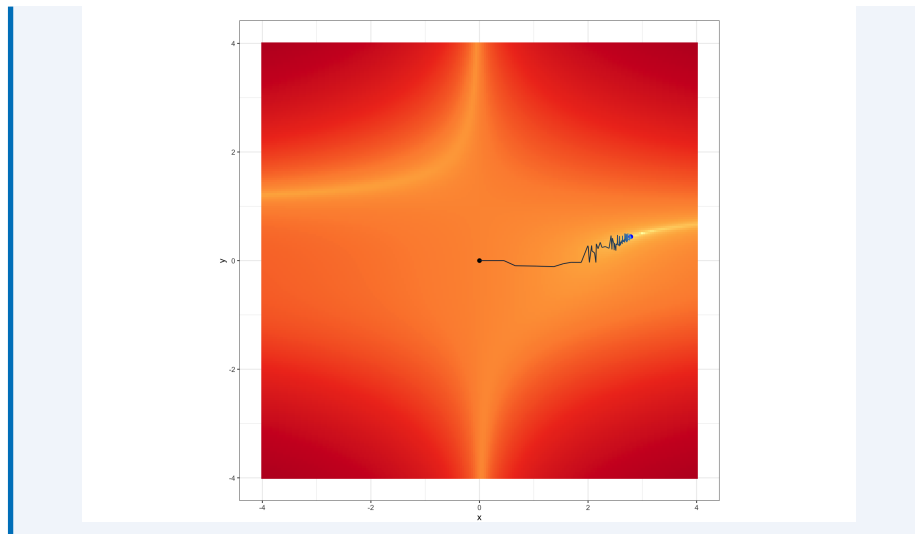
Definition 18 (Stochastic gradient descent). Starting from $\boldsymbol{\theta}_0 \in \mathbb{R}^d$, **stochastic gradient descent** (SGD) builds the sequence

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \gamma_t g_m(\boldsymbol{\theta}_{t-1}), \quad t \geq 1$$

where $g_m(\boldsymbol{\theta})$ is the stochastic gradient defined above, $(\gamma_t)_{t \geq 1}$ is a (well-chosen) **step-size sequence** and t denotes the iteration of SGD.

Remark (SGD or mini-batch SGD?). Some authors limit the terminology "SGD" only to the case where $m = 1$, i.e. we only select one example at random among all n samples. When $m \geq 1$, they use the terminology **mini-batch SGD** in the sense that a small batch of m samples is drawn at random. Here we preferred to use the general terminology for all cases.

Example. We illustrate SGD on the Beale function with a sequence of constant step-size $\gamma = 10^{-1}$ and 100 iterations. The Beale function writes as a sum of 3 terms, so we build the stochastic gradient with $m = 1$ by computing the gradient of only one of those terms chosen at random, at each iteration. The initialization is the black point and the last iterate is the blue one. The true minimizer is $(3, 0.5)$.



Just like for subgradient descent, stochastic gradient descent **is not** a descent method since due to the approximation we cannot guarantee to reduce the objective function at each iteration. A few important remarks still need to be highlighted:

- **Convergence.** As for GD, we can state convergence results towards the minimum, but this time the step-size cannot be taken constant. The obtained bounds and rates will depend on the step-size (as well as the convexity properties of F , obviously). If you are interested in these, you may refer to https://gowerrobert.github.io/pdf/M2_statistique_optimisation/grad_conv.pdf.
- **Practical considerations.** In practice people very often use a constant step-size (as opposed to the theory)! In addition, SGD is at the heart of the training of (deep) neural networks, with highly popular variants that you will learn in the Deep Learning class (AdaGrad, RMSProp, Adam, ...).
- **Variance reduction.** The convergence rates of SGD obviously depend on the variance of the gradient estimator it uses; there is thus an interest in trying to reduce this variance:
 - A straightforward idea consists in increasing m , but this directly increases the iteration complexity.
 - A more general approach builds upon **averaging** ideas, where either past iterates are averaged (Momentum, ...) or past gradients (SVRG, SAG, SAGA, ...).

3.4 Concluding remarks

The optimization methods introduced in this chapter are now ubiquitous in machine learning. If we take a step back, we may conclude with a few additional comments:

- For all methods, we only stated convergence results when the objective function is convex, since without this assumption we may have several local minima (this is the case for neural networks for example). In practice, if you can pay for the additional computational burden (with e.g. parallel computing facilities), you may try to perform *multi-start* optimization, which simply consists in repeating the optimization process several times with a different (random) initialization θ_0 .
- When evaluating the gradient is cheap (for machine learning this would correspond to n small and l cheap) and the dimension d is small (say $d \leq 1000$), BFGS is quite recommended.
- For regularized ERM with non-smooth penalties, proximal methods are now commonly used, except for the specific case of the Lasso penalty, for which coordinate descent is quite efficient - we comment on this below.

We finish this chapter by mentioning a specific optimization approach known as *coordinate descent*, since it is the one used to solve the Lasso regression problem from the previous chapter in the famous `sklearn` Python package.

Definition 19 (Coordinate descent). Starting from $\theta_0 \in \mathbb{R}^d$, **coordinate descent** successively minimizes along coordinate directions to find the minimum of $F : \mathbb{R}^d \rightarrow \mathbb{R}$ as follows.

For k in $1, \dots, d$ (loop along the coordinates):

- $\theta^{(-k)}$ is left unchanged
- The k -th coordinate only is updated through solving the one-dimensional optimization problem:

$$\theta_k = \arg \min_{\theta \in \mathbb{R}^d} F(\theta^{(-k)}, \theta)$$

The above loop is performed several times until convergence, and it is possible to add randomization to select the coordinates instead of the *cyclic* loop introduced here.

This optimization procedure is particularly interesting if the one-dimensional optimization problems inside the loop can be computed efficiently, which is the case for the Lasso regression problem, as illustrated in the example below.

Example. For the Lasso regression problem, the inner one-dimensional problem has an explicit minimum given by:

$$\arg \min_{\theta_k} \frac{1}{2} \|Y - Z\theta\|_2^2 + \lambda \|\theta\|_1 = \eta_{\lambda/\|Z_k\|_2^2} \left(\langle r^{(-k)}, \frac{Z_k}{\|Z_k\|_2^2} \rangle \right)$$

with $r^{(-k)} = Y - Z\theta + \theta_k Z_k$, $Z = (Z_1, \dots, Z_d)$ and η_λ is the **soft-**

thresholding function given by

$$\begin{aligned}\eta_\lambda(x) &= \arg \min_z \frac{1}{2}(z - x)^2 + \lambda|z| \\ &= \operatorname{sgn}(x)(|x| - \lambda)_+\end{aligned}$$

Lecture 3: Support Vector Machines

September 28, 2023

- Reminder on classification
 - Risk & Bayes classifier
 - Linear classifiers & Logistic Regression
- Support Vector Machines
 - Linearly separable case – Hard margin classifier
 - Non-linearly separable case – Soft margin classifier
 - Non-linear extension with kernels
 - The kernel jungle / The kernel zoo
 - Concluding notes
 - * Hyperparameters
 - * Extension to multi-class classification

Chapter 4

Reminder on classification

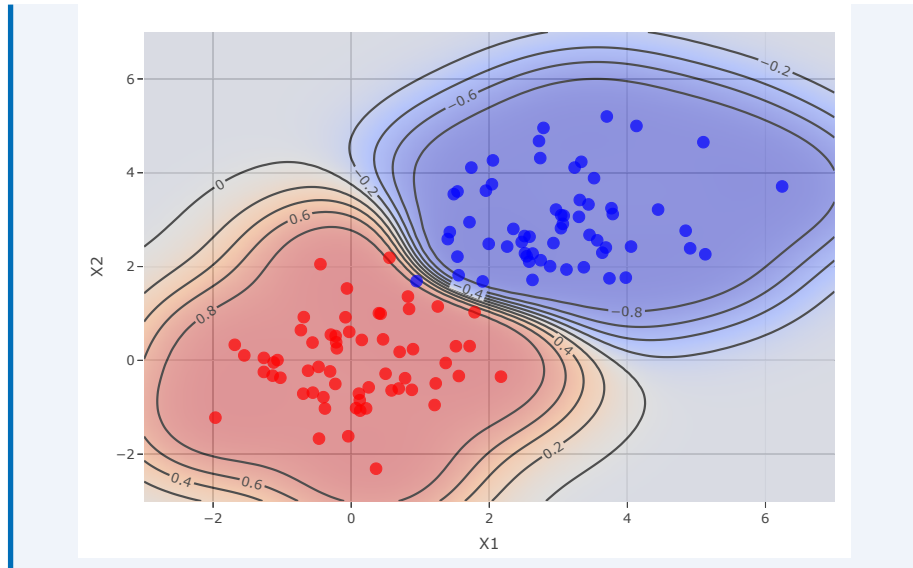
4.1 Problem setting and notations

During this lecture, we will focus on a classification problem, where we are given a training sample $(\mathbf{x}_i, y_i)_{i=1, \dots, n}$ of size n from an unknown joint distribution $\mathbb{P}_{\mathbf{X}Y}$. Here $\mathbf{X} = (X^{(1)}, \dots, X^{(d)}) \in \mathcal{X} \subset \mathbb{R}^d$ is the vector of the d -dimensional features and $Y \in \{C_1, \dots, C_M\}$ is the target variable with values taken from the M classes $\{C_1, \dots, C_M\}$.

Our goal is to build a *classifier* or *classification rule* $h : \mathbb{R}^d \rightarrow \{C_1, \dots, C_M\}$ from the training sample, which predicts the class $h(\mathbf{x})$ of any other individual with features $\mathbf{x}$. In the following, we will focus on **binary** classification, where the target variable can only take two values. Be aware that in the next sections these two labels will be encoded differently depending on the mathematical context (mainly $\{0, 1\}$ and $\{-1, +1\}$), but we will always make clear which encoding is being used.

Any classifier h in binary classification can be written through a so-called **decision function** $f(\mathbf{x})$, which serves as an intermediary to predict the class. When $f(\mathbf{x}) > 0$, the classifier takes the value $h(\mathbf{x}) = +1$ and when $f(\mathbf{x}) < 0$ we have $h(\mathbf{x}) = -1$ when using the $\{-1, +1\}$ label encoding. This means that in this case we can write the classifier as $h(\mathbf{x}) = \text{sign}(f(\mathbf{x}))$ where $\text{sign}(a) = a/|a|$ is the **sign** function. The set of feature values $\{\mathbf{x} \in \mathbb{R}^d : f(\mathbf{x}) = 0\}$ corresponding to decision function values equal to 0 is known as the **decision boundary**.

Example (Illustration of the decision function in two dimensions). We give below an example of a decision function with a training sample of size 120 with two features $X^{(1)}$ and $X^{(2)}$ and a target variable Y with values in $\{-1, +1\}$. Samples from each class are represented in red (+1) and blue (-1). We draw the level set values of the decision function. Observe in particular in the middle the decision boundary corresponding to a 0 value.



Recall that the **risk** $R(h)$ of a classifier h is defined through a **loss function** $l(y, y')$:

$$R(h) = \mathbb{E}_{\mathbb{P}_{\mathbf{X}Y}} [l(Y, h(\mathbf{X}))].$$

When the loss function is equal to the 0-1 loss $l(y, y') = \mathbb{1}_{y \neq y'}$, we can show that the binary classifier h^* that minimizes the risk relies on a decision function related to the conditional probability of the target variables given the features.

Definition 20 (Bayes classifier). For binary classification, the classifier h^* that minimizes the risk $R(h)$ with a loss function $l(y, y') = \mathbb{1}_{y \neq y'}$ is the **Bayes classifier** defined as

$$h^*(\mathbf{x}) = \begin{cases} 1 & \eta(\mathbf{x}) > \frac{1}{2} \\ 0 & \text{otherwise} \end{cases}$$

where $\eta(\mathbf{x}) = \mathbb{P}(Y = 1 | \mathbf{X} = \mathbf{x})$.

Here the two classes are encoded as $\{0, 1\}$.

The Bayes classifier is thus the best possible classifier with respect to the risk. Unfortunately its expression depends on the unknown joint distribution $\mathbb{P}_{\mathbf{X}Y}$, meaning that it is out of reach in practice since we only observe a n -sample $(\mathbf{x}_i, y_i)_{i=1, \dots, n}$ from $\mathbb{P}_{\mathbf{X}Y}$. Most machine learning algorithms for classification aim at building an approximation of the Bayes classifier h^* from this training sample. Since $\mathbb{P}_{\mathbf{X}Y}$ is unknown and that the risk cannot be computed, they typically rely on the **empirical risk** $\hat{R}(h)$ defined as the empirical mean of the loss function on the training sample instead of the true expectation with respect to $\mathbb{P}_{\mathbf{X}Y}$:

$$\hat{R}(h) = \frac{1}{n} \sum_{i=1}^n l(y_i, h(\mathbf{x}_i)).$$

The difference between all of them lies in the assumptions they make, either on

the joint distribution $\mathbb{P}_{\mathbf{X}Y}$, on the form of the classifier h they use to approximate h^* and on the loss function appearing in the risk. In the next section, we recall some basic algorithms which belong to the class of linear classifiers.

4.2 Linear classifiers

We first start by recalling what linear classifiers are.

Definition 21 (Linear classifier). A linear classifier h is a classifier with a decision function f such that the decision boundary $\{\mathbf{x} \in \mathbb{R}^d : f(\mathbf{x}) = 0\}$ is linear.

The following remark is trivial but essential.

Remark. A linear classifier does not necessarily require the decision function f itself to be linear.

In particular, any monotonic transformation of a linear function produces a decision function which is no longer linear, but with a linear decision boundary.

The first linear classifier you should have seen during the Supervised Learning course in 2nd year is the Bayesian Linear Discriminant Analysis (LDA) one. In this model, the main assumption is that for each class label, the features are distributed according to a multivariate normal distribution, i.e.

$$\mathbb{P}(\mathbf{x}|Y = C_m) \sim \mathcal{N}_{\mathbf{x}}(\mu_m, \Sigma).$$

The LDA classifier is obtained by replacing inside the Bayes classifier the value of $\eta(\mathbf{x})$ which derives from this assumption:

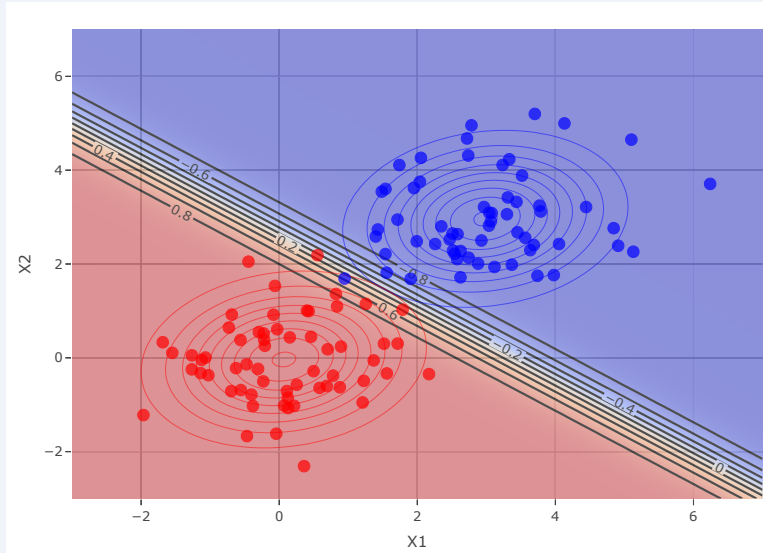
$$\begin{aligned} \eta_{\text{LDA}}(\mathbf{x}) &= \mathbb{P}_{\text{LDA}}(Y = 1 | \mathbf{X} = \mathbf{x}) = \frac{\mathbb{P}_{\text{LDA}}(X = \mathbf{x} | Y = 1) \pi_1}{\sum_{m=0}^1 \mathbb{P}_{\text{LDA}}(X = \mathbf{x} | Y = m) \pi_m} \\ &= \frac{\mathcal{N}_{\mathbf{x}}(\mu_1, \Sigma) \pi_1}{\sum_{m=0}^1 \mathcal{N}_{\mathbf{x}}(\mu_m, \Sigma) \pi_m} \end{aligned}$$

by Bayes theorem and the law of total probability, where $\pi_m := \mathbb{P}(Y = m)$. The decision boundary then corresponds to $\eta_{\text{LDA}}(\mathbf{x}) = 1/2$, or equivalently

$$\begin{aligned} \mathcal{N}_{\mathbf{x}}(\mu_1, \Sigma) \pi_1 &= \mathcal{N}_{\mathbf{x}}(\mu_0, \Sigma) \pi_0 \\ -\frac{1}{2}(\mathbf{x} - \mu_1)^T \Sigma^{-1}(\mathbf{x} - \mu_1) + \log \pi_1 &= -\frac{1}{2}(\mathbf{x} - \mu_0)^T \Sigma^{-1}(\mathbf{x} - \mu_0) + \log \pi_0 \\ \mathbf{x}^T \Sigma^{-1}(\mu_1 - \mu_0) - \frac{1}{2} \mu_1^T \Sigma^{-1} \mu_1 + \frac{1}{2} \mu_0^T \Sigma^{-1} \mu_0 + \log \frac{\pi_1}{\pi_0} &= 0 \end{aligned}$$

which is the equation of an hyperplane as expected. Note that in practice μ_k , π_k and Σ are estimated with the training sample. We give below an example of Bayesian LDA on the dataset already used above.

Example (Illustration of Bayesian LDA). The linear decision boundary from Bayesian LDA corresponds to the line in the middle, and the estimated Gaussian for each class is represented with the blue and red ellipses (level set).



Remark. If for each class label we assume different covariance matrices Σ_k in the formula above, this leads to Bayesian Quadratic Discriminant Analysis which involves a quadratic decision boundary. The equation of the decision boundary is left as an exercise.

As opposed to Bayesian LDA, another approach is to not make such strong assumptions on $\mathbb{P}_{\mathbf{X}Y}$, but instead build a so called *plug-in classification rule* $\hat{h}$ defined as

$$\hat{h}(\mathbf{x}) = \begin{cases} 1 & \hat{\eta}(\mathbf{x}) > \frac{1}{2} \\ -1 & \text{otherwise} \end{cases}$$

where $\hat{\eta}(\mathbf{x})$ is an estimate of $\eta(\mathbf{x})$ built such that the empirical risk is minimized. If we focus on linear classifiers, the first intuitive idea is to propose a **parametric form** of $\hat{\eta}(\mathbf{x})$ such that the decision boundary is linear. Following what you saw for regression problems, you may be tempted to consider for example

$$\hat{\eta}(\mathbf{x}) = \beta_0 + \mathbf{x}^T \beta.$$

This will obviously produce a linear decision boundary, but since $\eta(\mathbf{x})$ is a (conditional) probability it may not be suited to use such an unconstrained parametric form (that does not guarantee values between 0 and 1). Instead, it is wiser to build upon a monotonic transformation following our previous remark. For example, we may write

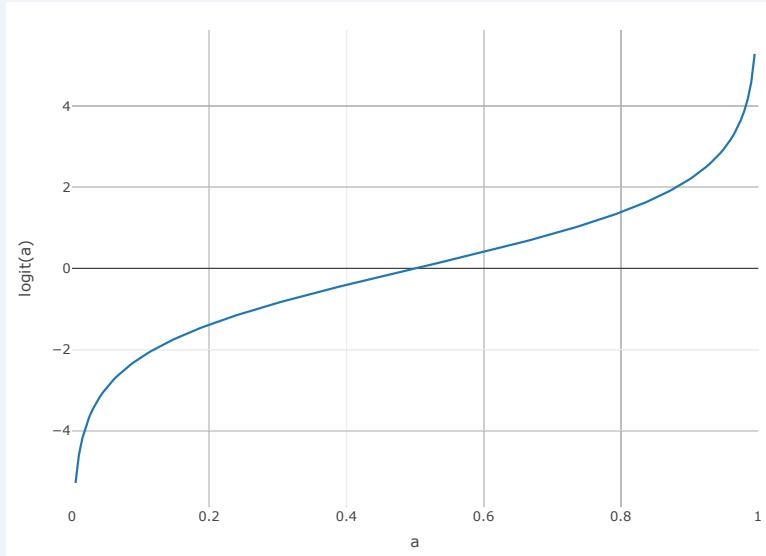
$$\text{logit}(\hat{\eta}(\mathbf{x})) = \beta_0 + \mathbf{x}^T \beta$$

where $\text{logit}(a) = \log(a/(1-a))$, $a \in [0, 1]$ is the so-called *logit* function, see the example below for a visualization of this function. Equivalently, we have

$$\hat{\eta}(\mathbf{x}) = \frac{\exp(\beta_0 + \mathbf{x}^T \beta)}{1 + \exp(\beta_0 + \mathbf{x}^T \beta)}.$$

The logit function is monotonic, and so is its inverse, meaning that the corresponding plug-in classification rule will have a linear decision boundary.

Example (Logit function). Here is the shape of the logit function.



Now the last step is to find the best values for the parameters β_0 and β , and it is natural to try to minimize the empirical risk. If we consider the 0-1 loss, the corresponding empirical risk minimization problem writes

$$\min_{\beta_0, \beta} \hat{R}(\hat{h}) = \frac{1}{n} \sum_{i=1}^n \mathbb{1}_{\text{sign}(\beta_0 + \mathbf{x}_i^T \beta) \neq y_i}.$$

But here the indicator function is neither continuous nor differentiable, making the optimization problem hard to solve. A workaround is to consider a so-called *surrogate loss*, i.e. a loss function which penalizes classification errors similarly to the 0-1 loss while being smoother. A popular surrogate loss is the **cross-entropy loss** l_{CE} , given by

$$l_{\text{CE}}(y, \eta(\mathbf{x})) = -\mathbb{1}_{y=0} \log(1 - \eta(\mathbf{x})) - \mathbb{1}_{y=1} \log(\eta(\mathbf{x})).$$

When applied to our parametric model, this leads to the new empirical risk minimization problem with $\{0, 1\}$ labels:

$$\min_{\beta_0, \beta} \hat{R}(\hat{h}) = \frac{1}{n} \sum_{i=1}^n [\log(1 + \exp(\beta_0 + \mathbf{x}_i^T \beta)) - y_i(\beta_0 + \mathbf{x}_i^T \beta)].$$

The proof is left as an exercise. You may recognize this minimization problem from your Generalized Linear Regression course in 2nd year, it is called Logistic Regression (because it uses the logit function).

Remark. On a side note, you certainly derived it from another point of view. Indeed the same estimator can also be recovered with a (conditional) maximum likelihood approach by considering that Y is a Bernoulli random variable with probability $\hat{\eta}(\mathbf{x})$.

A few practical words about Logistic Regression:

- There is no closed-form solution to the minimization problem as opposed to OLS.
- But the loss function is convex, so you can use gradient descent, stochastic gradient descent or even more advanced optimizers.
- For high-dimensional problems, you can also use regularization with L^1 or L^2 penalties (you will have the opportunity to try that during the last computer class assignment).

Chapter 5

Support Vector Machines

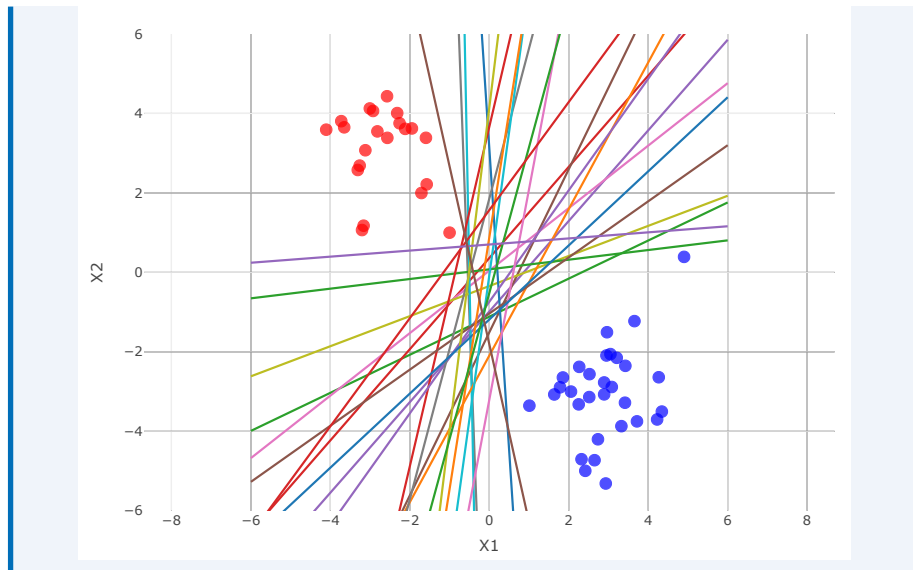
Now that we have seen a brief summary of linear classifiers, we can take a step back and ask ourselves the kind of decision boundaries they can produce.

We will first assume that the training sample is *linearly separable*, meaning that there exists a hyperplane which can perfectly separate the two classes. This simplified situation will guide our intuition towards a linear classifier which should be robust, and help us design the *hard margin classifier*. When no perfect linear separation is possible, we will then extend the underlying ideas to the *soft margin classifier*. Finally, to account for even more non-linearity, we will introduce the concept of feature functions and kernels. The theory of kernels will be briefly tackled during Lecture 2.

5.1 Linearly separable case

To begin with, we will assume that the training sample is *linearly separable*, meaning that there exists a hyperplane which can perfectly separate the two classes. From a classification perspective, this means that we could train many linear classifiers with an empirical risk equal to 0 (i.e. they do not make classification errors). For instance, we may encounter the following situation depicted in the example below.

Example. We depict a linearly separable case where several linear classifiers have been trained on the training samples in red (+1) and blue (−1) according to their class. The different classifiers are represented with different colors.



By looking at their empirical risk, it is impossible to choose the 'best' one since they all have zero classification error on the training set. However, intuitively we could argue that from a generalization perspective (i.e. the classification errors made on an independent test set), it may be wiser to select a linear classifier with a decision boundary lying as far as possible to the training samples. This intuition is formalized mathematically through the concept of *margin*.

Definition 22 (Margin). The **margin** γ of a separating hyperplane is the distance between this hyperplane and the closest training sample.

The main idea is thus to find the hyperplane with the largest margin, while still separating perfectly the training sample. In the following, when we will mention a maximal or optimal margin, it will always imply that it is optimal among all hyperplanes which make zero classification errors. Before diving into the mathematical details, let us comment a few points (we will only consider the $\{-1, +1\}$ encoding below):

- Previously we have denoted a linear decision function as $f(\mathbf{x}) = \beta_0 + \mathbf{x}^T \beta$, but now following standard textbooks on support vector machines we will use the equivalent notation $f(\mathbf{x}) = b + \langle \mathbf{x}, \mathbf{w} \rangle$ where $\langle \cdot, \cdot \rangle$ is the Euclidean dot product in $\mathbb{R}^d$;
- When the margin is maximal, there is at least one $+1$ sample and one -1 sample with a distance equal to the margin γ (this is easy to prove);
- We will denote H the hyperplane with maximum margin, H^+ the hyperplane parallel to H at a distance γ on the $+1$ side and similarly H^- the hyperplane parallel to H at a distance γ on the -1 side;
- Training samples with a distance to H equal to the maximum margin γ are called **support vectors**, the terminology coming from the fact that

they support H^+ and H^- ;

- Multiplying b and $\mathbf{w}$ by a constant does not change the hyperplane equation. So for identification purposes we need to impose a scaling, and choose the following standard one:

$$\begin{aligned} b + \langle \mathbf{x}^+, \mathbf{w} \rangle &= +1, \quad \forall \mathbf{x}^+ \in H^+ \\ b + \langle \mathbf{x}^-, \mathbf{w} \rangle &= -1, \quad \forall \mathbf{x}^- \in H^- \end{aligned}$$

by symmetry of H^+ and H^- .

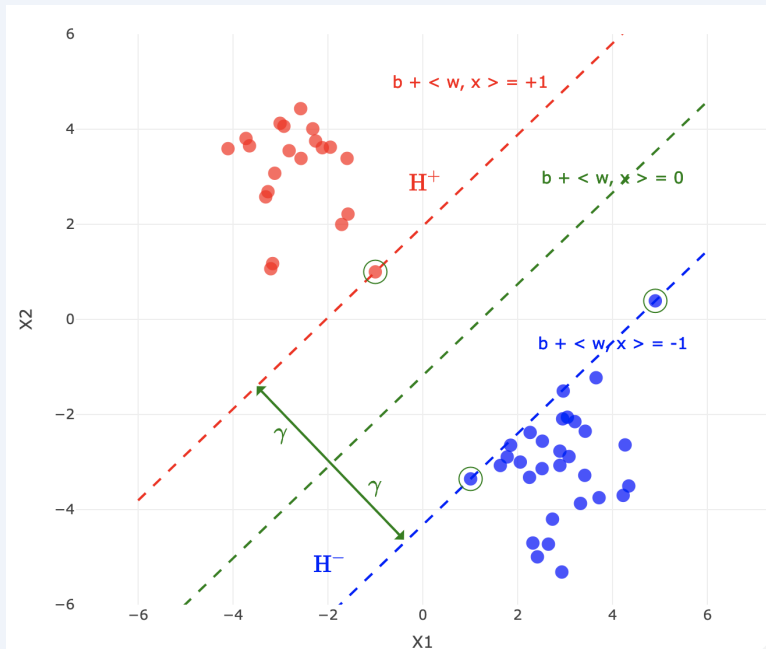
- With this scaling, the optimal margin is equal to

$$\gamma = \frac{1}{\|\mathbf{w}\|_2}.$$

The proof is left as an exercise (hint: consider two points on H^+ and H^- which are symmetric with respect to H and compute their distance).

We represent below all the introduced notations on a simple two-dimensional example.

Example. In the same example as above, the decision boundary with maximum margin is depicted in dashed green, while the H^+ and H^- hyperplanes are represented in dashed red and blue, respectively. Support vectors are identified with green circles.



Wrapping things up, we can now write the final optimization problem we have to solve:

1. We want to maximize $\gamma = 1/\|w\|_2$, **this is equivalent to minimizing** $\frac{1}{2}\|w\|_2^2$...
2. ... while respecting that the hyperplane separates the classes, i.e.

$$\begin{aligned} \forall i = 1, \dots, n \quad \text{sign}(f(\mathbf{x}_i)) &= y_i \\ \iff y_i f(\mathbf{x}_i) &\geq 1 \\ \iff y_i (b + \langle \mathbf{x}_i, \mathbf{w} \rangle) &\geq 1 \end{aligned}$$

Optimization Problem (Linearly separable maximum margin problem – Primal form). When the training sample is linearly separable, the maximum margin hyperplane $f(\mathbf{x}) = b + \langle \mathbf{x}, \mathbf{w} \rangle$ is obtained by solving

$$\begin{aligned} \min_{\mathbf{w}, \beta} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 \\ \text{subject to} \quad & y_i (b + \langle \mathbf{x}_i, \mathbf{w} \rangle) \geq 1 \quad \forall i = 1, \dots, n. \end{aligned}$$

This is a quadratic optimization problem in dimension $d + 1$ with n linear inequality constraints. Usually, we consider instead the *dual problem* (i.e. with Lagrange multipliers) which leads to the following equivalent optimization problem.

Optimization Problem (Linearly separable maximum margin problem – Dual form). When the training sample is linearly separable, the maximum margin hyperplane $f(\mathbf{x}) = b + \langle \mathbf{x}, \mathbf{w} \rangle$ is obtained by solving the dual problem

$$\begin{aligned} \min_{\alpha_1, \dots, \alpha_n} \quad & \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle \\ \text{subject to} \quad & 0 \leq \alpha_i \quad \forall i = 1, \dots, n \\ \text{and} \quad & \sum_{i=1}^n \alpha_i y_i = 0. \end{aligned}$$

The optimal hyperplane is retrieved with

- $\mathbf{w}^* = \sum_{i=1}^n \alpha_i^* y_i \mathbf{x}_i$;
- b^* is obtained with the scaling constraints for support vectors;
- The training samples with $\alpha_i^* > 0$ correspond to **support vectors**;
- By plugging $\mathbf{w}^*$ inside f , the final decision boundary writes

$$f^*(\mathbf{x}) = \sum_{i=1}^n \alpha_i^* y_i \langle \mathbf{x}_i, \mathbf{x} \rangle + b^*.$$

In particular, **only support vectors contribute to the decision boundary** (since all other training samples have $\alpha_i^* = 0$).

The proof is left as an exercise. This time, this is a quadratic optimization problem in dimension n with one linear equality constraint. Since this classifier makes zero classification errors and leaves no sample at a distance to the hyperplane less than the margin, it is called a **hard margin classifier**.

5.2 Non-linearly separable case

What we just saw before is only suited if classes are linearly separable, since the previous optimization problems do not have a solution if this is not the case. The main question now is: How can we generalize the previous ideas to the non-separable case?

Ingredient 1. We still keep the idea of maximizing $1/\|w\|_2$

Ingredient 2. But we will authorize classification errors, meaning that some examples will be on the wrong side of the hyperplane, and as well tolerate samples which are classified correctly but which are at a distance smaller than $1/\|w\|_2$ (inside the margin). But of course we want these errors to be as small as possible.

As was done for logistic regression, we will minimize the errors by minimizing the empirical risk with a loss function. Recall that a linear classifier writes $h(\mathbf{x}) = \text{sign}(f(\mathbf{x})) = \text{sign}(b + \langle \mathbf{x}, \mathbf{w} \rangle)$ for the $\{-1, +1\}$ encoding. Previously, we talked about:

- The 0-1 loss $\mathbb{1}_{y \neq h(\mathbf{x})}$, which can be rewritten $\mathbb{1}_{yf(\mathbf{x}) < 0}$;
- The cross-entropy loss, which can be rewritten $\log(1 + \exp(yf(\mathbf{x})))$.

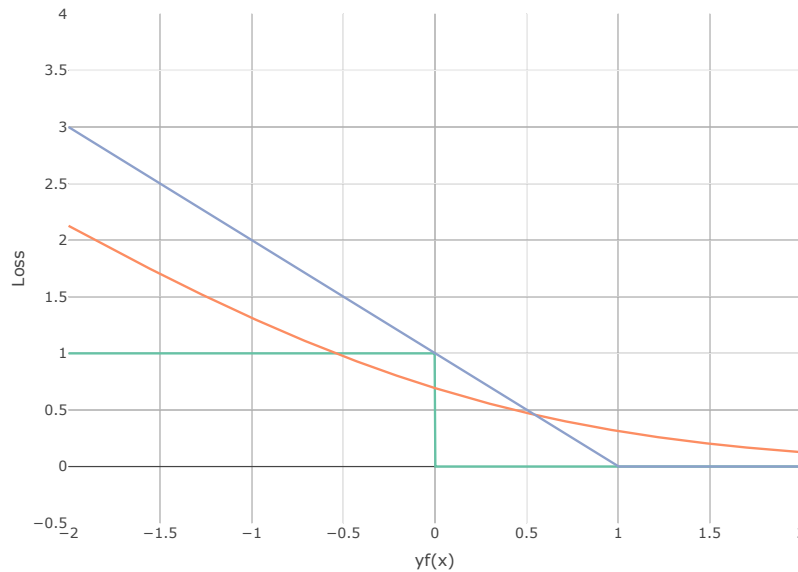
For support vector machines, another loss function was introduced, the *hinge loss*.

Definition 23 (Hinge loss). With the $\{-1, +1\}$ encoding, the **hinge loss** of a binary classifier $h(\mathbf{x}) = \text{sign}(f(\mathbf{x}))$ is given by

$$\max(0, 1 - yf(\mathbf{x})) = [1 - yf(\mathbf{x})]_+$$

where $[\cdot]_+$ is the positive part function.

We give below a graphical comparison of these three loss functions (0-1 in green, cross-entropy in orange and hinge in blue).



Remark. A justification for using the hinge loss is sometimes found in textbooks: this is the smallest convex function which lies above the 0-1 loss.

We can now come back to our original problem: we want to maximize the margin and at the same time minimize the empirical risk measured with the hinge loss. This is achieved by defining an objective function as a weighted combination of these two objectives:

$$\min_{\mathbf{w}, \beta} \frac{1}{2} \|\mathbf{w}\|_2^2 + \frac{C}{n} \sum_{i=1}^n [1 - y_i f(\mathbf{x}_i)]_+.$$

Here, the constant $C > 0$ determines a **trade-off** between increasing the margin and ensuring that the training samples lie on the correct side of the hyperplane. To make a connection with what you saw in the previous class, we can rewrite the problem as

$$\min_{\mathbf{w}, \beta} \frac{1}{n} \sum_{i=1}^n [1 - y_i f(\mathbf{x}_i)]_+ + \lambda \|\mathbf{w}\|_2^2$$

where $\lambda = 1/2C$ is a regularization parameter for an **empirical risk minimization** with **L_2 regularization**. This looks a lot like Ridge Regression, but with another loss function specific to classification (the hinge loss).

This optimization problem involves a continuous but non-differentiable loss function (due to the hinge loss). Even if now more modern approaches have been proposed to directly handle it (sub-gradient or stochastic sub-gradient descent), below we will detail the original proposal from the literature on support vector machines, which is the one implemented in most legacy packages. It is based on the classical trick to add *slack variables* to the optimization problem.

Optimization Problem (Non-linearly separable maximum margin problem – Primal form). When allowing classification errors, the maximum margin hyperplane $f(\mathbf{x}) = b + \langle \mathbf{x}, \mathbf{w} \rangle$ is obtained by solving

$$\begin{aligned} \min_{\mathbf{w}, \beta, \xi_1, \dots, \xi_n} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 + \frac{C}{n} \sum_{i=1}^n \xi_i^2 \\ \text{subject to} \quad & \xi_i \geq 0 \quad \forall i = 1, \dots, n. \\ \text{and} \quad & y_i (b + \langle \mathbf{x}_i, \mathbf{w} \rangle) \geq 1 - \xi_i \quad \forall i = 1, \dots, n. \end{aligned}$$

where the ξ_i 's are **slack variables**.

This is a quadratic optimization problem in dimension $d + n + 1$ with n linear inequality constraints. As before, we will prefer working with its dual formulation.

Optimization Problem (Non-linearly separable maximum margin problem – Dual form). When allowing classification errors, the maximum margin hyperplane $f(\mathbf{x}) = b + \langle \mathbf{x}, \mathbf{w} \rangle$ is obtained by solving the dual problem

$$\begin{aligned} \min_{\alpha_1, \dots, \alpha_n} \quad & \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle \\ \text{subject to} \quad & 0 \leq \alpha_i \leq \frac{C}{n} \quad \forall i = 1, \dots, n \\ \text{and} \quad & \sum_{i=1}^n \alpha_i y_i = 0. \end{aligned}$$

An **important remark** is that this dual problem is exactly the same as in the linear separable case **with just the additional bound constraints** $\alpha_i \leq \frac{C}{n}$.

The optimal hyperplane is retrieved with

- $\mathbf{w}^* = \sum_{i=1}^n \alpha_i^* y_i \mathbf{x}_i$;
- b^* is obtained with the scaling constraints for support vectors;
- The training samples with $\alpha_i^* > 0$ correspond to **support vectors**;
- By plugging $\mathbf{w}^*$ inside f , the final decision boundary writes

$$f^*(\mathbf{x}) = \sum_{i=1}^n \alpha_i^* y_i \langle \mathbf{x}_i, \mathbf{x} \rangle + b^*.$$

In particular, **only support vectors contribute to the decision boundary** (since all other training samples have $\alpha_i^* = 0$).

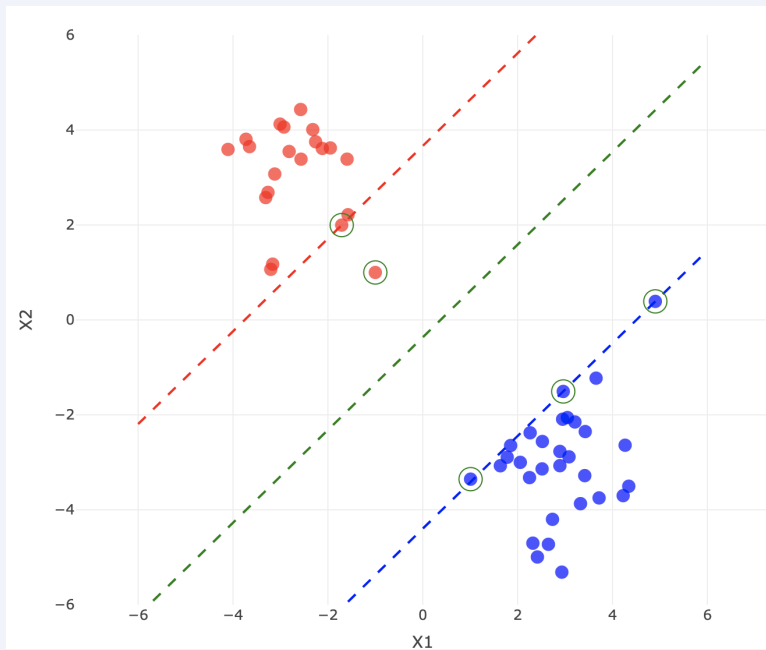
Contrary to the previous hard-margin classifier, since now the classifier is allowed to make misclassifications and tolerates samples at a distance to the hyperplane less than the margin, it is called a **soft margin classifier**.

As mentioned before, solving this dual problem is what is done in legacy libraries, with specific ultra-optimized solvers. A famous library is **LIBSVM**, which is in particular used in **scikit-learn**.

Concerning the choice of C , it is typically found with cross-validation following standard practice for regularized problems. But note that there is a support vector machines variant called ν -SVM where C is replaced with a more interpretable hyperparameter ν (number of margin errors and support vectors).

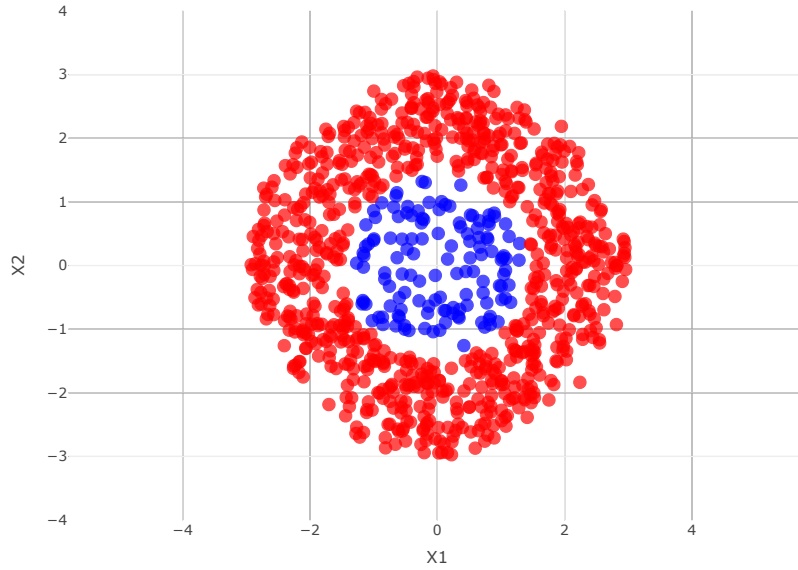
Finally, even if data are linearly separable, using a soft margin classifier instead of the hard one may be beneficial! We discuss the example presented at the beginning of the Section below.

Example. In the same example as above, the red support vector in the middle implies that the hard margin hyperplane in the previous figure is closer to the -1 blue class than to the $+1$ class in average. The red support vector is not really an outlier, but a value more 'extreme' than the other $+1$ samples. In such a case to balance the classes, a soft margin classifier, allowing the $+1$ support vector in the middle to be at a distance smaller than γ , may produce a more robust hyperplane. We show below how it looks like.



5.3 Non-linear extension with kernels

Now you know how to deal with the non-separable case, by balancing the margin and the classification errors. That is great, but do you think what you have learned would perform well if the training samples look like this?



Every linear classifier will make a lot of classification errors in this case! But here, in this specific example, a boundary decision function like

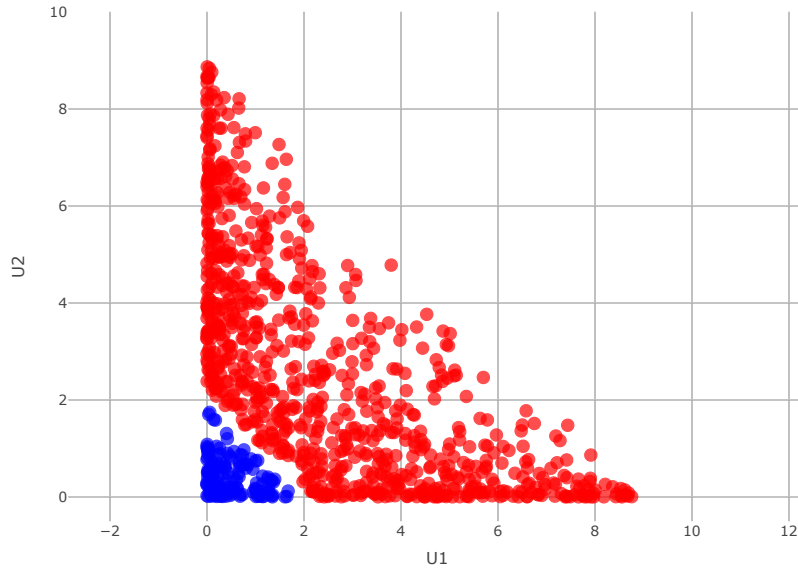
$$f(\mathbf{x}) = (x^{(1)})^2 + (x^{(2)})^2 - r^2,$$

would lead to a much better classifier. Of course, f is not linear in $x^{(1)}$ and $x^{(2)}$. But it is linear in $(x^{(1)})^2$ and $(x^{(2)})^2$! Now if we define the function

$$\begin{aligned} \phi: \mathbb{R}^2 &\rightarrow \mathbb{R}^2 \\ (x^{(1)}, x^{(2)}) &\mapsto (u^{(1)} = (x^{(1)})^2, u^{(2)} = (x^{(2)})^2), \end{aligned}$$

then f is linear in $\phi(\mathbf{x})^{(1)}$ and $\phi(\mathbf{x})^{(2)}$.

This remark thus gives us a straightforward method to recycle what we developed for linear classifiers: we first transform the training sample through function ϕ , meaning that we replace the original features $X^{(1)}$ and $X^{(2)}$ with $U^{(1)} = \phi(\mathbf{X})^{(1)}$ and $U^{(2)} = \phi(\mathbf{X})^{(2)}$, see how the training samples look like after such transformation below.



In this transformed space, the samples can easily be classified with a linear boundary decision function. From a practical point of view, in our previous optimization problems we just need to do one thing: everytime an $\mathbf{x}_i$ appears, we simply replace it with the transformed sample $\phi(\mathbf{x}_i)$, the image of the original sample $\mathbf{x}_i$ through the function ϕ . For example, the soft margin dual problem is now

$$\begin{aligned} \min_{\alpha_1, \dots, \alpha_n} \quad & \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle_{\mathcal{H}} \\ \text{subject to} \quad & 0 \leq \alpha_i \leq \frac{C}{n} \quad \forall i = 1, \dots, n \\ \text{and} \quad & \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned}$$

and the final decision function writes

$$f^*(\mathbf{x}) = \sum_{i=1}^n \alpha_i^* y_i \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}) \rangle_{\mathcal{H}} + b^*.$$

Note that we added a subscript $\mathcal{H}$ for the dot product, we will comment this point soon below.

We now give a few words about the function ϕ :

- (1) In our example, we have defined ϕ **manually**, being guided by the training samples.
This was easy because we were just in dimension two (easy visualization) and the pattern was simple (a circle). But in general, this would be much more complicated! So this manual trick is not really generalizable.
- (2) Related to the above, we just used $\phi : \mathcal{X} \rightarrow \mathbb{R}^2$ with $\mathbb{R}^2$ only because it was easy to identify.
In general, we may want to build a much larger number of new features,

with the hope that with some of them the data will become linearly separable. In other words, in a high-dimensional new space, we expect to increase the chances of a linear separation. We even can (and will do!) use a function $\phi : \mathcal{X} \rightarrow \mathcal{H}$ going to an infinite-dimensional space $\mathcal{H}$.

- (3) But to have the right to do so with our previous results, $\mathcal{H}$ cannot be any space. Since ϕ is only involved in scalar products, we only need $\mathcal{H}$ to be an Hilbert space.
- (4) However specifying a (potentially) high-dimensional ϕ is cumbersome. But we have observed that ϕ only appears through dot products in both the optimization problem and the final decision function, so that we only have terms like

$$\langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle_{\mathcal{H}}.$$

Let us write such dot products as

$$k(\mathbf{x}, \mathbf{x}') := \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle_{\mathcal{H}}.$$

Here k is a function $\mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ which is symmetric and positive definite (due to the properties of dot products), which is called a **kernel**.

- (4.1) From a computational and practical perspective, this means that we only need to specify the kernel k instead of ϕ (as long as it is symmetric and positive definite). This is easier, because:
- It is defined on $\mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ versus $\phi : \mathcal{X} \rightarrow \mathcal{H}$ where $\mathcal{H}$ is high-dimensional;
 - It can be intuitively interpreted as a dot product in an Hilbert space, that is to say a **function which measures the similarity between objects in $\mathcal{X}$** (the feature space). This means for example that in classification we can easily handle features that are more 'exotic' than simple real variables, such as images, text sequences, time series and so on, as long as we are able to provide a similarity measure between them in the form of the kernel.
- (4.2) From a theoretical perspective, specifying just the kernel k is also justified by the so-called **Moore-Aronszajn theorem**:

Theorem 3 (Moore-Aronszajn). For any symmetric positive definite function $\kappa : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$, there exists a unique Hilbert space $\mathcal{F}$ and a function ψ such that for all $\mathbf{x}, \mathbf{x}' \in \mathcal{X} \times \mathcal{X}$,

$$\kappa(\mathbf{x}, \mathbf{x}') := \langle \psi(\mathbf{x}), \psi(\mathbf{x}') \rangle_{\mathcal{F}}.$$

Note that $\mathcal{F}$ is unique, but ψ is not.

- (5) Such a kernel k is at the heart of the famous class of **kernel** methods in machine learning and the famous **kernel trick**. This kernel trick is simple to state: everytime you have an algorithm which involves dot products $\langle \mathbf{x}, \mathbf{x}' \rangle$ for features $\mathbf{x}$ (like SVM, but also PCA, and also Ridge Regression, more on this next class), meaning that

this is a linear algorithm, you can build a nonlinear counterpart by using $k(\mathbf{x}, \mathbf{x}')$ instead of $\langle \mathbf{x}, \mathbf{x}' \rangle$. Internally this will correspond to a nontrivial transformation ϕ of the features that you do not need to specify.

- (6) Now coming back to SVM, the dual problem with the kernel k writes in the following form.

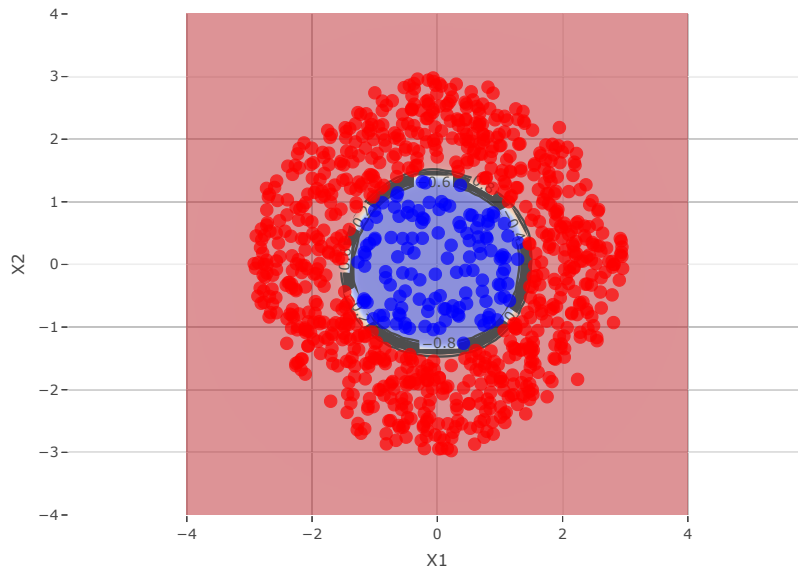
Optimization Problem (Non-linearly separable maximum margin problem with a kernel k – Dual form). When allowing classification errors, the maximum margin hyperplane with a kernel k is obtained by solving the dual problem

$$\begin{aligned} \min_{\alpha_1, \dots, \alpha_n} \quad & \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j k(\mathbf{x}_i, \mathbf{x}_j) \\ \text{subject to} \quad & 0 \leq \alpha_i \leq \frac{C}{n} \quad \forall i = 1, \dots, n \\ \text{and} \quad & \sum_{i=1}^n \alpha_i y_i = 0. \end{aligned}$$

The final decision boundary writes

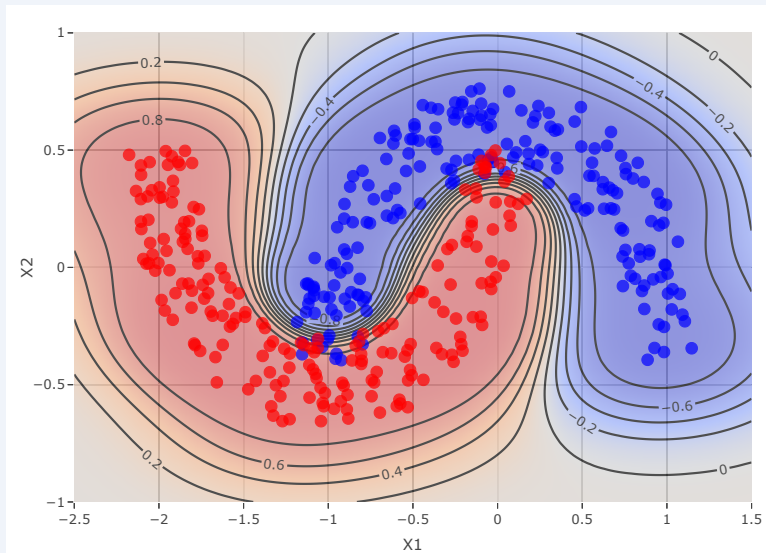
$$f^*(\mathbf{x}) = \sum_{i=1}^n \alpha_i^* y_i k(\mathbf{x}_i, \mathbf{x}) + b^*.$$

If you now use a kernel on our motivating example (instead of specifying a function ϕ), we obtain the following decision boundary. We choose here the Gaussian kernel (see the next section for the definition).



We also illustrate SVM on a famous classification dataset below.

Example (Two Moons dataset). The Two Moons dataset is famous in classification and clustering due to its specific form, where each class is arranged along a moon shape : it is not linearly separable, and any linear classifier will make misclassification errors in the central area. But we can use SVM with a Gaussian kernel, here are the level sets of the decision function.



Finally, now that we have simplified the problem by replacing ϕ with k , how do you specify it in practice? A practical answer is given in the next section.

5.4 The kernel jungle / The kernel zoo

Fortunately, at your level and in the scope of this class, things are quite easy for you because many smart people have worked on this subject and there is a list of **commonly used kernel expressions**. This list of kernels is usually referred to as the **kernel jungle** or **kernel zoo**. Here we will limit ourselves to $\mathcal{X} = \mathbb{R}^d$, and a typical list of kernels in this case is the following:

- (a) The **vanilla dot product** or **linear** kernel

$$k(\mathbf{x}, \mathbf{x}') = \langle \mathbf{x}, \mathbf{x}' \rangle.$$

Ok with this one, you just do standard linear SVM.

- (b) The **polynomial** kernel

$$k(\mathbf{x}, \mathbf{x}') = (a + \langle \mathbf{x}, \mathbf{x}' \rangle)^p.$$

Not very powerful in practice, because it corresponds to a finite-dimensional Hilbert space.

- (c) The **sigmoid** or **hyperbolic tangent** or **neural network** kernel

$$k(\mathbf{x}, \mathbf{x}') = \tanh(a + \langle \mathbf{x}, \mathbf{x}' \rangle)$$

or more generally

$$k(\mathbf{x}, \mathbf{x}') = \tanh(a + \kappa(\mathbf{x}, \mathbf{x}')).$$

It had quite a success in the 'old days', despite the fact that it is not a kernel because it is not positive definite!

- (d) The **Gaussian** or **RBF** (Radial Basis Function) or **Squared-Exponential** kernel

$$\begin{aligned} k(\mathbf{x}, \mathbf{x}') &= \exp\left(-\frac{1}{2\theta^2}\|\mathbf{x} - \mathbf{x}'\|_2^2\right) \\ &= \exp(-\gamma\|\mathbf{x} - \mathbf{x}'\|_2^2). \end{aligned}$$

(Be careful that here γ is not the margin, just the notation of a constant which is used in `scikit-learn`).

This is a very popular choice, which is still used a lot in practice. But in my experience it is always behind other kernels in terms of predictive accuracy.

- (e) The **Laplacian** or **Exponential** kernel

$$k(\mathbf{x}, \mathbf{x}') = \exp(-\gamma\|\mathbf{x} - \mathbf{x}'\|_2).$$

Not that it is not available in `scikit-learn` but in `kernlab`.

- (f) The **spline** kernel (we will see in next class why such a name)

$$k(\mathbf{x}, \mathbf{x}') = \prod_{l=1}^d \left[1 + x^{(l)} x'^{(l)} + \frac{1}{2} x^{(l)} x'^{(l)} \min(x^{(l)}, x'^{(l)}) - \frac{1}{6} \min(x^{(l)}, x'^{(l)})^3 \right]$$

if $\mathbf{x}, \mathbf{x}' \in [0, 1]^d$. It is not available in `scikit-learn` but in `kernlab`.

- (g) The **Matérn** or **Sobolev** kernel

$$k(\mathbf{x}, \mathbf{x}') = \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\sqrt{2\nu} \frac{\|\mathbf{x} - \mathbf{x}'\|_2}{\theta} \right)^\nu K_\nu \left(\sqrt{2\nu} \frac{\|\mathbf{x} - \mathbf{x}'\|_2}{\theta} \right)$$

where $\Gamma(\cdot)$ is the Gamma function and K_ν is a modified Bessel function. There are two interesting particular cases:

$$\text{for } \nu = 3/2, k(\mathbf{x}, \mathbf{x}') = \left(1 + \sqrt{3} \frac{\|\mathbf{x} - \mathbf{x}'\|_2}{\theta} \right) \exp\left(-\sqrt{3} \frac{\|\mathbf{x} - \mathbf{x}'\|_2}{\theta}\right),$$

$$\text{for } \nu = 5/2, k(\mathbf{x}, \mathbf{x}') = \left(1 + \sqrt{5} \frac{\|\mathbf{x} - \mathbf{x}'\|_2}{\theta} + \frac{5}{3} \frac{\|\mathbf{x} - \mathbf{x}'\|_2^2}{\theta^2} \right) \exp\left(-\sqrt{5} \frac{\|\mathbf{x} - \mathbf{x}'\|_2}{\theta}\right).$$

Such kernels are very efficient in practice, but they are not available for SVMs in `scikit-learn` nor in `kernlab`.

5.5 Concluding notes

5.5.1 Choice of hyperparameters

For almost all previous kernels, there are hidden tuning parameters: a , p , γ , θ , ν and so on. These are additional hyperparameters (on top of the regularization constant C) which **must be tuned by cross-validation**.

But even in this case, you will sometimes see that it is not sufficient to reach a high prediction accuracy. This is often explained by the intrinsic limitation called *isotropic* kernels: in the list above you can see that there is only **one hyperparameter** in front of $\|\mathbf{x} - \mathbf{x}'\|_2$, which means that there is one hyperparameter common to all features. This implies two important things:

1. Before using a SVM algorithm with a isotropic kernel, you should **always normalize the features**.
2. If you have a problem where some features have predictive power larger than the others (this is often the case in practice), it is wiser to consider **one hyperparameter per feature**. Of course this will increase the computational cost of cross-validation, but this is the price to pay for flexibility.

Unfortunately this is not an option for SVMs in `scikit-learn` or `kernlab`. You can either code it yourself, or use other similar kernel methods for which `scikit-learn` or other R packages offer the possibility to activate anisotropy, namely Gaussian Process Classification or Regression (but these methods are outside the scope of this class).

5.5.2 Extension to multi-class classification

We only tackled binary classification here. How do we generalize SVM to multi-class classification? There are two main approaches:

- (1) The **one-versus-rest (OVR)** or **one-versus-all (OVA)** approach.
If there are M classes, we start by building M separate classifiers, each of them being trained on a binary classification problem (the class of interest versus all the others). For prediction, we finally use

$$h(\mathbf{x}) = \text{sign} \left(\arg \max_{m=1, \dots, M} f_m(\mathbf{x}) \right)$$

where f_m is the decision function trained for the binary classification problem of class m versus all the others. This is the default in `scikit-learn` for Logistic Regression, for example.

- (2) The **one-versus-one (OVO)** approach.
As the name suggests, it splits the training dataset into one dataset for each class versus every other class. This implies that we train $M(M-1)/2$ separate classifiers, but with smaller datasets. For prediction, each model predicts one class label and the class which receives the most votes is selected. This is the default in `scikit-learn` for SVM.

Lecture 4: Regression & Kernel Methods

October 5-19, 2023

- Reminder on regression and focus on local averaging techniques
 - Problem setting and notations
 - Local averaging techniques
 - * Nearest-neighbors
 - * Nadaraya-Watson estimator
 - * Local polynomials
- Introduction to splines
 - Natural cubic splines
 - Towards a representer theorem
- Reproducing Kernel Hilbert Spaces

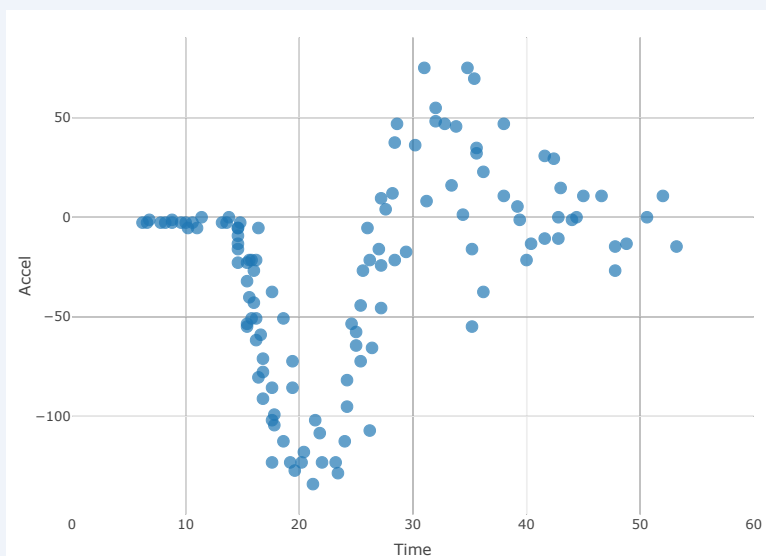
Chapter 6

Reminder on regression

6.1 Problem settings and notations

During this lecture, we will now focus on regression problems, where we are given a training sample $(\mathbf{x}_i, y_i)_{i=1, \dots, n}$ of size n from an unknown joint distribution $\mathbb{P}_{\mathbf{X}Y}$. Here $\mathbf{X} = (X^{(1)}, \dots, X^{(d)}) \in \mathcal{X} \subset \mathbb{R}^d$ is the vector of the d -dimensional features and $Y \in \mathbb{R}$ is the target variable with real continuous values. Our goal is to build an approximation of the *regression function* $f_0(\mathbf{x}) = \mathbb{E}(Y|\mathbf{X} = \mathbf{x})$, see the example below for an illustration.

Example (Illustration of the regression problem in dimension 1.). During this lecture, we will study the `mcycle` dataset from the `MASS` package in R, which consists of a sample of size 133 of observed head accelerations in g (the target y to be predicted) as a function of time in ms (the feature x). Here we only consider samples with times between 5 and 55, resulting in a sample of size $n = 124$. The final dataset looks like below.



Generally, such an approximation will be denoted as $\hat{f}(\mathbf{x})$, and similarly to the classification setting discussed in the previous lecture, we define the **risk** $R(f)$ of any function f through a loss function $l(y, y')$:

$$R(f) = \mathbb{E}_{\mathbb{P}_{\mathbf{X}Y}} [l(Y, f(\mathbf{X}))].$$

Here again $\mathbb{P}_{\mathbf{X}Y}$ is unknown and we will consider the **empirical risk** $\hat{R}(f)$ calculated on the training sample:

$$\hat{R}(f) = \frac{1}{n} \sum_{i=1}^n l(y_i, f(\mathbf{x}_i)).$$

The typical loss function is chosen as $l(y, y') = (y - y')^2$ most of the time, but we will discuss an extension of SVM for regression where another one is used. In practice, regression methods and how they propose to build an approximation $\hat{f}$ of f_0 are classified into two groups:

1. Local averaging methods, which consist in using *local information* for $\hat{f}(\mathbf{x})$ for each $\mathbf{x}$, by using training samples such that $\mathbf{x}_i$ is close to $\mathbf{x}$.
2. Global methods, which start with an initial assumption on the *global* shape of $\hat{f}(\mathbf{x})$ for all $\mathbf{x}$.

We will start with local averaging methods, which you have already studied during your 2nd year.

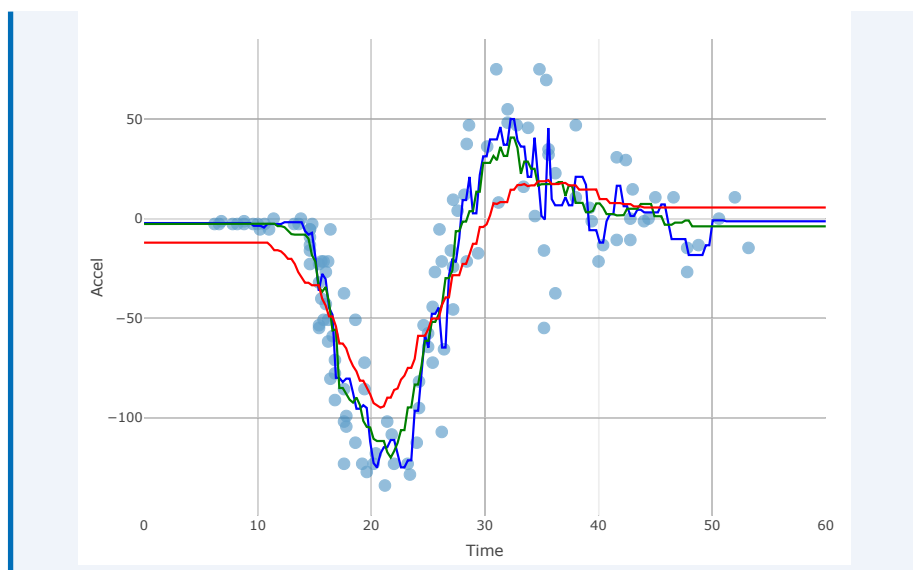
6.2 Local averaging techniques

The first basic local method that you have learned is *k-nearest neighbors*, and in the context of this lecture their version for regression. After having just selected the number k of nearest neighbors, the estimation of the regression function at a given point $\mathbf{x}$ is simply given by

$$\hat{f}(\mathbf{x}) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{x})} y_i$$

where $\mathcal{N}_k(\mathbf{x})$ is the set of all indices of the k closest points to $\mathbf{x}$ among the n training samples $\mathbf{x}_1, \dots, \mathbf{x}_n$. In words, to compute the estimator at a point $\mathbf{x}$, we just average the target values of the training samples which have the k -th closest features values to $\mathbf{x}$. When k is small $\hat{f}(\mathbf{x})$ will be very 'wiggly', while for a large k it will tend to a constant function. We illustrate this behavior on the example introduced before.

Example (Illustration of the impact of the number of nearest neighbors.). On the mcycle dataset, here are the nearest-neighbors estimates for $k = 3$ (blue), $k = 10$ (green) and $k = 30$ (red).



To understand why $\hat{f}$ exhibits such low regularity when k is small and to make a connection later with other approaches, we can rewrite the estimator as

$$\hat{f}(\mathbf{x}) = \frac{1}{k} \sum_{i=1}^n w_i(\mathbf{x}) y_i$$

with weights $w_i(\mathbf{x})$ equal to

$$w_i(\mathbf{x}) = \begin{cases} 1/k & \text{if } \mathbf{x}_i \text{ is one of the } k \text{ nearest neighbors of } \mathbf{x} \\ 0 & \text{otherwise.} \end{cases}$$

Since the weights $w_i(\mathbf{x})$ have jumps when going from nearest neighbors to others, this is also the case for $\hat{f}$. As a side note, the last expression is useful to show that this estimator is a part of the class of the so-called *linear smoothers*. Indeed, if we want to compute the nearest-neighbor predictions for new values $\mathbf{x}_1^*, \dots, \mathbf{x}_m^*$, they are given by the simple formula

$$\hat{\mathbf{f}} = S\mathbf{y}$$

where $\hat{\mathbf{f}} = (\hat{f}(\mathbf{x}_1^*), \dots, \hat{f}(\mathbf{x}_m^*)) \in \mathbb{R}^m$, $\mathbf{y} = (y_1, \dots, y_n) \in \mathbb{R}^n$ and $S \in \mathbb{R}^{m \times n}$ is a matrix with row j consisting of values 0 and $1/k$ depending on the indices of the nearest neighbors of $\mathbf{x}_j^*$. Of course this does not mean that the final function $\hat{f}(\mathbf{x})$ is linear in $\mathbf{x}$ (remember the examples above!), it means that a prediction is obtained by assembling a linear combination of the y_i . Many very famous machine learning models are in fact part of this class, we will see more of them in the next Sections.

The k -nearest-neighbors algorithm is simple and is not a bad estimator. In practice you can always try it to build easily a 'baseline', i.e. a reference algorithm to be compared to more advanced estimators. However, let us mention two important points concerning this algorithm:

- To identify the neighbors, this implies that we use a distance between the features $\mathbf{x}$. This is typically done with the Euclidean distance, meaning that they should be normalized beforehand: if this is not the case and that some features have larger order of magnitude than the others, they will be the main contributors to the distance calculation;
- This estimator suffers from the so-called *curse of dimensionality*, meaning that estimation becomes exponentially harder as the number of features d increases. Indeed, one can show that if the true regression function f_0 is Lipschitz continuous, the risk of $\hat{f}$ is of the order of

$$R(\hat{f}) \lesssim n^{-2/(2+d)}.$$

Starting from the expression as a weighted average above, *kernel smoothing* replaces the discontinuous weight function by a smoother weight function known as a *kernel* function $K : \mathbb{R} \mapsto \mathbb{R}$ satisfying $\int K(t)dt = 1$, $\int tK(t)dt = 0$ and $\int t^2K(t)dt < \infty$. In a nutshell, instead of assigning a weight equal to $1/k$ to the nearest neighbors and 0 to the other training samples, we use the kernel to define the weights as a function of the distance between $\mathbf{x}$ and the training samples $\mathbf{x}_i$, with a weight which decreases with the distance. Examples of common kernels are the Gaussian kernel

$$K(t) = \frac{1}{\sqrt{2\pi}} \exp(-t^2/2)$$

or the Epanechnikov kernel

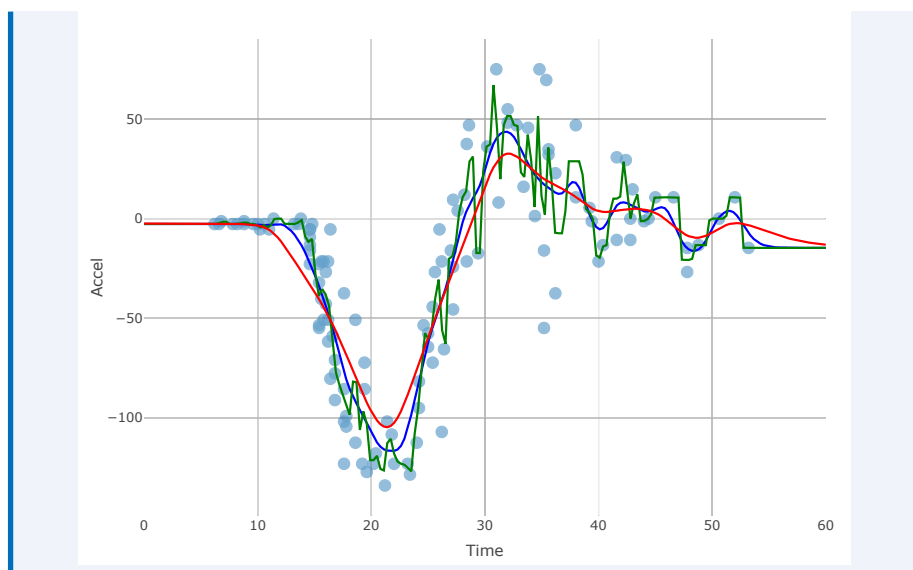
$$K(t) = \frac{3}{4}(1 - t^2)\mathbb{1}(|t| \leq 1).$$

Observe that in these two cases $K(t)$ effectively decreases with t . The weights are thus given by

$$w_i(\mathbf{x}) = \frac{K(\|\mathbf{x} - \mathbf{x}_i\|_2/h)}{\sum_{j=1}^n K(\|\mathbf{x} - \mathbf{x}_j\|_2/h)}$$

where h is an hyperparameter controlling the speed of the weight decay with respect to the distance. This is the **Nadaraya-Watson** estimate. See an example of kernel smoothing on the previous case.

Example (Illustration of kernel smoothing.) On the mcycle dataset, here are the Nadaraya-Watson estimates for h estimated by cross-validation (blue), h too small (green) and h too large (red).



Note that although it produces an estimate smoother than k -nearest-neighbors, it also suffers from the curse of dimensionality with the same order for the risk.

Another layer of generalization can be achieved by writing the previous kernel smoother as the solution of a minimization problem. More precisely, the estimate $\hat{f}(\mathbf{x})$ is equal to $\beta^{\mathbf{x}}$, where $\beta^{\mathbf{x}}$ is the solution of the minimization problem as:

$$\min_{\beta^{\mathbf{x}}} \sum_{i=1}^n K(\|\mathbf{x} - \mathbf{x}_i\|_2/h) (y_i - \beta^{\mathbf{x}})^2.$$

This is simply a least-squares regression constant estimator, but with training samples reweighted thanks to the kernel. It is then natural to consider a linear estimator $\hat{f}(\mathbf{x}) = \beta_0^{\mathbf{x}} + \mathbf{x}^T \beta^{\mathbf{x}}$ obtained by solving

$$\min_{\beta_0^{\mathbf{x}}, \beta^{\mathbf{x}}} \sum_{i=1}^n K(\|\mathbf{x} - \mathbf{x}_i\|_2/h) (y_i - \beta_0^{\mathbf{x}} - \mathbf{x}_i^T \beta^{\mathbf{x}})^2.$$

This is known as *local linear regression*, or more generally *local polynomial regression* (when we use any polynomial inside the weighted least-squares problem). Interestingly, kernel smoothing or general local polynomial estimators are also linear smoothers. Indeed, if $\Omega \in \mathbb{R}^{n \times n}$ is a diagonal matrix composed of the values $K(\|\mathbf{x} - \mathbf{x}_i\|_2/h)$, then we have

$$\hat{\mathbf{f}} = S\mathbf{y}$$

where $S = B(B^T \Omega B)^{-1} B^T \Omega$ and B is the matrix composed of the polynomials used in the model evaluated at $\mathbf{x}_1^*, \dots, \mathbf{x}_m^*$.

6.3 Splines

Contrary to local averaging methods that we recalled before, splines adopt a different perspective: they build the estimator $\hat{f}(\mathbf{x})$ **globally**, from a set of

basis functions. Focusing on the one-dimensional case $d = 1$ here, from a general point of view a k -th order *spline* is simply a piecewise polynomial function of degree k that is continuous and has continuous derivatives of orders $1, \dots, k-1$ at its *knot points* (i.e. the points defining the pieces). The knots are typically denoted $t_1 < \dots < t_p$ and then $\hat{f}$ is a polynomial of degree k on each interval $(-\infty, t_1], [t_1, t_2], \dots, [t_p, \infty)$ with $\hat{f}^{(j)}$ continuous at $t_1, \dots, t_p$ for each derivative order $j = 0, \dots, k-1$. Without resorting to details, such a spline estimator can be written as a linear combination of basis functions

$$\hat{f}(x) = \sum_{j=1}^{p+k+1} \beta_j g_j(x)$$

where the $g_j(x)$ are basis functions leading to piecewise polynomials. More precisely, they can be expressed as the *truncated power basis* - a natural basis -:

$$g_1(x) = 1, g_2(x) = x, \dots, g_{k+1}(x) = x^k$$

$$g_{k+1+j} = (x - t_j)_+^k, j = 1, \dots, p.$$

They can also be written as the *B-spline* basis - the common choice nowadays for computational efficiency, but we do not detail here their expression. The main point is to see that the spline estimator writes as a linear combination. Starting from this point, it is thus straightforward to find the optimal spline by determining the optimal coefficients $\beta_1, \dots, \beta_{p+k+1}$ as solutions of the least-squares problem

$$\min_{\beta \in \mathbb{R}^{p+k+1}} \|y - G\beta\|_2^2$$

where $\beta = (\beta_1, \dots, \beta_{p+k+1})$ and G is the basis matrix composed of the basis functions evaluated at the training samples $\mathbf{x}_i$. This is just standard linear regression with a specific basis, so the optimal coefficients are given by

$$\hat{\beta} = (G^T G)^{-1} G^T y.$$

Such a spline is called *regression spline*, and it belongs to the class of linear smoother since we have

$$\hat{\mathbf{f}} = S\mathbf{y}$$

with $S = G^*(G^T G)^{-1} G^T$ where G^* is the matrix composed of the basis functions evaluated at $\mathbf{x}_1^*, \dots, \mathbf{x}_m^*$. Regression splines are popular, and their efficiency are mainly linked to our ability to choose good knot locations $t_1, \dots, t_p$, which may be quite difficult and expensive in practice.

A popular category of regression splines consists of *natural splines*, which add constraints to the spline definition to enforce a more robust behavior at the boundaries of the input domain. More precisely, natural splines are defined for only odd orders k and impose $\hat{f}$ to be a polynomial of order $(k-1)/2$ on both boundary intervals $(-\infty, t_0]$ and $[t_p, \infty)$. These constraints change the number of degrees of freedom from $p+k+1$ to p , and the natural spline estimate then writes

$$\hat{f}(x) = \sum_{j=1}^p \beta_j \eta_j(x)$$

where the $\eta_j(x)$ are basis functions (truncated basis or B-spline) modified to accommodate these boundary constraints. Most used natural splines are of cubic order with $k = 3$, which leads to linear functions outside the boundaries on the left and on the right.

Finally, the famous *smoothing splines* build upon what we just saw with two main additional ingredients on top of natural splines:

1. The knot locations are simply chosen as the training samples $\mathbf{x}_1, \dots, \mathbf{x}_n$;
2. The potentially induced overfitting is controlled with a penalty on the coefficients $\beta_1, \dots, \beta_p$.

As the name suggests, with smoothing splines, the penalty focuses on controlling the **smoothness** of the estimate, measured here by its second-order derivative. The optimal coefficients $\beta_1, \dots, \beta_p$ in this case are then the solutions of the regularized least-squares problem

$$\min_{\beta \in \mathbb{R}^p} \sum_{i=1}^n \left(y_i - \sum_{j=1}^p \beta_j \eta_j(x_i) \right)^2 + \lambda \int_0^1 \left(\sum_{j=1}^p \beta_j \eta_j^{(2)}(x) \right)^2 dx$$

where we assume without loss of generality that $x \in [0, 1]$ and $\lambda > 0$ is the regularization constant. If N is the matrix of basis functions with knots $\mathbf{x}_1, \dots, \mathbf{x}_n$ and evaluated also at $\mathbf{x}_1, \dots, \mathbf{x}_n$, and $\Omega_{ij} = \int_0^1 \eta_i^{(2)}(x) \eta_j^{(2)}(x) dx$ then the previous optimization problems writes

$$\min_{\beta \in \mathbb{R}^p} \|y - N\beta\|_2^2 + \lambda \beta^T \Omega \beta$$

with solution

$$\hat{\beta} = (N^T N + \lambda \Omega)^{-1} N^T y$$

and we have

$$\hat{f} = S y$$

with $S = N^* (N^T N + \lambda \Omega)^{-1} N^T$ where N^* is the matrix composed of the basis functions evaluated at $\mathbf{x}_1^*, \dots, \mathbf{x}_m^*$, showing once again that smoothing splines are also a linear smoother.

Now the beauty of smoothing splines (or natural splines obtained with $\lambda = 0$ as a special case) is that they can be **directly obtained from a functional minimization perspective**, i.e. without even assuming a specific form of piecewise polynomials like we did at the beginning. Indeed, let us consider the functional optimization problem

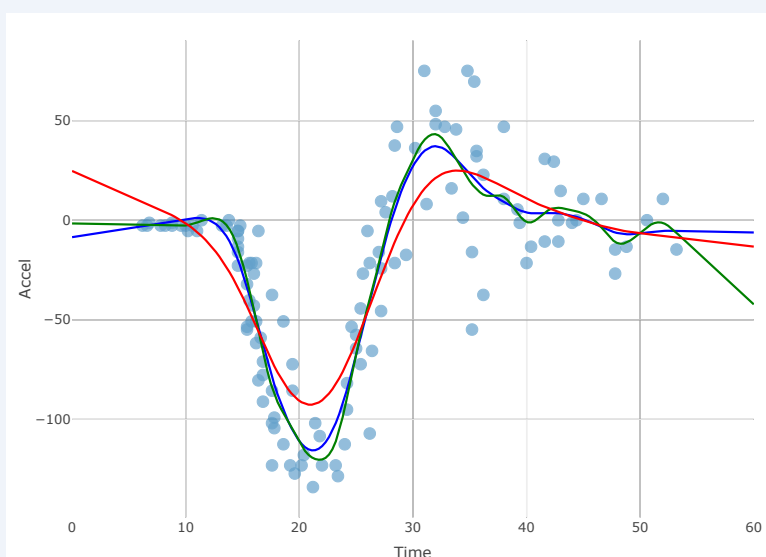
$$\min_{f \in \mathcal{S}^2(0,1)} \sum_{i=1}^n (y_i - f(x_i))^2 + \lambda \int_0^1 (f^{(2)}(x))^2 dx$$

where $\mathcal{S}^2(0,1)$ is the space of functions with continuous derivatives up to order 2 and such that $\int_0^1 (f^{(2)}(x))^2 dx$ is finite. Remark that here we do not assume a specific form for f : this is an infinite-dimensional optimization problem over

all functions $f \in \mathcal{S}^2(0, 1)$, where the criterion is a trade-off between the least-squares errors and a penalty term related to the smoothness of f . Then it can be shown that the solution is given by the cubic smoothing spline! In other words, we are able to transform an infinite-dimensional problem into a finite dimensional one that involves the coefficients of the natural spline basis functions. This kind of result is known as a **representer theorem**, which is central in kernel methods and will be stated in a general setting in the next Chapter. Finally, note that we only discussed a penalty on the second-order derivative, but the same results hold for any m -th order derivative, leading to k -th order smoothing splines with $k = (m + 1)/2$.

We end this Section by giving an example of smoothing splines on the previous example.

Example (Illustration of smoothing splines.). On the mcycle dataset, here are the smoothing spline estimates for λ estimated by cross-validation (blue), λ too small (green) and λ too large (red).



Chapter 7

Reproducing Kernel Hilbert Spaces (RKHS)

In the flow of these lectures, we encountered kernels on two occasions: first when we discussed SVM where we introduced a kernel as an inner product between feature functions in a Hilbert space, and then when we talked about a representer theorem for natural splines. Although these roads towards kernels are different, they lead to the same central construction on which kernel methods rely on: a specific space of functions with very nice, useful and practical properties for machine learning known as **Reproducing Kernel Hilbert Spaces**. Before introducing them, we need to recall some basic ingredients.

7.1 Definitions and notations

First, a Hilbert space $\mathcal{H}$ is a vector space endowed with an inner product, which is complete (i.e. any Cauchy sequence converges) in the metric induced by its norm. If you recall, Hilbert spaces are nice: we have the Hilbert projection theorem, the Pythagorean identity and can define an orthonormal basis.

Example (Hilbert space examples). Among the spaces you already know, here are some Hilbert spaces:

- $\mathbb{R}^d$ with inner product $\langle x, y \rangle = \sum_{i=1}^d x^{(i)} y^{(i)}$
- The sequence space ℓ^2 whose elements are infinite sequences of real numbers such that $\sum_n x_n^2$ converges, with inner product $\langle x, y \rangle = \sum_n x_n y_n$
- The Lebesgue space $L^2(\mu)$ of real measurable functions such that $\int f^2 \mu$ is finite, with inner product $\langle f, g \rangle = \int f g \mu$

We also need to consider continuous linear operators in a Hilbert space.

Definition 24 (Continuous linear operator). An operator $L : \mathcal{H} \mapsto \mathbb{R}$ is a function that maps elements of the Hilbert space $\mathcal{H}$ to the real numbers, which is **linear** if and only if $L(\alpha h + \beta g) = \alpha L(h) + \beta L(g)$ for all $h, g \in \mathcal{H}$ and $\alpha, \beta \in \mathbb{R}$.

We say that L is **continuous** if it is continuous at any point h , i.e. $\lim_{n \rightarrow \infty} L(h_n) = L(h)$ for all sequence $h_n \rightarrow h$. Equivalently, L is continuous if it is bounded for every h , i.e. $|L(h)| \leq c \|h\|_{\mathcal{H}} \forall h \in \mathcal{H}$.

Finally, the **Riesz representation theorem** states that if L is a continuous linear operator, there exists a unique function $g_h \in \mathcal{H}$ such that the action of L on any function $h \in \mathcal{H}$ can be written as an inner product:

$$L(h) = \langle h, g_L \rangle_{\mathcal{H}}$$

where we use the subscript g_h to emphasize the fact that it depends on L .

We will now restrict ourselves to an Hilbert space $\mathcal{H}$ of functions defined on $\mathcal{X}$ with values in $\mathbb{R}$. Among all linear operators from $\mathcal{H}$ to $\mathbb{R}$, we will focus on the *evaluation functional* denoted by $L_{\mathbf{x}}$, which maps any function $h \in \mathcal{H}$ to its value at $\mathbf{x}$, i.e. $L_{\mathbf{x}}(h) = h(\mathbf{x})$ defined for any $\mathbf{x} \in \mathcal{X}$. A *Reproducing Kernel Hilbert Space* (RKHS) is simply such an Hilbert space such that the evaluation functional $L_{\mathbf{x}}$ is a **continuous** linear operator for every $\mathbf{x} \in \mathcal{X}$ (it is obviously linear by definition), or equivalently $L_{\mathbf{x}}$ is bounded.

Definition 25 (RKHS). A RKHS is an Hilbert space of functions $\mathcal{X} \mapsto \mathbb{R}$ such that the evaluation functional $L_{\mathbf{x}}$ is bounded for every $\mathbf{x} \in \mathcal{X}$. For a RKHS, we can then use the Riesz representation theorem for $L_{\mathbf{x}}$, which implies that for every $\mathbf{x}$ we can find a unique function $k_{\mathbf{x}} \in \mathcal{H}$ such that

$$L_{\mathbf{x}}(h) = \langle h, k_{\mathbf{x}} \rangle_{\mathcal{H}}.$$

This allows us to define the **reproducing kernel** $k(\mathbf{x}, \mathbf{x}') : \mathcal{X} \times \mathcal{X} \mapsto \mathbb{R}$ simply as $k(\mathbf{x}, \mathbf{x}') = k_{\mathbf{x}}(\mathbf{x}')$, or equivalently $k(\mathbf{x}, \mathbf{x}') = \langle k_{\mathbf{x}}, k_{\mathbf{x}'} \rangle_{\mathcal{H}}$ (through Riesz representation theorem). It can be shown that (proof left as an exercise):

- k has the **reproducing property** $\langle k(\cdot, \mathbf{x}), k(\cdot, \mathbf{x}') \rangle_{\mathcal{H}} = k(\mathbf{x}, \mathbf{x}')$ (hence the name)
- k is symmetric
- k is positive definite, i.e. $\sum_{i,j=1}^n a_i a_j k(\mathbf{x}_i, \mathbf{x}_j) \geq 0, \forall a_i, a_j \in \mathbb{R}, \forall \mathbf{x}_i, \mathbf{x}_j \in \mathcal{X}$ (think of it as a generalization of positive definite matrices which would be infinite dimensional)
- $h(\mathbf{x}) = \langle h, k_{\mathbf{x}} \rangle_{\mathcal{H}}, \forall h \in \mathcal{H}, \forall \mathbf{x} \in \mathcal{X}$
- k is unique

We stated above that each RKHS has a unique kernel, but actually an im-

portant theorem strengthens this property:

Theorem 4 (Correspondence between RKHS and kernel). To every RKHS there is a unique nonnegative definite kernel with the reproducing property, and conversely for any symmetric, nonnegative definite $k(\mathbf{x}, \mathbf{x}') : \mathcal{X} \times \mathcal{X} \mapsto \mathbb{R}$, there is a unique RKHS $\mathcal{H}_k$ of functions on $\mathcal{X}$ whose kernel is k .

Note that the converse is the Moore-Aronszajn theorem that we already mentioned in the SVM class, but now we know that there is in fact a complete equivalence: this means that a RKHS can be defined either with the continuous property of the evaluation functional or directly with the kernel. Interestingly, this converse can be proven with an explicit construction of the RKHS associated to a given kernel. In this setting, we start with just a symmetric positive definite k and define the space $\mathcal{H}_0$ consisting of all linear combinations of the functions $\{k(\cdot, \mathbf{x}'), \mathbf{x}' \in \mathcal{X}\}$:

$$\mathcal{H}_0 = \left\{ \mathbf{x} \mapsto \sum_{i=1}^n a_i k(\mathbf{x}, \mathbf{x}_i) \right\}$$

for any number n of functions $k(\cdot, \mathbf{x}_i)$ taken from $\{k(\cdot, \mathbf{x}'), \mathbf{x}' \in \mathcal{X}\}$ and any real numbers $a_i, i = 1 \dots, n$, and define an inner product for $\mathcal{H}_0$ as

$$\left\langle \sum_i a_i k(\cdot, \mathbf{x}_i), \sum_j a'_j k(\cdot, \mathbf{x}'_j) \right\rangle = \sum_{i,j} a_i a'_j k(\mathbf{x}_i, \mathbf{x}'_j).$$

Then we can prove the following (left as an exercise):

1. k has the reproducing property in $\mathcal{H}_0$
2. A Cauchy sequence of functions in $\mathcal{H}_0$ is guaranteed to converge to a limiting function pointwise (hint: by Cauchy-Schwartz)

Therefore, if we consider the space $\mathcal{H}_0$ of all linear combinations extended with the limits of Cauchy sequences of these functions, we obtain a complete inner product space denoted by $\mathcal{H}_k$ which is a RKHS.

Now and most importantly, we will discuss why RKHS are so useful spaces to work with in machine learning.

7.2 RKHS properties for machine learning

We will focus here on three main RKHS properties.

7.2.1 Representer theorem

First and foremost, the celebrated **representer theorem** that we already saw but in a specific situation.

Theorem 5 (Representer theorem in a RKHS). Assume that:

- $k : \mathcal{X} \times \mathcal{X} \mapsto \mathbb{R}$ is a symmetric positive definite kernel with associated RKHS $\mathcal{H}$ of functions $\mathcal{X} \mapsto \mathbb{R}$,
- $(\mathbf{x}_i, y_i) \in \mathcal{X} \times \mathbb{R}$, $i = 1, \dots, n$ is a sample of data,
- l is any loss function and $\lambda > 0$ is a real positive constant,
- J is a strictly increasing real-valued function.

Then any solution f^* of the minimization problem

$$\min_{f \in \mathcal{H}} \sum_{i=1}^n l(y_i, f(\mathbf{x}_i)) + \lambda J(\|f\|_{\mathcal{H}})$$

admits a representation of the form

$$f^*(\mathbf{x}) = \sum_{i=1}^n \alpha_i k(\mathbf{x}, \mathbf{x}_i)$$

for some coefficients $\alpha_i \in \mathbb{R}$ for all $i = 1, \dots, n$.

This theorem is particularly powerful, because it states that if we write a regularized empirical risk minimization problem in an infinite-dimensional space which is a RKHS, then it can **always be solved via an equivalent finite-dimensional problem**. Let us see below an illustration for regression, with a penalization $J(\|f\|_{\mathcal{H}}) = \|f\|_{\mathcal{H}}^2$.

Example (Representer theorem for regression with squared-norm penalty – **Kernel Ridge Regression**). When we consider the least-squares loss and the penalty $J(\|f\|_{\mathcal{H}}) = \|f\|_{\mathcal{H}}^2$, by defining K the **Gram matrix** such that $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$ the minimization problem writes

$$\min_{\alpha} \|y - K\alpha\|_2^2 + \lambda \alpha^T K \alpha$$

since $(f(\mathbf{x}_1), \dots, f(\mathbf{x}_n)) = K\alpha$ and $\|f\|_{\mathcal{H}}^2 = \alpha^T K \alpha$ by the representer theorem. The minimizer over α is

$$\hat{\alpha} = (K + \lambda I)^{-1} y$$

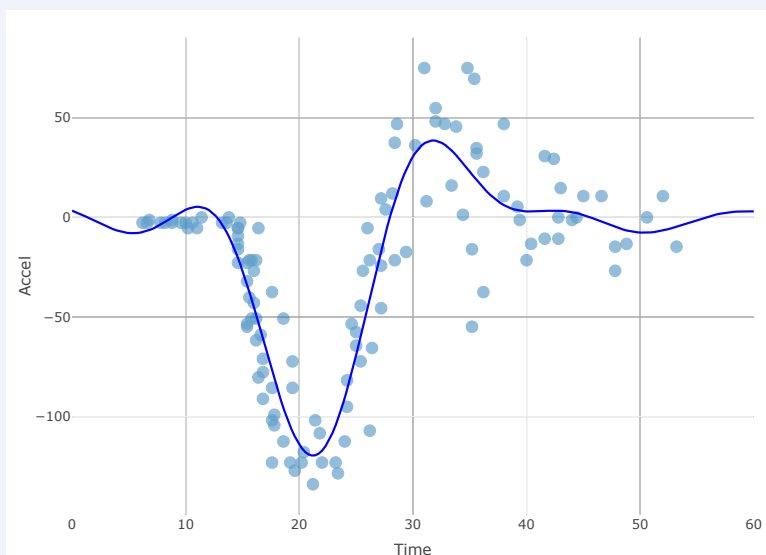
and the estimate is finally

$$\hat{f} = Sy$$

with $S = K^*(K + \lambda I)^{-1}$ where $K_{ij}^* = k(\mathbf{x}_i^*, \mathbf{x}_j)$, showing that this is also a linear smoother. This approximating model is known as **Kernel Ridge Regression**. When using the linear kernel $k(\mathbf{x}, \mathbf{x}') = \langle \mathbf{x}, \mathbf{x}' \rangle$ (that we saw in the kernel jungle previously), it is easy to show that $K = XX^T$ and thus kernel ridge regression boils down to simple linear regression with the ridge penalty.

We give below an illustration of Kernel ridge regression with a Gaussian kernel on our previous example.

Example (Illustration of Kernel ridge regression.). On the mcycle dataset, here is the kernel ridge regression estimate for λ and γ (parameter of Gaussian kernel) both estimated by cross-validation (blue).



Finally, we can also make a connection with the smoothing splines discussed above, for which we stated a representer theorem for the first time. It is actually a special case of the representer theorem in a specific RKHS.

Example (Representer theorem and link with smoothing splines). We refer to <https://math.mit.edu/~sswatson/pdfs/rkhs.pdf> for the derivation, which involves the *spline kernel* mentioned in the kernel jungle, hence the name.

Another prominent example is classification with SVM. In the previous lecture we saw how to introduce kernels inside their formulation to make them nonlinear (we *kernelized* them). The other road towards kernelization is simply to write the empirical risk minimization problem and make use of the representer theorem.

Example (Representer theorem for classification – SVM). Recall the soft-margin primal problem from SVM

$$\min_{\mathbf{w}, \beta} \frac{1}{n} \sum_{i=1}^n [1 - y_i f(\mathbf{x}_i)]_+ + \lambda \|\mathbf{w}\|_2^2$$

where we restricted to linear decision function $f(\mathbf{x}) = \beta_0 + \mathbf{x}^T \beta$. Now we place ourselves in a RKHS of nonlinear functions with associated reproducing kernel k :

$$\min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n [1 - y_i f(\mathbf{x}_i)]_+ + \lambda \|f\|_{\mathcal{H}}^2.$$

By use of the representer theorem, we have that at the optimum $\|f\|_{\mathcal{H}}^2 = \alpha^T K \alpha$ such that we have to solve

$$\min_{\alpha} \frac{1}{n} \sum_{i=1}^n [1 - y_i (K\alpha)_i]_+ + \lambda \alpha^T K \alpha,$$

and we end up with the same formulation as before (proof left as an exercise).

As a sidenote, we can also kernelize logistic regression: the resulting optimization problem however cannot be solved analytically, but can be handled with convex optimization algorithms.

7.2.2 Feature maps

A second property of a RKHS worth mentioning is related to the so-called **feature maps**. During the previous lecture, we kernelized SVM by considering a nonlinear transformation of the features and introduced the kernel as the inner product between these transformations, known as *feature maps*. The other way around, to have a better understanding of the kernel we choose, we can try to find what kind of feature maps it is related to.

Definition 26 (Feature map). A function ϕ is a **feature map** of a kernel k as soon as $\langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle_{\mathcal{F}} = k(\mathbf{x}, \mathbf{x}')$ where $\mathcal{F}$ is some Hilbert space (which may be different from $\mathcal{H}$, the RKHS associated to k). Note that in particular, a feature map always exists – the *canonical feature map* $\phi(\mathbf{x}) = k(\cdot, \mathbf{x})$ for example is always defined – but needs not be unique.

When $\mathcal{X}$ is compact (most case studies you will have to handle will verify this assumption), the following theorem readily gives features maps for a kernel.

Theorem 6 (Mercer's theorem). Suppose k is a positive definite kernel such that $\int k(\mathbf{x}, \mathbf{x}) d\mu(\mathbf{x}) < \infty$, $\forall \mathbf{x} \in \mathcal{X}$ and a given finite countably additive measure μ . Then there is an orthonormal basis $\{e_j\}_{j \geq 1}$ of $L^2(\mu)$ solution of the eigenvalue problem

$$\int k(\mathbf{x}, \mathbf{x}') e_j(\mathbf{x}') d\mu(\mathbf{x}') = \lambda_j e_j(\mathbf{x})$$

with eigenvalues $\lambda_j \geq 0$. The eigenfunctions corresponding to non-zero

eigenvalues are continuous on $\mathcal{X}$ and k has the representation

$$k(\mathbf{x}, \mathbf{x}') = \sum_{j=1}^{\infty} \lambda_j e_j(\mathbf{x}) e_j(\mathbf{x}')$$

where the convergence is absolute and uniform.

Mercer's theorem then implies that if we take $\mathcal{F} = \ell^2$, $\phi(\mathbf{x}) = (\sqrt{\lambda_j} e_j(\mathbf{x}))_{j \geq 1}$ is a feature map of k . In other words, functions of the RKHS, in addition of having an expansion in terms of k as we saw before, can also be expanded in terms of the basis functions e_j :

$$f(\mathbf{x}) = \sum_{j=1}^{\infty} a_j e_j(\mathbf{x})$$

and the inner product between two functions of the RKHS $f(\mathbf{x}) = \sum_{j=1}^{\infty} a_j e_j(\mathbf{x})$

and $g(\mathbf{x}) = \sum_{j=1}^{\infty} b_j e_j(\mathbf{x})$ is thus given by

$$\langle f, g \rangle_{\mathcal{H}} = \sum_{j=1}^{\infty} \frac{a_j b_j}{\lambda_j}.$$

This shows that penalizing with the norm of a function in the RKHS is equivalent to penalizing the squared norm of the coefficients of the function on these basis functions, normalized by the eigenvalues.

7.2.3 Smoothing

Via Mercer's theorem above, we have just seen that roughly speaking, when $\|f\|_{\mathcal{H}}$ is small, then f tends to be smooth since its coefficients in the orthonormal basis of eigenfunctions will be small. This argument can be made more general thanks to Cauchy-Schwartz inequality:

$$\begin{aligned} |f(\mathbf{x}) - f(\mathbf{x}')| &= |\langle f, k_{\mathbf{x}} \rangle - \langle f, k_{\mathbf{x}'} \rangle| \text{ (Reproducing property)} \\ &= |\langle f, k_{\mathbf{x}} - k_{\mathbf{x}'} \rangle| \text{ (Linearity of inner product)} \\ &\leq \|f\|_{\mathcal{H}} \|k_{\mathbf{x}} - k_{\mathbf{x}'}\|_{\mathcal{H}} \text{ (Cauchy-Schwartz)} \end{aligned}$$

This means that the variations of f on the left-hand side are bounded by the norm of f in the RKHS (and a constant independent of f but related to the kernel). A small norm thus implies small variations.

In even more specific cases such as splines discussed before, the norm is directly related to integrals of derivatives of f .

7.3 Building kernels

Now that we have seen a part of RKHS theory and how it helps solving machine learning problems written as an regularized empirical risk minimization, the last

and central question is how to design a kernel suited for this task. There are (at least) three main roads:

1. Pick a kernel from the kernel zoo detailed in the first lecture
2. Manually design a feature map like we did for SVM
3. Combine existing kernels

Points 1. and 2. have already been discussed, so we will only give recipes to design new kernels obtained as a combination of others:

Sum. If k_1 and k_2 are kernels, then $k = ak_1 + bk_2$ is a kernel for $a, b \geq 0$

Product. If k_1 and k_2 are kernels, then $k = k_1k_2$ is a kernel

Limit. If $(k_j)_{j \geq 1}$ is a sequence of kernels that converges pointwise to a function k , then k is a kernel

Exponential. If k is a kernel, then $\exp(k)$ is a kernel

Tensorization Let k_j , $j = 1, \dots, d$ be kernels on $\mathbb{R}$, then $k(\mathbf{x}, \mathbf{x}') = \prod_{j=1}^d k_j(x_j, x'_j)$ is a kernel on $\mathbb{R}^d$

Additive kernel Let k_j , $j = 1, \dots, d$ be kernels on $\mathbb{R}$, then $k(\mathbf{x}, \mathbf{x}') = \sum_{j=1}^d a_j k_j(x_j, x'_j)$ is a kernel on $\mathbb{R}^d$ for $a_j \geq 0$

7.4 Final remarks

In this lecture we focused on kernel methods for supervised learning, but it is worth mentioning that they can also be used for unsupervised learning, most notably for kernel PCA and kernel k-means.

From a computational perspective, most algorithms scale poorly with the number of training samples n , having a complexity of $\mathcal{O}(n^3)$. This is the reason why very often kernel methods are said to be unusable in big data settings. However many recent research works have proposed alternatives and tricks to heavily reduce this complexity, even achieving $\mathcal{O}(n)$ in some cases.

Finally, as already discussed for SVM, do not forget that in general you will have hyperparameters to tune with cross-validation, from the regularizing constant λ to the kernel parameters!

Lecture 5: Ensemble learning

October 26, 2023

- Bagging & Random Forests
- Boosting
- Stacking

Chapter 8

Ensemble learning

In the broad sense, *ensemble learning* refers to all machine learning methods which make use of several estimates to build a new model that aggregates all the predictions. The hope is that by combining many models, the final one will be able to achieve better accuracy.

In these notes we propose to present the most prominent methods from ensemble learning, by dividing them in two categories:

1. Methods that combine many *weak learners*, i.e. estimates that are not highly predictive alone (just a little better than random guessing). In this category you will find **Bagging** and **Boosting**,
2. Methods that aggregate a small number of *strong learners*, i.e. estimates that already achieve a pretty good accuracy. **Stacking** is part of this category.

In any case, ensemble learning builds upon the general idea of "Wisdoms of Crowds": the collective knowledge of a diverse and independent body of people typically exceeds the knowledge of any single individual. In our context, an individual is a machine learning model, the crowd is an ensemble of machine learning models, that we want diverse and independent. We will see below how we can achieve diversity and independence, or at least aim for it.

8.1 Bagging

8.1.1 General principle

Bagging follows ideas from Leo Breiman (inventor of CART and random forests) by using three main principles:

1. It uses several **basic models**, or weak learners,
2. These basic models are trained on **different samples** taken from the training data (to aim for independence),
3. The final estimate is obtained by a **vote** (classification) or the **average** (regression) among all the basic models.

In Bagging the different samples seen by each weak learner are obtained by **bootstrap**, hence the name (**B**ootstrap **A**ggregating).

From a mathematical perspective, such an approach can be motivated in both a regression and a classification setting. Denote $g_1(\mathbf{x}), \dots, g_B(\mathbf{x})$ an ensemble of B base estimators. For regression, the bagged estimator is simply

$$\hat{g}_B(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B g_b(\mathbf{x}).$$

We can investigate the performance of $\hat{g}_B$ by looking at its bias and its variance, by making an **unfeasible and unrealistic assumption** but which serves as an illustration: each g_b is a random variable (since it is trained on a random sample = the training set) and we assume that all $g_1, \dots, g_B$ are independent and identically distributed. In this setting, we have that

Bias. $\mathbb{E}(\hat{g}_B(\mathbf{x})) = \mathbb{E}(g_b(\mathbf{x}))$

Variance $\text{Var}(\hat{g}_B(\mathbf{x})) = \frac{1}{B} \text{Var}(g_b(\mathbf{x}))$

In other words, the bias of the bagged estimate is unchanged, but it has a much smaller variance when compared to the weak learners. Similarly for 0/1 classification, assume further that each weak learner has an independent error $\varepsilon < 0.5$ to predict the correct decision $Y = 1$, i.e. $\mathbb{P}(g_b(\mathbf{x}) \neq 1) = \varepsilon < 0.5$. The bagged classifier writes $\hat{g}_B(\mathbf{x}) = \mathbb{1} \left(\frac{1}{B} \sum_{b=1}^B g_b(\mathbf{x}) > 0.5 \right)$. Then, by making use of the Hoeffding inequality applied to Bernoulli variables $X_i, i = 1, \dots, n$

$$\mathbb{P} \left(\mathbb{E}(X_i) - \frac{1}{n} \sum_{i=1}^n X_i > t \right) < \exp(-2nt^2),$$

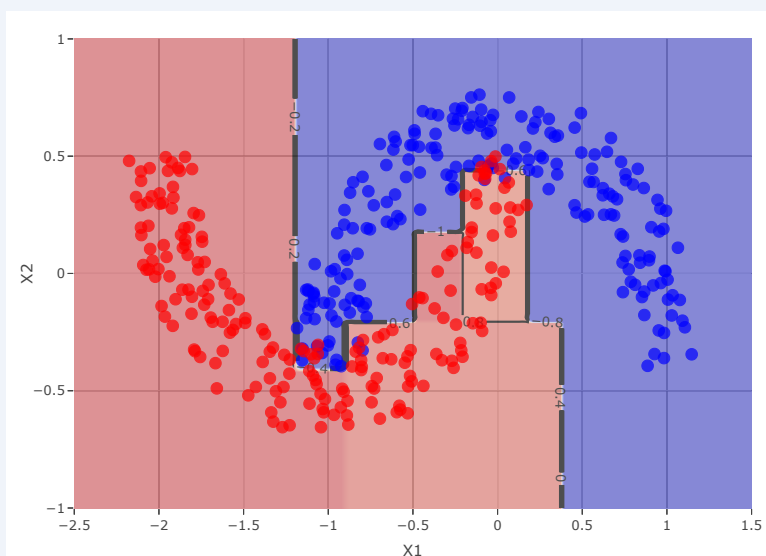
we can show that

$$\begin{aligned} \mathbb{P}(\hat{g}_B(\mathbf{x}) \neq 1) &= \mathbb{P} \left(\frac{1}{B} \sum_{b=1}^B g_b(\mathbf{x}) < 0.5 \right) \\ &= \mathbb{P} \left(1 - \varepsilon - \frac{1}{B} \sum_{b=1}^B g_b(\mathbf{x}) > 0.5 - \varepsilon \right) \\ &= \mathbb{P} \left(\mathbb{E}(g_b(\mathbf{x})) - \frac{1}{B} \sum_{b=1}^B g_b(\mathbf{x}) > 0.5 - \varepsilon \right) \\ &< \exp \left(-\frac{1}{2} B (1 - 2\varepsilon)^2 \right), \end{aligned}$$

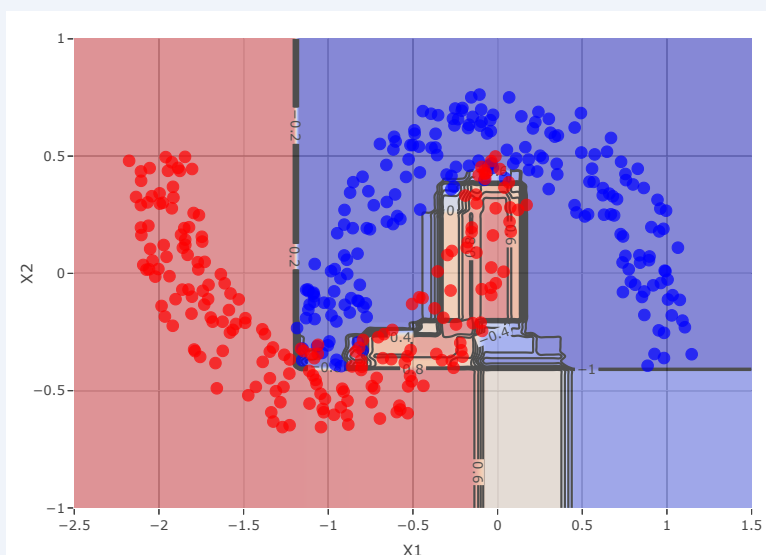
which is less than ε for B large enough, meaning that the bagged estimator has once again smaller error than the weak learners. Of course we had to use an unrealistic assumption to derive these results, since all the weak classifiers are built from the same training sample. However they illustrate the kind of benefits we hope for by using Bagging. Now in practice, to build weak learners as diverse and independent as possible, Bagging introduces a source of randomness in the sample used to build each weak learner with the bootstrap, meaning that we draw with replacement among the initial training sample to generate several new

different samples for each learner. In addition, it also uses unstable learners (i.e. estimates that are not robust when the training sample changes), such as trees which are typically not pruned. You should have seen last year how unstable they are. We illustrate below Bagging of trees on the same classification example we used for SVM.

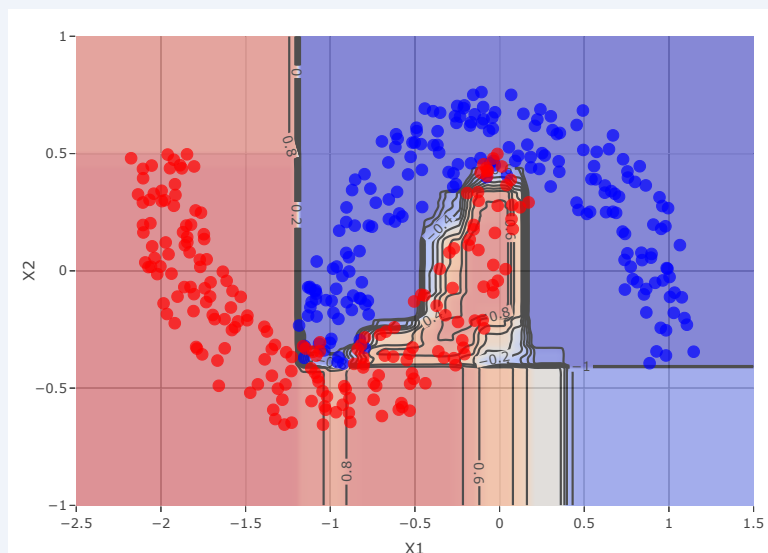
Example (Two Moons continued). We first start by showing the results obtained by a single tree.



By adding more trees to the bagging estimator, we get decision boundaries more and more complex. Here is what we obtain with 10 trees:



and finally 100 trees:



From a practical point of view, let us finish with some notes and recommendations:

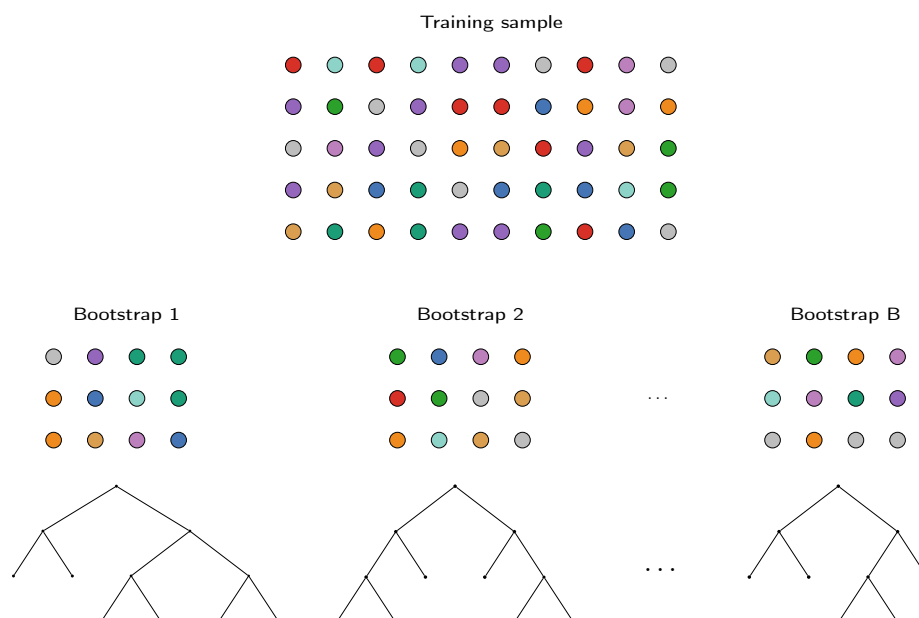
- By the law of large numbers, B should be chosen as large as possible,
- Bagging a bad classifier can make it worse!,
- On very large datasets, do not expect Bagging to yield significant improvement. This is because bootstrapped samples will be very similar in this case. However subsampling may improve things,
- Sometimes Bagging can be improved by using a more robust aggregation of the weak learners, by using for example the median or the trimmed mean instead of the mean for regression.

8.1.2 Random Forests

Random forests have been introduced by Breiman as a more efficient version of Bagging. As the name suggests forests are a collection of trees, so the weak learners which are used here are trees. But they also introduce a substantial modification of Bagging which aims at increasing the diversity and independence of the trees.

More precisely, if you recall how trees are grown, for each node the best split is searched for among all features. If you have a situation where one of the features has a much larger impact for prediction than the others, trees will tend to propose the same split with this important feature even if they are built with different samples from bootstrap. This will severely limit their diversity and independence. To mitigate this behavior, random forests propose to search for the best split not among all d features, but on a **random subset of the features** of size $m_{\text{try}} < d$, which is randomly generated independently at each

node and for each tree. This trick insures that all the trees will be very diverse and will not look alike. You will find below a graphical illustration.

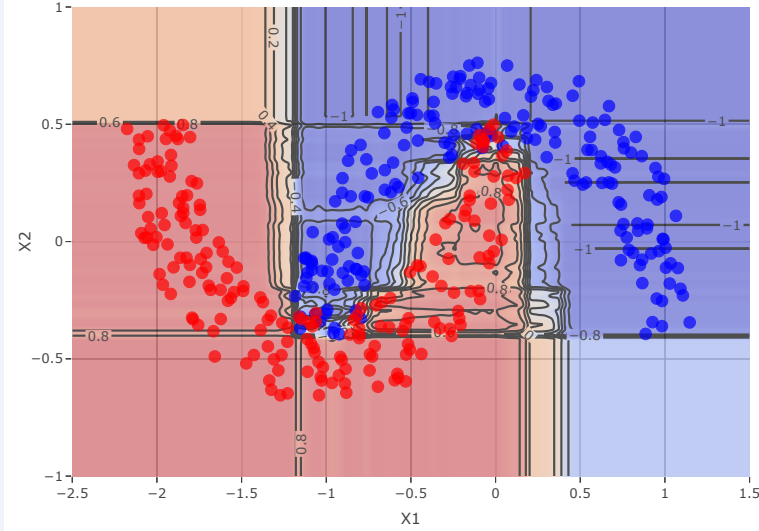


From a practical point of view:

- So in addition to the number of bootstrapped samples B , the number of features m_{try} also needs to be specified (but let us also mention n_{min} the minimum number of samples in the node to allow a split). The final estimate is known to be quite robust to the choice of these hyperparameters, but in practice it is recommended to at least tune m_{try} , which corresponds to a balance between having more predictive but less diverse trees (large m_{try}) and less predictive but more diverse trees (small m_{try}). Default recommended values are $m_{try} = \sqrt{d}$ for classification and $m_{try} = d/3$ for regression,
- Random forests are extremely popular and very efficient implementations are available in `scikit-learn` for Python and `ranger` in R. They achieve quite state-of-the-art accuracy on complex tabular data with many features, missing data, categorical features, ...

We give below an example of random forests on the same classification dataset examined previously.

Example (Two Moons continued). We use a random forest with 500 trees and the default values for m_{try} and n_{min} .



Note that the predictions are similar to what we obtained with Bagging and 100 trees: since we are only in dimension $d = 2$, the additional randomness on the splits from random forests does not activate here.

Finally, random forests (but more generally all bagging techniques) can benefit from the so-called **out-of-bag** samples to compute several indicator of performance and variable importance without having the need of an independent test sample.

Error Estimation - Out-Of-Bag Samples (OOB)

An interesting feature of random forests is that the sample selection procedure using bootstrap implies that the individuals in the training set are not used in the construction of all the trees but only in some of them. These are referred to as *Out-Of-Bag* (OOB) samples. This means that it is possible to build an estimation of the empirical risk calculated on a test sample without having a separate test sample! As a reminder, using a test sample $\mathcal{D}_n^{\text{test}} = \{\mathbf{x}_i^*, y_i^*\}_{i=1, \dots, n^{\text{test}}}$, the generalization error of the predictor $\bar{g}_B$ of a random forest is given by

$$\hat{R}_{\mathcal{D}_n^{\text{test}}(\bar{g}_B)} = \frac{1}{n^{\text{test}}} \sum_{i=1}^{n^{\text{test}}} l \left(y_i^*, \frac{1}{B} \sum_{b=1}^B g_b(\mathbf{x}_i^*) \right).$$

Using the OOB samples, we can construct a similar error using only the training sample $\mathcal{D}_n^{\text{train}} = \{\mathbf{x}_i, y_i\}_{i=1, \dots, n^{\text{train}}}$:

$$\hat{R}_{\text{OOB}}(\bar{g}_B) = \frac{1}{n^{\text{train}}} \sum_{i=1}^{n^{\text{train}}} l \left(y_i, \frac{1}{|B_{-i}|} \sum_{b \in B_{-i}} g_b(\mathbf{x}_i) \right)$$

where $B_{-i} \subset \{1, \dots, B\}$ denotes the subset of trees that were not trained with the i -th individual in the training set. You can find several studies where OOB

error is used to validate the final quality of a random forest. However, it is important to keep in mind that it often underestimates the generalization error. Therefore, it is advisable to always keep a separate test sample for calculating it, as you usually do. On the other hand, an interesting alternative use (in terms of computation time) is to use this OOB error to estimate the hyperparameters of the random forest, instead of using cross-validation. In this case, the underestimation of the error is less detrimental since tuning hyperparameters only requires a metric for comparing their values in a relative rather than absolute sense.

Variable Importance

One final aspect of random forests that you will most certainly encounter when using them in internships or projects is related to what is called "variable importance." Behind this generic term lies an extremely vast and diverse research theme in statistics, which focuses on measuring whether an explanatory variable is important in predicting the target variable. There is a whole zoology of measures of this type, each with a myriad of advantages, disadvantages, and application frameworks.

When we talk about variable importance for random forests, we refer to two quantitative measures of the effect of an explanatory variable on the output:

1. **Mean Decrease in Impurity (MDI):** This is the most common measure and is often the default choice in random forest libraries (in Python and R). It is based on a fairly simple intuition: if an explanatory variable is important for explaining/predicting the output, then when it is selected for a split in one of the trees, it should significantly decrease impurity. The MDI calculates, for each explanatory variable, the reduction in impurity it has caused (*Decrease in Impurity*), averaged over all nodes and all trees in which it was involved (*Mean*). More formally, the unnormalized version of MDI for the explanatory variable $X^{(j)}$ is written as:

$$\text{MDI}(X^{(j)}) = \frac{1}{B} \sum_{b=1}^B \sum_{\text{node } k \in T_b | \text{split}(k)=X^{(j)}} p_{T_b}(k) \Delta i_{T_b}(k)$$

where T_b denotes the b -th tree, $p_{T_b}(k)$ is the proportion of the sample in node k of tree T_b , and $\Delta i_{T_b}(k)$ is the impurity reduction observed when splitting node k of tree T_b . In practice, after calculating MDI for all explanatory variables, they are normalized to have a sum equal to 1.

2. **Mean Decrease Accuracy (MDA):** The main idea behind this measure is that if an explanatory variable is not important for predicting the target, whether or not it is considered in the construction of the forest should not change the generalization error. So, we could calculate the decrease in generalization error due to the removal of one of the explanatory variables (*Decrease Accuracy*), averaged over a test sample (*Mean*):

$$\text{MDA}(X^{(j)}) = \frac{1}{n^{\text{test}}} \sum_{i=1}^{n^{\text{test}}} l \left(y_i^*, \frac{1}{B} \sum_{b=1}^B g_b^{(-j)}(\mathbf{x}_i^*) \right) - l \left(y_i^*, \frac{1}{B} \sum_{b=1}^B g_b(\mathbf{x}_i^*) \right)$$

where $g_b^{(-j)}$ denotes the random forest constructed without the j -th explanatory variable. In practice, calculating the MDA in this way involves constructing the random forest multiple times, each time removing one of the explanatory variables. For computational reasons, a common alternative is often used: random permutation. A single forest is trained, and during the prediction phase on the test sample, we calculate (1) their usual prediction, (2) their prediction when we perform a random permutation of the values of an explanatory variable, and (3) the decrease in generalization error between these two predictions. Such a random permutation of observations "breaks" the statistical dependency that may have existed between the explanatory variable and the output, mimicking what would have been achieved by removing the explanatory variable from the start. Therefore, you will also see the term **Permutation Importance** for this method. Finally, in the vast majority of libraries, the calculation is not performed on a separate test sample but rather using the OOB samples described earlier.

It is important to note that both of these variable importance methods **suffer from strong biases** and should be analyzed with caution. For example, they are both highly sensitive to categorical explanatory variables with many categories (i.e., they often appear to be much more influential than they actually are). They are also highly biased when explanatory variables are dependent on each other, especially when the relationship with the target variable involves interactions.

8.2 Boosting

In Bagging, we just saw that the weak learners are learned independently and in parallel with several sub-training samples. In *Boosting*, the principle is still to aggregate weak learners, but this time they are built **sequentially**, the underlying idea being that each additional weak learner will be trained to correct and improve the previous ones.

Among all Boosting algorithms, the most two famous are based on the two different roads we can take to build a new weak learner to improve upon the previous ones. They both rely on the fact that at a given iteration of the algorithm we can evaluate the errors of the current aggregation of weak learners. But we can use these errors in two different ways:

- **Grade & Re-weight.** First, the errors can be summarized to give a grade to the last incorporated weak learner, which will be used to assign it a weight in the aggregation. Second, training samples which are not predicted accurately receive more weights than the others for training the next weak learner.
- **Train on the residuals.** Here the weak learners and the training samples are left unchanged, but at each iteration, instead of predicting the target, the new weak learner is trained to predict the residual between the target and the previous predictions.

The terminology *Boosting* comes from the fact that at each iteration, adding a new weak learner makes it possible to make a little step or 'boost' towards a

more predictive aggregated model.

In the first category, the most famous algorithm is without a doubt **AdaBoost**. We explain briefly how it works on a classification problem:

1. We start with equal weights $w_{i,1} = 1/n$ for all training samples $i = 1, \dots, n$ and train a first weak learner on them, we obtain $g_1(\mathbf{x})$.
2. We compute the classification error of $g_1(\mathbf{x})$ to obtain a note

$$\varepsilon_1 = \sum_{i=1}^n w_i^{(1)} \mathbb{1}(g_1(\mathbf{x}_i) \neq y_i)$$

where $w_i^{(1)} = w_{i,1} / \sum_{i=1}^n w_{i,1}$ are the weights normalized such that they sum to 1 (not necessary at this first iteration).

3. The aggregation weight of this first learner is given as a function of this note: $\alpha_1 = \ln \left(\frac{1-\varepsilon_1}{\varepsilon_1} \right)$
4. Before training the next weak learner, we update the weights

$$w_{i,2} = w_{i,1} \exp(\alpha_1 \mathbb{1}(g_1(\mathbf{x}_i) \neq y_i)),$$

meaning that we do not change the weights of the samples that were correctly classified, while those that were misclassified get an increased weight, this increase being larger if the note is good (large α_1).

5. We repeat the process for each iteration m with

$$\varepsilon_m = \sum_{i=1}^n w_i^{(m)} \mathbb{1}(g_m(\mathbf{x}_i) \neq y_i),$$

$$\alpha_m = \ln \left(\frac{1 - \varepsilon_m}{\varepsilon_m} \right)$$

and

$$w_{i,m+1} = w_{i,m} \exp(\alpha_m \mathbb{1}(g_m(\mathbf{x}_i) \neq y_i))$$

until $m = M$ the maximum number of iterations.

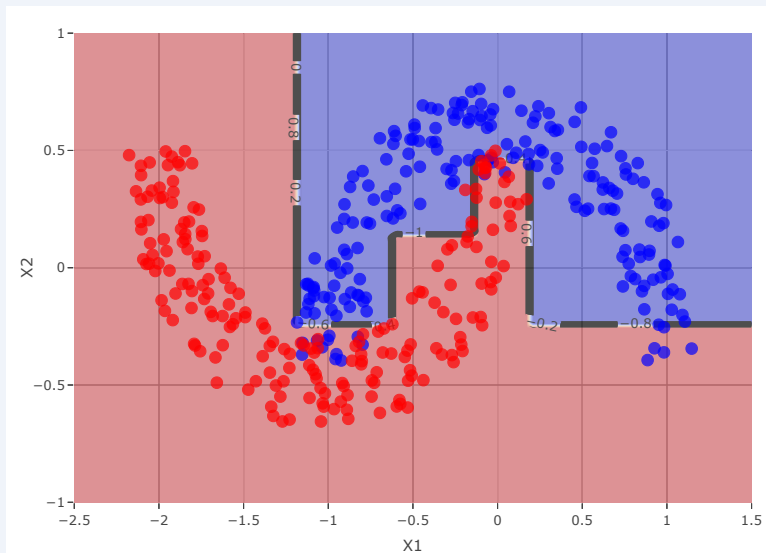
The final boosted classifier is given by

$$\hat{g}_M(\mathbf{x}) = \text{sign} \left(\sum_{m=1}^M \alpha_m g_m(\mathbf{x}) \right).$$

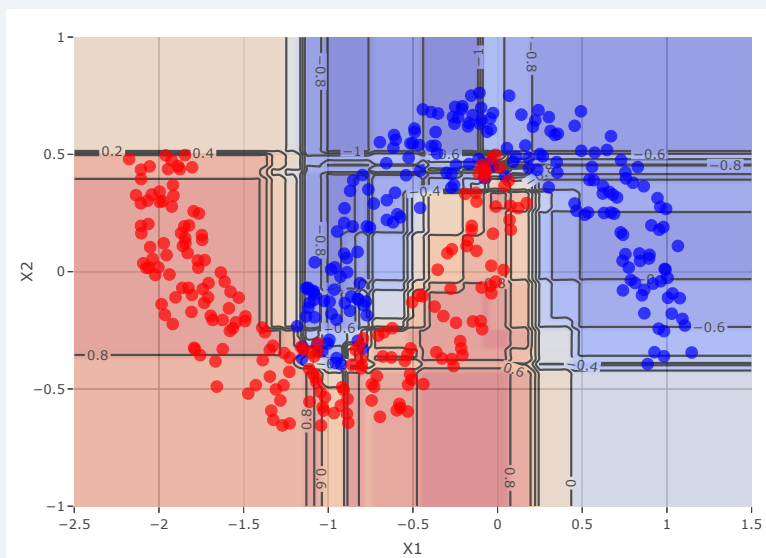
Note that other strategies exist for defining the weights α_m as a function of the errors ε_m . In practice, the weak learners are very often chosen as decision trees.

On the same example as before, we give below an illustration of AdaBoost at several iterations of the algorithm.

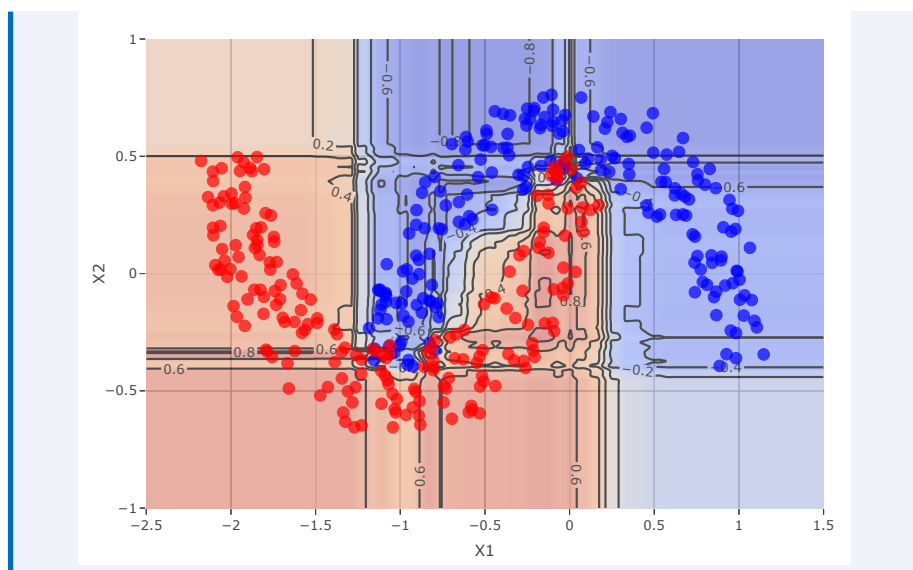
Example (Two Moons continued). At the first iteration of the algorithm, we just have a single tree.



After 10 iterations, the decision function is more complex and more predictive:



and here is the result after 100 iterations:



In the second category of Boosting methods relying on the residuals, **Gradient Boosting** is the most popular. Contrary to AdaBoost, here the weights of the training samples are not modified along the iterations of the procedure, but instead at each step the weak learners are built on different target functions (the residuals of the previous iterations).

Focusing on regression with the least-squares loss, the algorithm works as follows:

1. We seek an estimate $\hat{g}_M(\mathbf{x}) = \sum_{m=1}^M \alpha_m g_m(\mathbf{x})$,
2. At the first iteration, we use the simplest learner $g_1(\mathbf{x}) = \bar{y}$, the mean of the initial target values,
3. At each iteration $m = 2, \dots, M$, we have the $(m-1)$ -th aggregated model $\hat{g}_{m-1}$:
 - We compute the residuals $r_{i,m} = y_i - \hat{g}_{m-1}(\mathbf{x}_i)$,
 - The next weak learner $g_m(\mathbf{x})$ is built with the training sample $(\mathbf{x}_i, \mathbf{r}_{i,m})$, meaning that we build a model of the residual errors made by the previous models,
 - The m -th aggregated model $\hat{g}_m$ is defined as

$$\hat{g}_m(\mathbf{x}) = \hat{g}_{m-1}(\mathbf{x}) + \alpha_m g_m(\mathbf{x})$$

where α_m is optimized by solving

$$\min_{\alpha_m \in \mathbb{R}} \sum_{i=1}^n (y_i - \hat{g}_{m-1}(\mathbf{x}_i) - \alpha_m g_m(\mathbf{x}_i))^2.$$

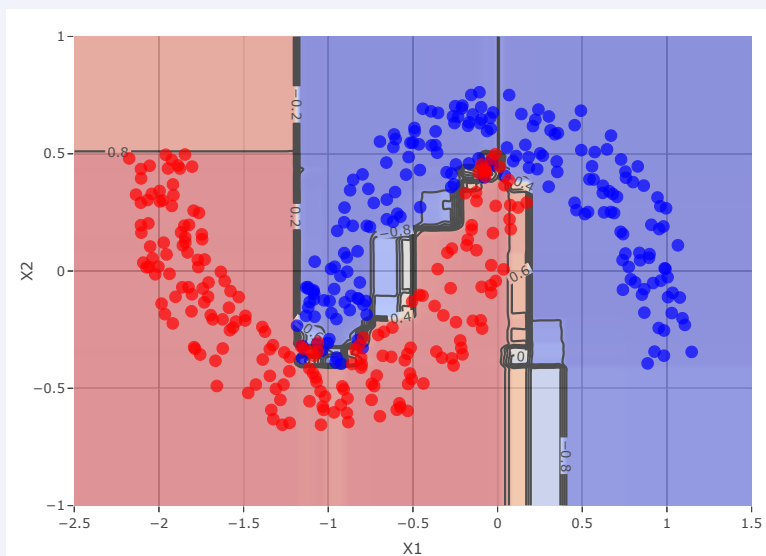
As with AdaBoost, the final boosted model is given by

$$\hat{g}_M(\mathbf{x}) = \sum_{m=1}^M \alpha_m g_m(\mathbf{x}).$$

The name *Gradient Boosting* may appear obscure by looking at the above description of the algorithm. In fact, we described it on the particular case of regression with the least-squares loss, which makes use of the true residuals. But in the general case, what the algorithm does is finding a descent direction of the loss function at iteration m , which is obviously given by the **gradient** of the loss function. The m -th weak learner is then just built on the training sample $\left(\mathbf{x}_i, \frac{\partial l(y_i, g(\mathbf{x}_i))}{\partial g(\mathbf{x}_i)} \Big|_{g=\hat{g}_{m-1}}\right)$, which happens to be equal to the residuals for the least-squares loss. This is the reason why it is called Gradient Boosting. Also note that Gradient Boosting is typically used with trees as weak learners just like AdaBoost.

Finally, among all Gradient Boosting algorithms, it is now common practice to use the highly popular variant called **XGBoost**, which achieves remarkable accuracy on many tabular data test cases, on par with and even better than random forests. In particular, XGBoost uses an improved descent direction based on second-order derivatives, an additional efficient penalization of the trees and parallelization, among other improvements. You will find below an example of XGBoost on the previous example.

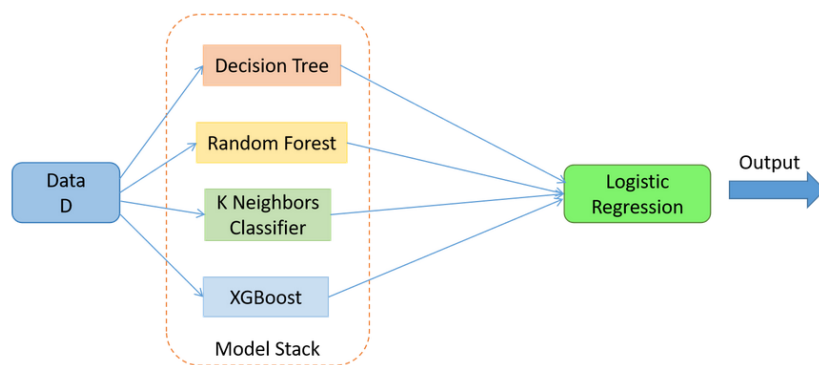
Example (Two Moons continued). Here are the results obtained by XGBoost with default parameters.



8.3 Stacking

Contrary to Bagging and Boosting and as mentioned in the introduction, *Stacking* does not combine many weak learners together, but rather tries to combine a small or moderate number of strong learners. Another difference is that for Bagging and Boosting the weak learners are of the same type (e.g. trees) but they are trained on different settings (different training samples or different

targets), while Stacking will rely on different types of strong learners (Random Forests, Kernel Ridge Regression, KNN, ...) but trained on the exact same task. Once the different strong learners have been built (they are then called *level 0 models*), Stacking simply proposes to write a new supervised learning problem, where the target is unchanged but now the **features are the predictions of the different strong learners**. In this new supervised learning problem, the strong learners are usually combined through a quite simple model, i.e. linear regression or logistic regression, being referred to as the *level 1 model*. Here is a graphical illustration with logistic regression taken from <https://www.analyticsvidhya.com/blog/2021/08/ensemble-stacking-for-machine-learning-and-deep-learning/>.



For regression, the raw predictions of the level 0 models are used, but for classification, the new features are typically not chosen as the raw predictions, but rather chosen as the probabilities of each class, since they are easier to handle in the level 1 model than classes (continuous vs discrete feature in the level 1 model).

In practice, it is even possible (and recommended by many) to include in the new supervised learning problem not only the predictions of the strong learners as new features, but also the initial features. This increases the dimensionality of the problem, but quite often it produces an even better model. When you want to make a prediction with this stacked model, the procedure is simple: you first compute the predictions of each level 0 models, and then pass them through the level 1 model.

Finally, in what we detailed above, level 0 and level 1 models are trained on the same sample, the training sample one. This may obviously lead to overfitting. To mitigate this effect, a variant of Stacking called *Blending* has been proposed: the training sample is first divided in two parts, the first one being used to train the level 0 models, which are then used to compute the predictions on the second part of the sample, where the level 1 model is finally trained.

Lecture 6: Practical ML

November 16, 2023

- Handling categorical variables
- High-dimensional problems
 - Feature selection
 - Dimension reduction via embedding
- Missing values
- Optimization of hyperparameters
- Visualization

Chapter 9

Practical machine learning

To conclude this course, we will discuss several specific aspects that you will most certainly encounter when using supervised learning methods in practical problems. The table below summarizes various characteristics of the methods introduced in this course, as well as methods covered in other 2nd and 3rd-year courses: handling categorical explanatory variables, training phase complexity (assuming $d < n$ below), model robustness with respect to variable normalization, handling missing values (NA), and predictive capacity.

Methods		Cat. Var.	Complexity Training	Normalization	NA	Capacity Predictive
Classification & Regression	2A SL	KNN	$O(d n)$	--	--	+ (distance choice)
		CART Tree	$O(d n \log(n))$	++	++	-
		Forests	$O(B \text{ mtry } n \log(n))$	++	++	++ (n large)
	3A	Bagging/Boosting	-	++	++	++ (n large)
Classification	2A SL	Naive Bayes	$O(d n)$	+	+	-
		LDA / QDA	$O(d^2 n)$	+	+	-
	2A	Logistic Regression	$O(d n)$	-	--	-
	3A	SVM	$O(d^2 n)$	--	--	++ (n small)
Regression	2A	Linear Regression	$O(d^2 n)$	-	--	-
	3A	Kernel Regression	$O(d n^2 + n^3)$	--	--	++ (n small)

Each of these points, along with other aspects, will be detailed in the following chapter.

9.1 Handling categorical features

9.1.1 Impact on the models

As summarized in the above table, the models covered at ENSAI vary in their sensitivity to categorical variables. Some of them handle categorical variables naturally, while others require special treatment. For example:

- CART trees, random forests, and Naive Bayes incorporate categorical variables without modification.
- LDA and QDA cannot be applied (even though the encoding strategy described later could be considered, it is generally not recommended for these methods).
- K-nearest neighbors either require a specific distance definition or an encoding strategy.
- SVM and kernel methods in general require a specific kernel or an encoding strategy.

In modern statistical learning, it is common to use a general strategy based on transforming categorical variables into continuous variables. Such a transformation is known as **encoding**.

9.1.2 General strategy: Encoding

Several methods for encoding categorical variables are commonly used, each with its own advantages and disadvantages.

1. **One-Hot Encoding:** This method is widely used for categorical variables. It creates a set of indicator variables (dummy variables) for each category. For example, consider the variable "Color" with categories "Red," "Green," "Blue". One-hot encoding will create three new variables: "Red," "Green," "Blue." Each observation will receive a 1 in the column corresponding to its color and zeros in the others. For example, if an element has the color "Green," it will be represented as (0, 1, 0).

Color	Red	Green	Blue
Red	1	0	0
Green	0	1	0
Blue	0	0	1
Green	0	1	0
Red	1	0	0

One-hot encoding is suitable when categories have no intrinsic order, and no order relationship should be imposed. However, it can significantly increase the dimensionality of the data. For linear regression and logistic regression, this is the encoding used (with a subtlety for the identifiability of coefficients, which was discussed in class).

2. **Frequency Encoding:** This method replaces each category with its frequency of occurrence in the dataset. For example, if "Red" appears 2 times, "Green" 2 times, and "Blue" 1 time in a dataset of size $n = 5$, the categories would be encoded as "Red": $2/5$, "Green": $2/5$, "Blue": $1/5$.

Color		Color
Red	→	0.4
Green		0.4
Blue		0.2
Green		0.4
Red		0.4

Frequency encoding can be useful when the frequency of each category is important for the task.

3. **Target Encoding:** This method, reserved for regression problems, uses the target values of the output to encode the categories. Each category is encoded based on the average of the target variable for that category.

Color	Y		Color
Red	2.3	→	1.7
Green	6.7		5.45
Blue	-2.67		-2.67
Green	4.2		5.45
Red	1.1		1.7

This transformation can help the model capture the relationships between categories and the target variable.

Remark (⚠ Encoding is done on the training sample). Any preliminary data transformation, when guided by the values of the dataset, must be estimated on the training sample. This is the case, for example, for frequency encoding or target encoding. Once the encoding is done and the model is trained, to make predictions on the test sample, you will need to replace the categorical variables with the estimated encoded value from the training sample. This implies that you need to be vigilant to ensure that the categories present in the training and test samples are the same after separation; otherwise, encoding will be impossible for the test sample (this comment also applies to one-hot encoding).

Remark (⚠ The type of encoding is a hyperparameter!). If, in practice, you want to test different types of encoding and choose the most suitable one

for your problem, encoding becomes, by default, a hyperparameter, and you will need to use cross-validation to identify the best one.

You may also come across two other types of encoding: label encoding, which replaces each category with a number between 1 and the total number of categories, and binary encoding, which represents each category with a binary number obtained by concatenating the values of one-hot encoding (for example, 01000). However, these two types of encoding artificially introduce an order and distances between categories, so they are to be avoided in practice.

9.1.3 Handling rare categories

Handling rare categories is an essential aspect of preprocessing categorical variables. Rare categories occur when some categories have very few occurrences in the dataset, which can be problematic for supervised learning models. In such cases, it is often necessary to consolidate these rare categories to avoid overfitting.

To address this problem, you can define a minimum frequency threshold below which categories are considered rare. Categories that do not meet this criterion can be grouped into a common category, for example, "Other." This consolidation reduces the dimension of the data and helps prevent model bias toward the majority categories.

9.2 Large datasets

Currently, in many fields, the increase in all-around data collection leads to the need to handle datasets with a large number of individuals, that is, a very large n . In these situations, it is essential to keep in mind that all supervised learning algorithms have potentially very different complexities, depending on both n and d , the number of explanatory variables (some complexities are provided in the summary table at the beginning of the chapter).

This means that you may not be able to use some of them in practice, or at least not in their original version (there are possible adaptations, outside of this class).

9.3 Handling high dimensionality of explanatory variables

9.3.1 Motivation and context

In many cases, you will work with datasets that have a large number of features, sometimes well beyond what is necessary to perform a supervised learning task. Dimensionality reduction comes into play to help manage this complexity:

- One of the main motivations for dimensionality reduction is model simplification. Supervised learning models, whether they are neural networks, decision trees, or other methods, can quickly become too complex and overfit when the number of features is high.

- Another essential aspect of dimensionality reduction is its role in improving model performance. By eliminating redundant or unnecessary features, this technique highlights the discriminative features that contribute most to the supervised learning task. This can have a significant impact on the accuracy and efficiency of the models.
- Furthermore, an additional point is that dimensionality reduction allows for easier data visualization. By reducing the dimension to two or three dimensions, it becomes possible to represent data in a graphical space, greatly facilitating the understanding of relationships between data points.

In practice, there are two main families for dimensionality reduction:

1. **Feature Selection:** This is a supervised learning-specific method that aims to select, from the initial explanatory variables, a smaller subset composed only of explanatory variables that best explain the target variable. In the end, the model is trained with the original explanatory variables, but only with some of them.
2. **Embedding:** Embedding methods are not specific to supervised learning and initially emerged in unsupervised learning. Here, unlike feature selection, the idea is to construct new explanatory variables obtained by transforming the original variables, and in a small number. Intuitively, the goal is to identify a transformation that "compresses" the information of the explanatory variables into a new representation of small size, and this operation is then called embedding (we "embed" the explanatory variables into a reduced-dimensional space).

In each of these families, there is a clear division between two classes of methods: those based on linear operations and those based on non-linear operations.

9.3.2 Feature selection

As mentioned earlier, variable selection aims to identify, among all the explanatory variables, those with the most predictive power for predicting the target. Variable selection methods fall into two categories:

1. **Agnostic methods:** These are methods, also known as preliminary variable selection methods, that solely use the dataset to estimate a measure of the relationship between each explanatory variable and the target variable and select only those with significant strength. The supervised learning model is then trained in a second phase with the selected variables. In this class of methods, you will find, for example, correlation to measure the strength of the relationship between a continuous explanatory variable and a continuous target variable (if one explanatory variable is categorical and the target variable is continuous, you can use a chi-square test, and if both are categorical, use a contingency table). The drawback of correlation is that it considers only potential linear relationships between variables, and more recently, effective nonlinear methods have been proposed in its place: mutual information, distance correlation, or HSIC.

2. **Model-based methods:** Unlike agnostic methods, other methods combine the construction of a supervised learning model with variable selection, using a coupling procedure (strong or weak). For example, you have seen forward/backward selection of explanatory variables in a linear regression model: this selection is entirely specific to the chosen model. A similar alternative for the linear model, more recent and very popular, is the Lasso introduced in this course. But beware, this does not mean that an unselected variable is not influential: it just means that it is not useful for the **specific model considered**. For example, a variable with a quadratic influence will never be detected with a linear model (as it would not have been with an agnostic method based on correlation), see also the remark below. It is worth noting that the idea of selection generalizes to other types of models, but it is less popular than in the context of linear or logistic regression.

Another approach, used for trees and forests, is recursive feature elimination, which is based on the ability of trees and forests to estimate the importance of explanatory variables after their construction (using measures like MDI or MDA, see the corresponding chapter). The idea is to start by building a model with all the variables, calculate the importance of each one with this model, then remove the one with the least influence and repeat the process.

Remark ( [Don't jump to conclusions too quickly](#)). When an explanatory variable is not selected during a model-coupled selection, it is important to reiterate that this does not mean it has no influence on the target variable. Beyond the example of a variable with a quadratic effect that would not be detected by a linear model, there is a more subtle but very common case: that of a variable that one could detect as influential but has no usefulness in improving the model. This happens when the model has already selected, in a previous iteration, an explanatory variable that is highly influential on the target but strongly dependent on the unselected variable. Thus, the new variable provides redundant information for explaining the target, and the model does not need to use it for making accurate predictions. This example is a common pitfall to remember.

9.3.3 Projection / Embedding

The other approach to dimension reduction, known as embedding, aims to construct a small number of new explanatory variables. There are three major classes of methods:

1. **Unsupervised methods:** These methods use only the explanatory variables to identify a transformation, which generally takes the form of a compression seeking to preserve the observed variability of the explanatory variables but with a reduced number of new variables. In this family, you are particularly familiar with Principal Component Analysis (PCA), which seeks new variables in the form of linear combinations of the original variables. A direct nonlinear generalization based on kernel methods,

called Kernel PCA, uses the kernel trick detailed earlier in this course. More recently, two methods have emerged and become extremely popular in data science in recent years: the **t-SNE** algorithm and the **UMAP** algorithm (which you have had an illustration of earlier in this course). These two algorithms are based on the common idea of finding a transformation in which the neighbors of an individual once embedded in the new space are close to their neighbors in the initial space. Initially developed for dimension reduction for visualization purposes (embedding in two or three dimensions at most), they are also used more generally upstream of supervised learning models.

For more exotic explanatory variables (see Bonus Section 1 at the very end), specific methods based on neural networks have been proposed, especially for images, such as autoencoders.

2. **Agnostic supervised methods:** For some of the previous methods, extensions have been proposed to guide the transformation with the values of the target variable (typically by generalizing the concept of neighbors in the space of explanatory variables, by adding the target value). A famous case is the UMAP method (available only in the Python library, not yet in R).
3. **Model-based supervised methods:** Finally, some supervised learning models are defined with an explicit embedding in their formulation, which is estimated simultaneously with the model's construction (either iteratively or not). The historical example, which combines PCA and a linear model, is Partial Least Squares (PLS). It seeks to identify linear combinations of the explanatory variables that linearly explain the target variable. This principle has been generalized for nonlinear models, but always with linear embedding inside.

Once again, for more exotic explanatory variables (see Bonus Section 1 at the very end), specific neural network architectures (e.g., U-net) consist of an initial embedding part, the construction parameters of which are estimated during the training phase.

Remark ( No useful embedding if explanatory variables are independent in unsupervised methods). If you use an unsupervised embedding method, keep in mind that the dimension of the new space will be reduced only if the original explanatory variables are dependent, meaning they actually live in a space of smaller dimension than d . If not, it is evident that it is impossible to compress the information contained in the dataset!

Remark ( Embedding is useful even without reducing dimension). Originally, embedding is, by definition, useful to find a lower-dimensional space for the explanatory variables. However, if you use a nonlinear method and keep the same dimension, you can potentially find new variables that simplify the supervised learning problem. You can use the last example in

Bonus Section 2 to form an opinion on this matter (in case all individuals were labeled in this example).

Remark ($\triangle$ Embedding is useful in unsupervised learning as well). We have discussed the usefulness of embedding in supervised learning, but unsupervised methods are also popular as preprocessing for unsupervised learning, such as clustering or density estimation. For example, using a nonlinear method, it is possible that in the transformed space, a k-means algorithm will perform better (once again, you can look at the last example in Bonus Section 2, imagining that the problem was unsupervised clustering). Moreover, reducing dimension potentially simplifies the density estimation problem. A recent popular example is generative neural networks (GANs)!

9.4 Normalization and preprocessing of variables

The normalization of variables is a crucial step in data preprocessing for supervised learning. Variables in a dataset can have very different scales, which can significantly impact some supervised learning algorithms. When variables are not normalized, supervised learning models may inadvertently assign more weight to features with larger scales, which can bias the results. For example, an algorithm like linear regression is sensitive to variable scaling, so variables with different scales can bias the model coefficients. Normalization aims to place all variables on the same scale (usually between 0 and 1 or with zero mean and unit variance) so that each variable contributes fairly to the model's construction. This facilitates comparing the importance of different variables and improves the convergence of many optimization algorithms. Additionally, it can help prevent the dominance of one variable over others in distance-based methods, such as k-nearest neighbors.

9.4.1 Impact on models

For the algorithms covered at ENSAI, normalization is particularly critical for k-nearest neighbors (KNN) because of the use of variable distance (more generally, this will be the case for all methods using distance). This is the same for methods based on kernels, such as SVM or kernel ridge regression. It is therefore recommended to systematically normalize the data for KNN. Normalization can also have a slight impact on LDA/QDA and Naive Bayes but only from a numerical stability perspective (for covariance matrix estimation and density estimation, respectively). Finally, CART trees and random forests are insensitive to any monotone transformations of the explanatory variables and theoretically do not require normalization. However, the general advice is to systematically normalize the variables at the beginning of the study because, in practice, it will be common to train and compare several models, some of which are sensitive to scaling.

9.4.2 Transformations

Before normalizing the variables, it is essential to determine if a preliminary transformation is necessary. The need for transformation can be detected in several ways. Firstly, a simple graph of the variable values can reveal asymmetric or heteroscedastic distributions. Data that does not follow a normal distribution or has extreme values may benefit from transformations to make them closer to a normal distribution. A histogram, a quantile-quantile (Q-Q) plot, or a normality test like the Shapiro-Wilk test can be used for this purpose.

Next, examining the relationships between variables and the target variable is essential. Transformations may be necessary if the relationship between an explanatory variable and the target variable is nonlinear. For example, if a quadratic or logarithmic relationship is present, a transformation can be used to linearize the relationship. Commonly used transformations include the logarithmic transformation, square root, reciprocal, or even more complex transformations like Box-Cox or Yeo-Johnson, which adapt the transformation based on the data distribution. However, it is essential to note that preliminary transformations must be applied with caution as they can influence the results and model interpretation. It is recommended to test different transformations and choose the one that best improves the model's quality without introducing unwanted distortions.

Remark ( Transformations are applied to the training sample). As with encoding, the identification of a possible transformation is done exclusively on the training sample.

9.5 Handling missing data

9.5.1 General overview

Definition of missing values. Missing values are a ubiquitous element in many datasets. They simply refer to the absence of information for certain observations or variables. This can occur for various reasons, such as data collection errors, respondent non-responses, or storage issues. Effective management of missing values is essential because they can significantly distort the results of supervised learning if not handled correctly.

Missing values are not uniform and can manifest in different ways. A deep understanding of their nature is crucial for choosing the best management strategy. In this context, we distinguish three main types of missing values. The first is MCAR (Missing Completely At Random), where missing values occur randomly and independently of other data. The second is MAR (Missing At Random), where missing values depend on other observed variables. Finally, MNAR (Missing Not At Random) concerns missing values that depend on unobserved information. This distinction is fundamental in determining appropriate methods and assessing the impact on results.

Impact on models. Among the algorithms covered at ENSAI, missing values have a relatively limited impact (provided they are not too numerous) for:

- For trees and forests, thanks to the principle of surrogate splits
- For LDA/QDA and Naive Bayes, density estimates are only slightly disturbed

For KNN, distance calculations are affected, and the algorithm cannot be used without modification, as for SVM and kernel ridge regression. That is why we discuss common workarounds below.

9.5.2 Handling missing values

Deletion of rows or columns. When dealing with missing values in a dataset, one of the simplest methods to handle them is deletion. There are two main approaches to this method: row deletion and column deletion. Row deletion involves removing individuals containing missing values. This approach can be effective if missing values are relatively few compared to the total size of the dataset. However, it can lead to a significant loss of information. On the other hand, column deletion involves eliminating variables (or features) that have missing values. This can be appropriate when some features are not essential for the current supervised learning task. Nevertheless, this approach is potentially risky and can have consequences on prediction quality. Here is an example of row deletion:

X^1	X^2	X^3		X^1	X^2	X^3
A	3.42		→	B	2.4	-11.02
A	5.24			C	-1.1	8.57
	-6.57	3.29				
B	2.4	-11.02				
C	-1.1	8.57				
	2.5	-4.02				

and Here is an example of column deletion:

X^1	X^2	X^3		X^2
A	3.42		→	3.42
A	5.24			5.24
	-6.57	3.29		-6.57
B	2.4	-11.02		2.4
C	-1.1	8.57		-1.1
	2.5	-4.02		2.5

You can see that these are risky strategies that potentially result in significant information loss.

Imputation of missing values. Imputation of missing values is a more sophisticated approach aimed at replacing missing values with estimated values. Several imputation methods exist, including some simple ones like mean, median, or mode imputation (of the column containing the missing value). These

methods are suitable when missing data is MCAR or MAR. However, for MNAR data, more advanced techniques like regression or classification imputation may be necessary (a regression model is trained to predict an explanatory variable based on the others, and the prediction of this model is used for imputation), but obviously, this only works if there is a dependence between the explanatory variables. A more modern version of this principle is based on matrix factorization. Finally, multiple imputation by the MICE method (Multiple Imputation by Chained Equations) is also widely used to handle missing values in a more robust and realistic way. However, the choice of the imputation method depends on the nature of the data, sample size, and the analysis objective. In the previous example, here is what mean imputation (for continuous explanatory variables) and most frequent category imputation (for categorical explanatory variables) would look like:

X^1	X^2	X^3		X^1	X^2	X^3
A	3.42		→	A	3.42	-0.795
A	5.24			A	5.24	-0.795
	-6.57	3.29		A	-6.57	3.29
B	2.4	-11.02		B	2.4	-11.02
C	-1.1	8.57		C	-1.1	8.57
	2.5	-4.02		A	2.5	-4.02

Remark (⚠ Imputation is done on the training sample). Like encoding and variable transformations, imputation is exclusively estimated on the training sample.

Remark (⚠ The type of imputation is a hyperparameter!). In practice, if you want to test different types of imputation and choose the most suitable one for your problem, imputation becomes a hyperparameter, and you will need to use cross-validation to identify the best one.

Encoding missing values. Another approach to handle missing values, only in the case of a categorical explanatory variable, is to encode missing data as a distinct special category. This means that missing values are treated as separate information rather than being replaced by an estimate. This method can be particularly useful when the reason for missing values is significant or when the absence of data is itself an indicator of information. However, it is essential to choose this option carefully because it can influence how supervised learning models interpret the data. Encoding missing values is commonly used in fields such as biology, where missing values can have specific biological meaning. In the previous example, we illustrate mean imputation (for continuous explanatory variables) and encoding with a new category "M" for "Missing" (for categorical explanatory variables):

X^1	X^2	X^3		X^1	X^2	X^3
A	3.42			A	3.42	-0.795
A	5.24			A	5.24	-0.795
	-6.57	3.29	→	M	-6.57	3.29
B	2.4	-11.02		B	2.4	-11.02
C	-1.1	8.57		C	-1.1	8.57
	2.5	-4.02		M	2.5	-4.02

9.6 Hyperparameter optimization

You now know that to optimize the hyperparameters of a supervised learning model, cross-validation is frequently used. In the libraries you have used, the procedure is generally as follows:

1. Choose the hyperparameters to vary and the values to test for each of them.
2. Build a grid containing all possible combinations of these values.
3. For each combination, compute the error through cross-validation.
4. Select the combination with the lowest error.

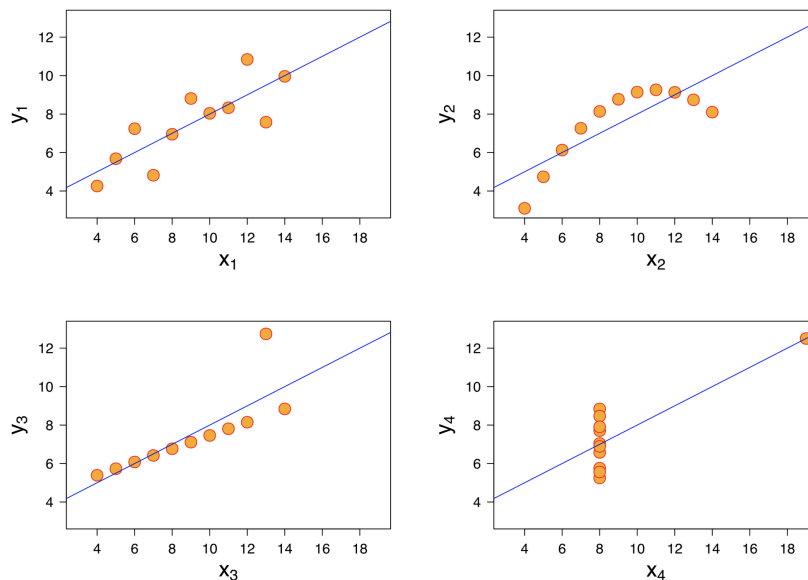
On real problems, possibly of large size, step 3, computing the error, can be particularly time-consuming (you may have experienced this yourself with certain methods). In cases where the number of combinations to test is large, this procedure can be very time-consuming and even become so long that it becomes unusable. A more modern approach, becoming increasingly popular, is to replace the construction of a combination grid with an optimization algorithm, which takes the hyperparameters as variables to optimize and provides bounds on each of them. The objective function minimized by the optimizer is the cross-validation error. In practice, the number of times the optimizer will request the computation of the objective function (which, it should be remembered, can be expensive) is often less than the total number of calculations that would have been done with a grid. Finally, it is also increasingly common to use a specific and adapted optimization strategy for cases of expensive objective functions, further reducing the computation time of the procedure.

9.7 Visualize, Visualize, Visualize

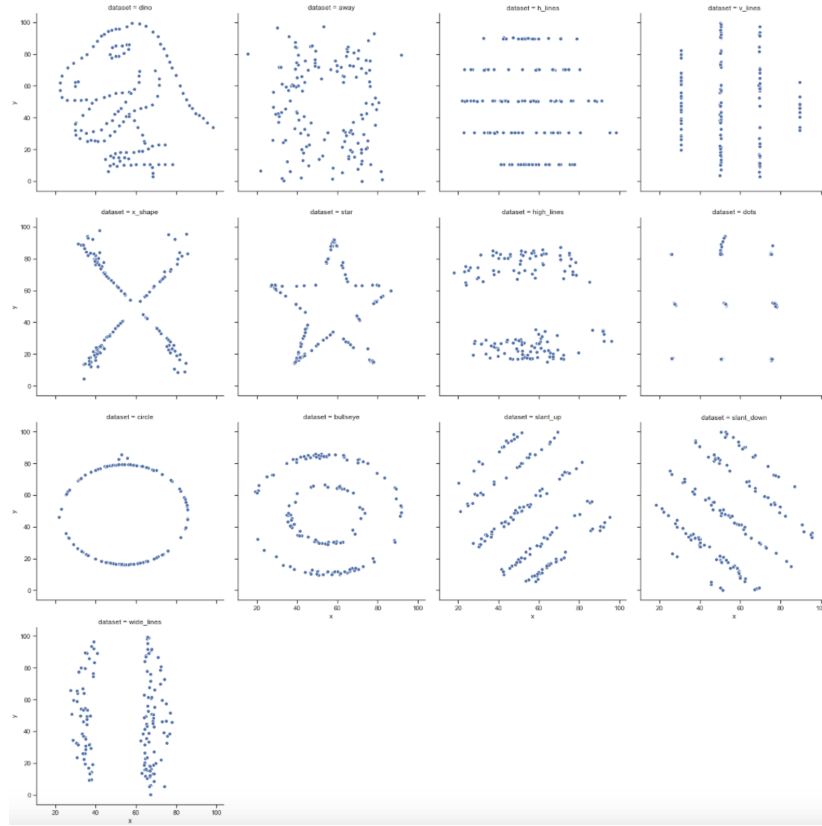
It cannot be emphasized enough that the tools of descriptive analysis you learned in your first year, especially visualizations, remain invaluable in supervised learning. It is highly recommended to systematically implement them. Visualizations are often overlooked in favor of quantitative indicators (averages, variances, correlations, R^2 , etc.), but they offer several significant advantages:

1. To identify potential variable transformations, as discussed earlier, it is often helpful to plot histograms of all explanatory variables and the target variable. You can find some examples here: <https://asq.org/quality-resources/histogram>.

2. For identifying transformations and exploring whether certain explanatory variables have more influence on the target than others, scatterplots of the target (on the y-axis) versus an explanatory variable (on the x-axis) are often informative.
3. **Quantitative indicators can be highly misleading** because they naturally aggregate information from a dataset, providing a coarse summary of the phenomenon. A well-known example is Anscombe's quartet, which consists of four datasets with identical standard quantitative indicators for the variables: the same mean, variance, correlation, and R^2 . If one were to rely solely on this view, it might be concluded that these four datasets represent similar phenomena. However, by simply visualizing the corresponding scatterplots, the true nature of the datasets becomes apparent:



It is evident from these plots that, in reality, the datasets correspond to very different practical situations: 1) a linear relationship with noise in the target variable, 2) a nonlinear relationship, 3) a linear relationship with very little noise but an outlier in the target variable, 4) noisy measurements repeated for a single value of the explanatory variable, with outliers in both the explanatory and target variables. These visualizations allow you to better understand the underlying phenomena in terms of relationships, presence of outliers, and more. To further emphasize this point, another example similar to Anscombe's quartet, with even more surprising variations but identical quantitative indicators, is the Datasaurus Dozen:



9.8 Bonus 1: Handling more "exotic" explanatory variables

Throughout the course, we have focused on a "classic" framework of supervised learning, where explanatory variables live in a space $\mathcal{X}$ contained in $\mathbb{R}^d$, meaning they are given in scalar values. This is the same framework you worked with in the clustering course in your first year. However, in practice, an increasing number of learning tasks (supervised or not) involve data that are not scalars. Common examples include images (image classification, image clustering, image segmentation, semantic image segmentation, super-resolution, completion, generation, etc.), as well as text (text classification, text clustering, generation, etc.). Other data types include audio, video, graphs, molecules, time series, and more.

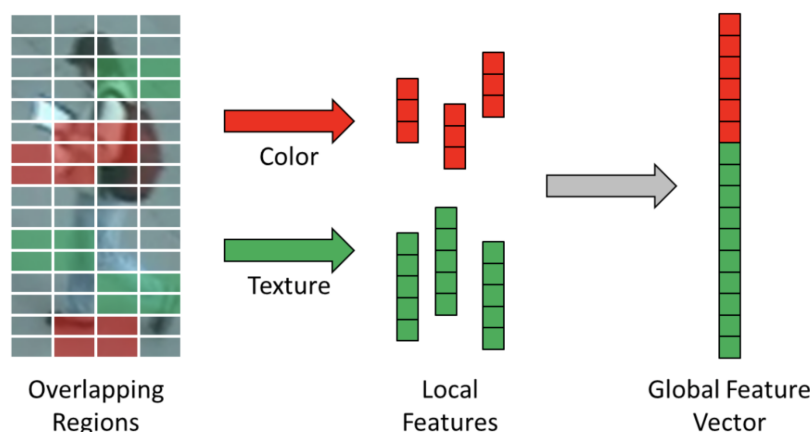
A full course would be needed to cover all these aspects. Here, we will focus on a few of them. Specifically, we will discuss the general principle of embedding, which involves constructing a vector representation (i.e., multiple scalar explanatory variables) for "exotic" objects. This embedding transforms all data into $\mathbb{R}^d$, allowing the use of standard algorithms on this new representation. This is a generalization of the embedding principle discussed in the dimensionality reduction section.

As mentioned earlier, embedding is based on identifying a vector represen-

tation of an object, which is then used for the learning task. The quality of the final learning result depends heavily on the quality of this representation. For example, for an image classification problem, it is possible to represent each image with a single value, such as the average pixel intensity. However, this representation is not expressive and does not capture the intrinsic properties of each image. As a result, this variable is unlikely to explain why an image belongs to one category over another.

Feature extractors (Unsupervised) Historically, the use of feature extractors to construct embeddings was popular and known as feature extraction. The principle is simple: each object is represented by a set of descriptors specific to the object type. These descriptors are summaries of the object guided by domain knowledge. Examples include:

- For images, local or global summaries of colors, textures, or edges.

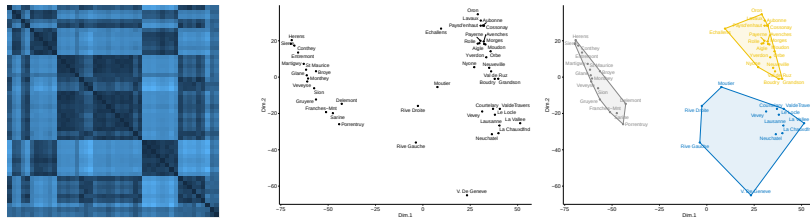


- For text, summaries based on word frequency in a dictionary (generalized with the famous "tf-idf") or more generally, n-grams.
- For molecules, counts of atoms, distances between them, or representations in string form (which turns them into text).
- For graphs, summaries of connectivity, eigenvectors from spectral analysis, and more.

This approach was highly effective for a long time, until deep neural networks, particularly those based on supervision, became prominent.

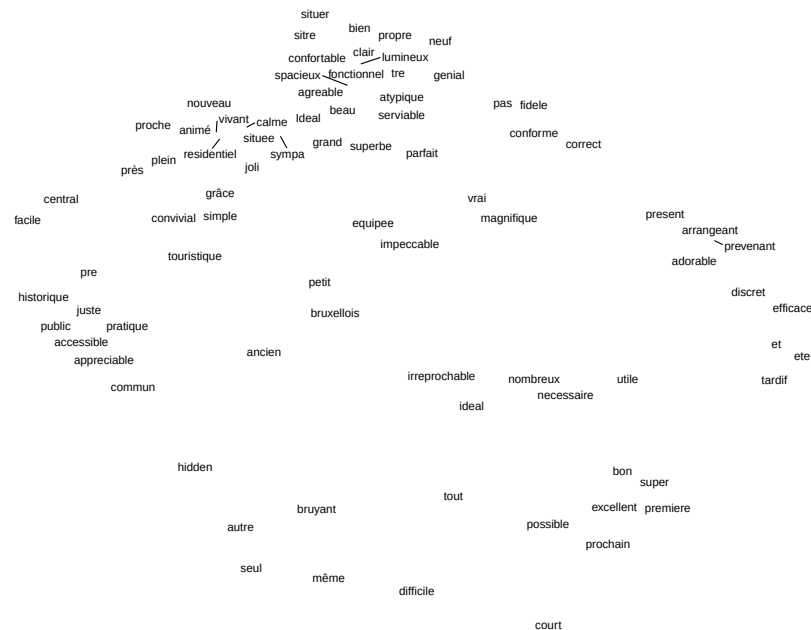
When an appropriate distance is available (Unsupervised) For some data types, it may not be feasible to define feature-based representations. However, a distance between objects can still be defined based on domain-specific knowledge. For instance, for segmented images with regions of specific types, a Hausdorff distance (i.e., between sets) can be constructed between regions of the same type. For molecules, distances based on physical criteria between different atom groups can be used. Although these distances can be directly used in

algorithms like KNN (and with precautions with kernel methods), k-means, or hierarchical clustering, this is not the case for most other algorithms you have learned. There is, however, a method that allows you to find a vector representation in $\mathbb{R}^d$ of each object based on a given distance, so that the Euclidean distance in this representation space is close to the initial distance. This final representation serves as embedding to transform all objects, which can be used in any learning task. This method is called Multidimensional Scaling (MDS), as illustrated below.



Of course, if the initial distance defined is not suitable for the learning task, the representation obtained through MDS will also be inappropriate and yield poor results.

Neural Networks (Supervised or Unsupervised) For data types like images or text, recently proposed deep neural network architectures have shown impressive efficiency in supervised learning tasks. Particularly, high-performing architectures often rely on the initial part of the network, which constructs an embedding of data through filters adapted to the data type (e.g., convolutional layers for images). Such networks have already been trained on classification or regression tasks with large image or text datasets, and they are readily available. You can use only the first part of the network that performs embedding to transform your own data into a vector representation and use this vector in any algorithm. This is commonly known as transfer learning. The example below shows an embedding obtained with a "word2vec" neural network for textual data from Airbnb reviews in Brussels, visualizing a two-dimensional vector representation of adjectives.

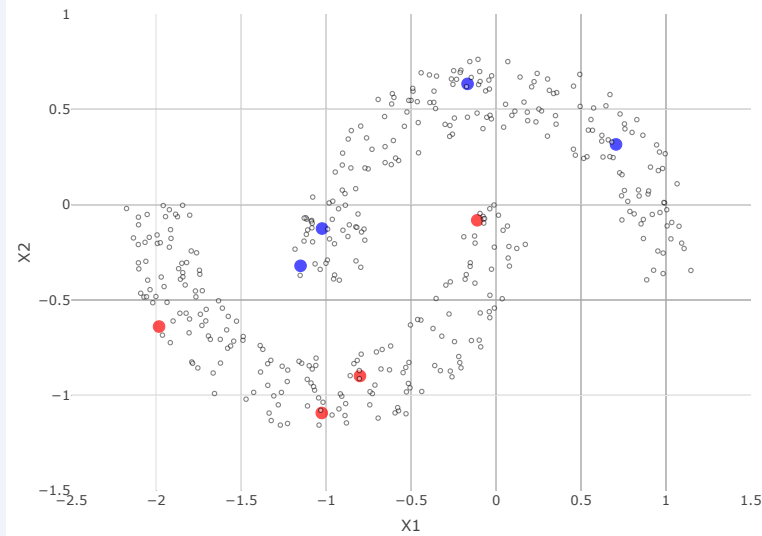


A particularly useful resource to access various pre-trained neural networks for different tasks and data types is available at <https://huggingface.co/models>. There are also unsupervised deep networks trained on images, such as autoencoders or variational autoencoders.

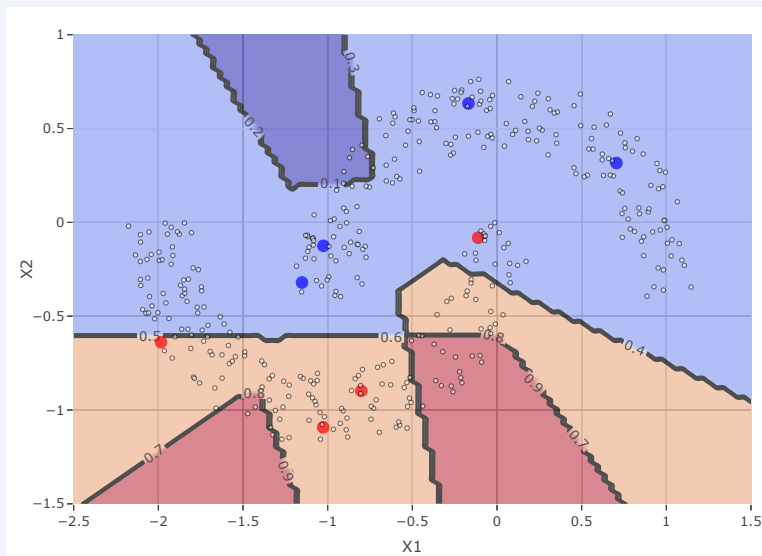
9.9 Bonus 2: Semi-supervised learning

Semi-supervised learning is an approach that falls between supervised learning and unsupervised learning. In this scenario, you have access to both labeled data (supervised learning, pairs $\{\mathbf{x}_i, y_i\}_{i=1}^n$) and unlabeled data (unsupervised learning, observations $\{\mathbf{x}_j\}_{j=1}^N$). Labeled data is typically scarce and expensive to obtain, while unlabeled data is more abundant ($N \gg n$). Here is an example:

Example (Semi-supervised learning on the "Two Moons" dataset). Imagine that, for the "Two Moons" dataset we have seen several times in this course, it is very expensive (in terms of time or money) to obtain the true label for each individual, and you have only managed to do so for a few of them. However, obtaining the features for each of them is relatively inexpensive. This leads to the following situation.



The individuals in red and blue are those for which you have the true labels (pairs $\{\mathbf{x}_i, y_i\}_{i=1}^n$), but for all others, you only have their features ($\{\mathbf{x}_j\}_{j=1}^N$). You can use a KNN with $k = 3$ on the labeled data only, which results in the following:

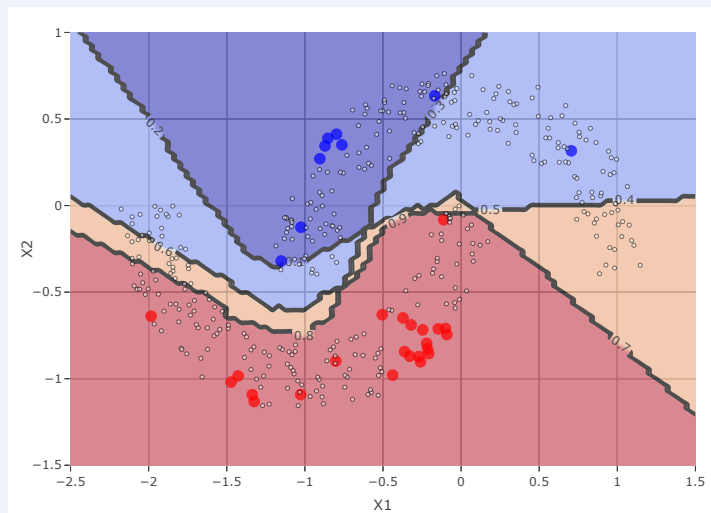


But, of course, you would like to use the unlabeled data to guide the classifier's construction and improve its quality.

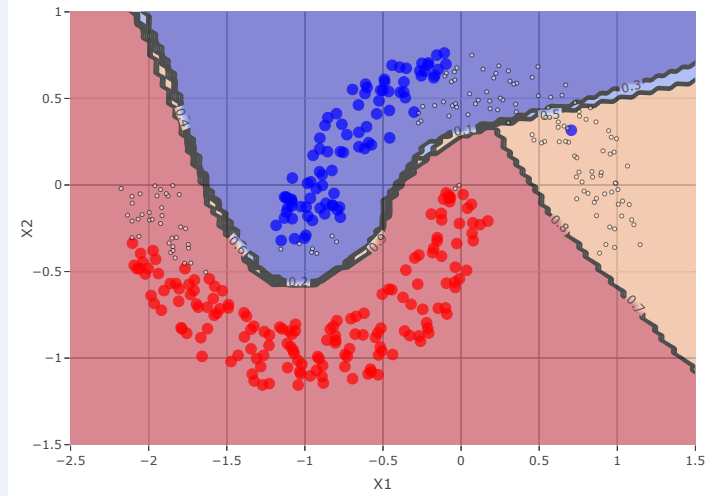
The goal of semi-supervised learning is to leverage both labeled and unlabeled data to build a higher quality model (compared to one using only labeled data). Several approaches are possible, and here we mention the three most commonly used methods in practice:

1. **Self-training.** This approach involves using an initial model to assign *virtual* labels (predicted by the model) to unlabeled data. These new virtual labels are then added to the labeled data as if they were real observations, and the model is retrained with this augmented dataset. Self-training can be iterative, with the model continually improving by adding new labels to unlabeled data. In practice, it is common to add only individuals for which labels are predicted with the highest confidence in each iteration.

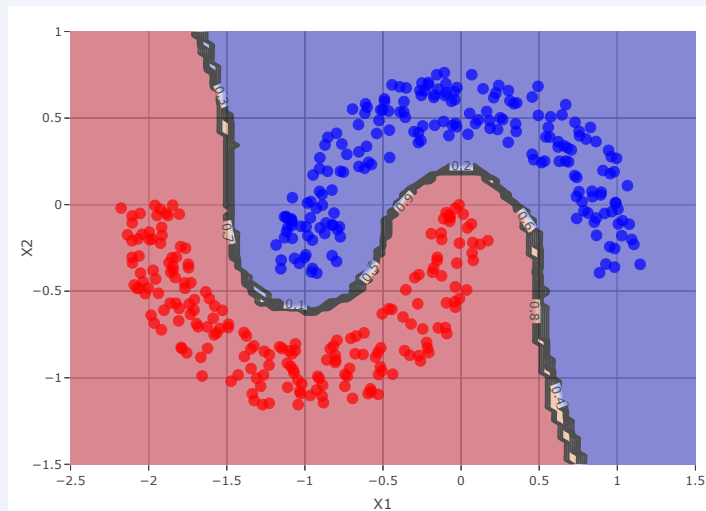
Example (Self-training on the "Two Moons" dataset). Here is what this method looks like with the KNN classifier with $k = 3$, by adding unlabeled individuals for which the KNN has confidence greater than 0.95. After 1 iteration:



After 2 iterations:



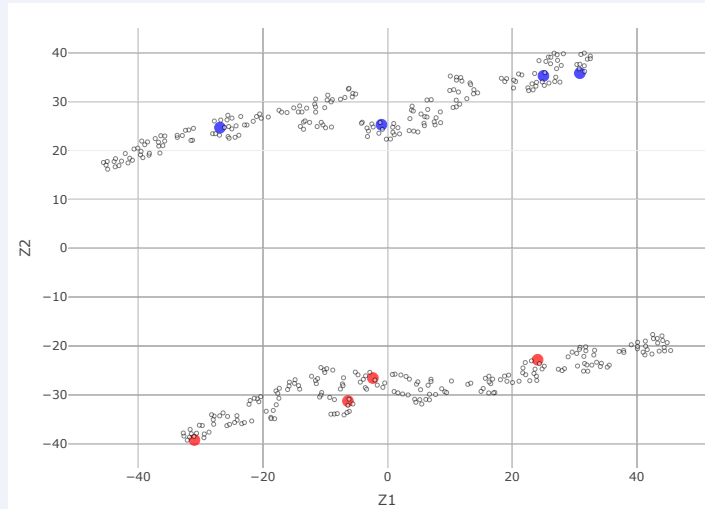
After 5 iterations:



Although self-training works particularly well in this case, it is important to keep in mind that this approach can be potentially risky. If the initial classifier is overly confident but incorrect, virtual labels will be erroneous, and the model will be trained on entirely false data.

2. **Representation Learning.** Another heuristic yet natural approach is to use the explanatory variables of all individuals, both labeled and unlabeled, $\{\mathbf{x}_i\}_{i=1}^n \cup \{\mathbf{x}_j\}_{j=1}^N$, to learn a representation. You can think of it as a dimensionality reduction method, as discussed in this chapter. Once this transformation $\mathbf{x} \mapsto \mathbf{z}$ is constructed, a supervised learning model is trained on individuals $\{\mathbf{z}_i, y_i\}_{i=1}^n$.

Example (Representation learning on the "Two Moons" dataset). Using the t-SNE algorithm (mentioned in this chapter) on the explanatory variables of all individuals, you obtain the following representation, where individuals with observed labels are colored in red and blue.



The algorithm has learned the underlying structure of the data well, and in this new space, a simple classifier will make few errors.

3. **Regularization.** Finally, an approach more closely related to supervised learning involves formulating the problem as an empirical risk minimization problem (on labeled data) with a penalty on the obtained classifier. The penalty enforces certain structure on unlabeled data, for example, by limiting the classifier's variations. In this category of methods, you can find manifold regularization or separation of regions with low density. This is popular for SVM (transductive support vector machine, and Laplacian regularization in a RKHS).